

AI Plasmid Design Platform

Phase R: Research and Knowledge Acquisition Report

May 29, 2026

Contents

1	Executive Summary	2
1.0.1	Required Answers From SYSTEM_DESIGN Section 4.5.2	2
1.0.2	Build Decisions To Carry Forward	5
2	Track A: Plasmid Biology & Structure	7
2.0.1	Scope	7
2.0.2	What the literature/sources establish	7
2.0.3	What this means for our build	9
2.0.4	Decisions proposed	10
2.0.5	Open questions / conflicts	11
2.0.6	Sources	11
3	Track B: Design & Validity Rules	13
3.0.1	Scope	13
3.0.2	What the literature/sources establish	13
3.0.3	What this means for our build	17
3.0.4	Decisions proposed	18
3.0.5	Open questions / conflicts	20
3.0.6	Sources	21
4	Track C: DNA Language Models & Generation	23
4.0.1	Scope	23
4.0.2	What the literature/sources establish	23
4.0.3	What this means for our build	27
4.0.4	Decisions proposed	28
4.0.5	Open questions / conflicts	29
4.0.6	Sources	29
5	Track D: Prior Art & Competing/Related Tools	32
5.0.1	Scope	32
5.0.2	What the literature/sources establish	32
5.0.3	What this means for our build	39
5.0.4	Decisions proposed	40
5.0.5	Open questions / conflicts	40
5.0.6	Sources	41

6	Track E: Data Sources & Access	43
6.0.1	Scope	43
6.0.2	What the literature/sources establish	43
6.0.3	What this means for our build	47
6.0.4	Decisions proposed	49
6.0.5	Open questions / conflicts	50
6.0.6	Sources	50
7	Track F: Sequence Representation & Tokenization	52
7.0.1	Scope	52
7.0.2	What the literature/sources establish	52
7.0.3	What this means for our build	54
7.0.4	Decisions proposed	56
7.0.5	Open questions / conflicts	57
7.0.6	Sources	58
8	Track G: Validation Tooling	60
8.0.1	Scope	60
8.0.2	What the literature/sources establish	60
8.0.3	What this means for our build	62
8.0.4	Decisions proposed	64
8.0.5	Open questions / conflicts	65
8.0.6	Sources	66
9	Track H: Visualization	68
9.0.1	Scope	68
9.0.2	What the literature/sources establish	68
9.0.3	What this means for our build	70
9.0.4	Decisions proposed	71
9.0.5	Open questions / conflicts	72
9.0.6	Sources	73
10	Track I: System Architecture & ML in Production	74
10.0.1	Scope	74
10.0.2	What the literature/sources establish	74
10.0.3	What this means for our build	76
10.0.4	Decisions proposed	78
10.0.5	Open questions / conflicts	79
10.0.6	Sources	80
11	Track J: Biosecurity & Compliance	82
11.0.1	Scope	82
11.0.2	What the literature/sources establish	82
11.0.3	What this means for our build	85
11.0.4	Decisions proposed	86
11.0.5	Open questions / conflicts	87
11.0.6	Sources	88

12 Annotated Bibliography	90
12.0.1 Track A — Plasmid Biology & Structure	90
12.0.2 Track B — Design & Validity Rules	91
12.0.3 Track C — DNA Language Models & Generation	92
12.0.4 Track D — Prior Art & Related Tools	94
12.0.5 Track E — Data Sources & Access	95
12.0.6 Track F — Sequence Representation & Tokenization	96
12.0.7 Track G — Validation Tooling	97
12.0.8 Track H — Visualization	98
12.0.9 Track I — System Architecture & ML In Production	99
12.0.10 Track J — Biosecurity & Compliance	100
13 Open Questions for Human Review	102

Chapter 1

Executive Summary

Status: Phase R research complete pending human review.

This synthesis distills Tracks A-J into build decisions. It is grounded in the cited findings files under `research/findings/`; those files remain the detailed source of record.

1.0.1 Required Answers From SYSTEM_DESIGN Section 4.5.2

1. Plasmid structure

A valid generic expression plasmid needs at least a propagation module and, when expression is requested, an expression cassette. The propagation module usually includes a compatible origin of replication and bacterial selectable marker. The expression cassette normally has a host-compatible promoter upstream of the GOI/MCS, correct GOI orientation and reading frame, and a downstream terminator or polyA element. Mammalian constructs often combine an E. coli propagation backbone with a mammalian expression cassette and sometimes a separate experimental marker. Circular topology and strand-aware coordinates must be first-class data, not display-only metadata. [Source: Track A, `research/findings/plasmid_biology.md`; Track B, `research/findings/design_rules.md`; Track F, `research/findings/representation.md`]

Vector-specific rules are mandatory. Lentiviral, AAV, shRNA, bacterial-expression, mammalian-expression, plant, yeast, CRISPR, and other designs have different required parts and failure modes, so the validator should use per-vector profiles rather than a single global "valid plasmid" rule. [Source: Track A, `research/findings/plasmid_biology.md`; Track B, `research/findings/design_rules.md`]

2. Design rules

Constructs fail when biological compatibility, sequence syntax, assembly strategy, propagation, or synthesis constraints are violated. Core codifiable rules include promoter/host/transcript compatibility, host-appropriate translation initiation context, correct cassette order and strand, in-frame tags/linkers/GOI joins, downstream terminator/polyA, compatible ORI and marker, cloning-method-specific restriction-site rules, codon-usage scoring against the target host, repeat/instability screening, and synthesis-provider GC/homopolymer/repeat thresholds. [Source: Track B, `research/findings/design_rules.md`; Track G, `research/findings/validation_tools.md`]

Provider thresholds conflict enough that synthesis readiness must be profile-driven. Twist, IDT, and GenScript publish overlapping but non-identical limits for GC content, repeats, homopolymers, and sequence-complexity review. The default should be conservative until the

user selects a provider. [Source: Track B, [research/findings/design_rules.md](#); Track G, [research/findings/validation_tools.md](#)]

3. Sequence generation

The generator should start from an autoregressive DNA model, not an encoder-only genomic model. Evo 2 7B is the strongest initial candidate for whole-plasmid experiments because it is autoregressive, single-nucleotide resolution, long-context, all-domain, and appears Apache-2.0 in the open local repository/model-card path. Carbon-3B is a credible lower-cost Apache-2.0 baseline with metadata-conditioned generation. DNABERT, DNABERT-2, Nucleotide Transformer, HyenaDNA, Caduceus, and GenSLM are better fits for embeddings, scoring, or validation unless a specific generative use is proven. [Source: Track C, [research/findings/sequence_models.md](#)]

No reviewed base model should be assumed to produce synthesis-ready plasmids directly. Whole-plasmid generation must be retrieval-grounded, component-aware, re-annotated, and passed through deterministic validation before user export. Model and data licenses are hard gates; non-commercial or unclear checkpoint licenses cannot be used in commercial production without legal clearance. [Source: Track C, [research/findings/sequence_models.md](#); Track I, [research/findings/architecture_patterns.md](#)]

4. Prior art

OriGen demonstrates learned generation of functional plasmid origins/replicons, including E. coli validation, but does not solve whole annotated expression-plasmid design. PlasmidGPT is relevant for plasmid-like sequence generation and embeddings, but its public validation and license status are insufficient for direct production reliance. Benchling, SnapGene, PlasMapper, pLannotate, VectorBuilder, Asimov Kernel, and Addgene cover parts of the workflow: manual CAD, annotation, maps, cloning simulation, component libraries, order handoff, or limited compiler-style design. [Source: Track D, [research/findings/prior_art.md](#)]

The product gap is the integrated workflow: plain-English experimental goal, structured intent, retrieval of grounded real plasmids, complete annotated design, deterministic validation, primers, synthesis-ready export, biosecurity screening, and provenance. No surveyed public source establishes that complete end-to-end capability. [Source: Track D, [research/findings/prior_art.md](#)]

5. Data

Addgene is the best contextual plasmid source because its approved API/bulk routes expose genes, species, experimental use, vector type, expression, promoters, resistance markers, cloning metadata, article fields, depositor comments, sequence files, and terms. It is approval-gated and license-scoped; ingestion must require an API token plus explicit accepted-license configuration, and records must preserve source terms and sequence provenance. Addgene sequence completeness varies across NGS full sequences, depositor full sequences, depositor partial sequences, and Sanger insert-verification records. [Source: Track E, [research/findings/data_sources.md](#)]

NCBI GenBank is the broad annotated sequence source. Use Entrez ESearch/EPost/EFetch with registered `tool` and `email`, conservative throttling, optional API key, and `rettype=gb` for feature-bearing ingestion. GenBank places no broad NCBI restrictions on use, but submitter IP claims remain possible, so commercial training clearance is still a review item. [Source: Track E, [research/findings/data_sources.md](#)]

6. Representation

Use a canonical internal sequence model with raw DNA, topology, feature intervals, strand, feature type, label, qualifiers, evidence, and provenance. Internally use zero-based half-open coordinates with explicit import/export conversions for GenBank and GFF3. Preserve original source locations because GenBank/INSDC annotations can include joins, complements, fuzzy bounds, and origin-wrapping features. [Source: Track F, [research/findings/representation.md](#)]

Retrieval should store composed text documents, structured metadata, sequence windows, and component-level chunks. Validation must run on exact bases and coordinates, not token IDs, because restriction sites, frames, primers, overlaps, and circular wraparound are base-level properties. Circular plasmids need rotation, origin-aware slicing, and cross-origin utilities. [Source: Track F, [research/findings/representation.md](#)]

7. Validation

The deterministic validation engine should be built as independent checks with PASS/WARN/FAIL results, coordinates, affected features, rule IDs, and remediation text. Start with:

- Restriction-site conflicts via Biopython `Bio.Restriction` plus version-pinned REBASE-derived metadata.
- Codon usage scoring via local HIVE-CUTs/Kazusa-style tables and Biopython CAI, without automatic rewrite in validation.
- Repeat/instability checks for homopolymers, direct/tandem/inverted repeats, palindromes, repeat density, and local GC windows.
- Regulatory compatibility via local curated rule tables seeded from Addgene guidance, EPD/JASPAR/RegulonDB where relevant, and plasmid-specific promoter catalogs.
- Provider-profile synthesis readiness for Twist, IDT, and GenScript-style constraints.
- Optional therapeutic/gene-therapy compliance flags coordinated with the biosecurity layer. [Source: Track G, [research/findings/validation_tools.md](#); Track B, [research/findings/design_rules.md](#); Track J, [research/findings/biosecurity.md](#)]

8. Visualization

Use `seqviz` as the Phase 4 default renderer because it supports circular, linear, and combined views and maps naturally from `AnnotatedSequence` to `{ name, seq, annotations, viewer }`. Keep visualization behind a frontend adapter, e.g. `PlasmidMapView`, so backend/API contracts remain based on the canonical `AnnotatedSequence` rather than viewer-specific props. Use deterministic colors by feature type. Test origin-spanning annotations before implementation because this pass did not verify `seqviz` behavior for features crossing base 1. [Source: Track H, [research/findings/visualization.md](#)]

Open Vector Editor is the likely future choice for browser editing, `CGView.js` for dense circular genome/plot export, and `JBrowse/igv.js` for genome-browser workflows. They are not the simplest MVP map renderer. [Source: Track H, [research/findings/visualization.md](#)]

9. Building a system of this scope

Use a layered, interface-first architecture: ingestion/normalization, canonical schemas, retrieval/indexing, intent parsing, recommendation generation, sequence generation, deterministic validation, biosecurity screening, export/assembly, API/jobs, and UI. RAG improves grounding and provenance but does not prove correctness; biological validation and screening are separate gates. [Source: Track I, [research/findings/architecture_patterns.md](#)]

Retrieval evaluation and generation evaluation must be separate. Track top-k retrieval, source faithfulness, recommendation quality, biological validation pass rate, synthesis readiness, latency, cost, and regression against curated prompts. Use deterministic fakes for every model/infrastructure interface. vLLM is the likely first real serving target for autoregressive DNA models if compatible; Triton is a later option for heterogeneous model serving. Quantized models must pass the same biological evals as full precision. [Source: Track I, [research/findings/architecture_patterns.md](#)]

10. Biosecurity and compliance

Every prompt, generated sequence, export, and synthesis handoff needs biosecurity handling. Prompt screening is triage, not a substitute for sequence screening. Add a mandatory `BiosecurityScreening` interface before export/order handoff, with states such as `CLEAR`, `REVIEW_REQUIRED`, `BLOCKED`, and `SCREENING_UNAVAILABLE`. No synthesis-ready export should proceed unless screening is clear and provider-side screening/attestation is recorded. [Source: Track J, [research/findings/biosecurity.md](#)]

Production screening must support current U.S. nucleic-acid screening expectations, including sequence-of-concern review, customer legitimacy, translation-aware screening where applicable, audit logging, data minimization, and a human-review path. IBBIS Common Mechanism or provider APIs are plausible implementation candidates, but their limitations for short sequences, proteins, ambiguous codons, and specialized oligo orders need fallback handling. Compliance output should be advisory and not a substitute for institutional biosafety, export-control, select-agent, or synthesis-provider review. [Source: Track J, [research/findings/biosecurity.md](#)]

1.0.2 Build Decisions To Carry Forward

- Phase 0 should build schemas and parsers around explicit topology, zero-based half-open coordinates, feature provenance, and source/legal metadata.
- Phase 0 ingestion should implement Addgene and NCBI adapters separately, with Addgene disabled until an approved license/token is configured.
- Phase 1 should be retrieval-first, using composed plasmid documents plus structured filters and provenance-preserving component chunks.
- Phase 2 should benchmark Evo 2 7B and Carbon-3B behind `SequenceGenerator`; do not depend on unclear/non-commercial model weights.
- Phase 3 should implement deterministic checks as pure functions with provider profiles and vector-type-specific rule tables.
- Phase 4 should use `seqviz` through an adapter component, not expose viewer-specific shapes through the backend contract.

- Cross-cutting safety must include a dedicated biosecurity screening interface before export/order handoff.

Chapter 2

Track A: Plasmid Biology & Structure

2.0.1 Scope

This track investigated plasmid anatomy and structure for the build: the roles and common placement of origin of replication, promoter, gene of interest, selectable marker, multiple cloning site, and terminator/polyadenylation elements; circular/supercoiled topology; and high-level rules that distinguish a biologically plausible construct from an invalid one.

2.0.2 What the literature/sources establish

- A cloning plasmid acts as a vector because an inserted DNA fragment can be propagated in a host cell when the plasmid retains replicative ability; Brown describes gene cloning as inserting DNA into a plasmid or virus chromosome that is then replicated in a bacterial or eukaryotic host. [Source: Brown, 2002, "Studying DNA", NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK21129/>]
- Bacterial plasmid propagation depends on an origin of replication recognized by host replication machinery; Brown states that plasmids replicate efficiently in bacterial hosts because each plasmid has an origin of replication recognized by polymerases and proteins that replicate bacterial chromosomes. [Source: Brown, 2002, "Studying DNA", NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK21129/>]
- Origins are not generic labels: del Solar et al. identify plasmid origin(s), often Rep initiator proteins, and replication-control genes as key elements in circular bacterial plasmid replication systems. [Source: del Solar et al., 1998, "Replication and Control of Circular Bacterial Plasmids", DOI: [10.1128/MMBR.62.2.434-464.1998](https://doi.org/10.1128/MMBR.62.2.434-464.1998)]
- Plasmid vector classification can use replication origin, selection marker, and promoter as core criteria, which supports treating those elements as first-class structured annotations rather than free text. [Source: Wang et al., 2009, "Classification of plasmid vectors using replication origin, selection marker and promoter as criteria", DOI: [10.1016/j.plasmid.2008.09.003](https://doi.org/10.1016/j.plasmid.2008.09.003)]
- Practical plasmid maps usually include an origin of replication, antibiotic-resistance gene, selectable marker, promoter, restriction sites, insert, and primer-binding sites; Addgene's reference map presents those as general plasmid features. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]

- A bacterial antibiotic-resistance gene is a selectable marker for plasmid-bearing bacteria; Brown gives pBR322 ampicillin and tetracycline resistance as selectable markers because cells carrying the plasmid survive on those antibiotics. [Source: Brown, 2002, "Studying DNA", NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK21129/>]
- A non-bacterial experimental selectable marker can be separate from the bacterial propagation marker; Addgene distinguishes bacterial antibiotic resistance used for plasmid-bearing bacteria from selectable markers used to select experimental target cells, often another antibiotic-resistance gene under a non-bacterial promoter or a fluorescent protein. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]
- A multiple cloning site is a short DNA segment containing several restriction-enzyme sites for insertion by digestion and ligation, and Addgene notes that MCS restriction sites are generally unique and absent elsewhere in the backbone. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]
- The classic pBR322 cloning vehicle illustrates backbone validity rules: Bolivar et al. designed pBR322 with antibiotic-resistance genes, a single PstI site in the ampicillin-resistance gene, and unique EcoRI, HindIII, BamHI, and SalI restriction sites. [Source: Bolivar et al., 1977, "Construction and characterization of new cloning vehicles. II. A multipurpose cloning system", DOI: [10.1016/0378-1119\(77\)90000-2](https://doi.org/10.1016/0378-1119(77)90000-2)]
- Promoters drive transcription of the insert by recruiting transcriptional machinery; Addgene states that promoter sequence controls RNA polymerase/transcription-factor binding and therefore affects where and when a gene of interest is expressed. [Source: Morgan/Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- Promoter-host compatibility is a validity requirement; Addgene states that bacterial promoters work in prokaryotic cells and generally in the same or related species, while mammalian, yeast, and plant systems require different promoter classes with little crossover. [Source: Morgan/Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- Promoter-transcript compatibility is also required; Addgene states that mRNA expression in eukaryotic systems requires an RNA polymerase II promoter, while small RNAs such as shRNA are transcribed from RNA polymerase III promoters. [Source: Morgan/Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- In expression plasmids, the MCS is often placed downstream from a promoter so the inserted gene can be expressed; Addgene states that expression vectors must contain a promoter, transcription terminator, and inserted gene. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]
- A typical mammalian expression cassette can be represented as promoter/ORF/polyadenylation; Molloy et al. describe a DNA-vaccine plasmid with a kanamycin-resistance cassette, pUC origin, and mammalian expression cassette consisting of a Pol II promoter, ORF, and polyadenylation element. [Source: Molloy et al., 2004, "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed", DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124)]

- For a coding gene of interest, orientation and reading frame matter when the design includes protein fusion tags or cloning into an expression cassette; Addgene's Gateway cloning guidance warns to include expression elements such as ribosome-recognition sequences, start codon, stop codons, and reading-frame considerations. [Source: Addgene, 2016, "Plasmids 101: Gateway Cloning", <https://blog.addgene.org/plasmids-101-gateway-cloning>]
- Engineered plasmids are commonly propagated in E. coli and are often kept small for manipulation; Addgene states that many common recombinant-DNA plasmids are replicated in E. coli and are relatively small, roughly 3,000 to 6,000 bp, to enable easy manipulation. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]
- Circular plasmid DNA propagated in E. coli is predominantly negatively supercoiled, while purification/storage can create open-circular and linear forms by single-strand or double-strand nicking; Molloy et al. report supercoiled, relaxed closed-circular, open-circular, linear, multimeric, and aggregate plasmid species in purified batches. [Source: Molloy et al., 2004, "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed", DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124)]
- Closed circular plasmid topology is biologically relevant because supercoiling, catenation, and knotting are physical states DNA molecules can adopt; Higgins and Vologodskii review plasmid topological behavior and note that closed circular DNA can be supercoiled. [Source: Higgins and Vologodskii, 2015, "Topological Behavior of Plasmid DNA", DOI: [10.1128/microbiolspec.PLAS-0036-2014](https://doi.org/10.1128/microbiolspec.PLAS-0036-2014)]
- Viral-vector transfer plasmids add vector-type-specific requirements beyond the generic plasmid backbone; Addgene describes lentiviral and retroviral transfer plasmids as carrying cargo while helper plasmids supply packaging or replication components. [Source: Addgene, 2023, "Viral Vectors 101: Viral Vector Elements", <https://blog.addgene.org/viral-vectors-101-viral-vector-elements>]

2.0.3 What this means for our build

- The canonical annotation model should require separate feature types for **ORI**, **promoter**, **GOI**, **marker**, **MCS**, **terminator**, and **other**, because those elements are repeatedly identified as distinct functional units in plasmid references. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>; Source: Wang et al., 2009, "Classification of plasmid vectors using replication origin, selection marker and promoter as criteria", DOI: [10.1016/j.j.plasmid.2008.09.003](https://doi.org/10.1016/j.j.plasmid.2008.09.003)]
- The parser should distinguish bacterial propagation features from experimental expression features, because a valid mammalian expression plasmid may need both an E. coli origin/antibiotic marker and a mammalian promoter/expression marker. [Source: Brown, 2002, "Studying DNA", NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK21129/>; Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]
- A generated expression cassette should normally place the promoter upstream of the GOI/MCS and a terminator or polyadenylation element downstream of the expressed sequence, because expression vectors are described as promoter plus inserted gene plus transcription terminator, and mammalian cassettes are described as promoter/ORF/polyadenylation.

[Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>; Source: Molloy et al., 2004, "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed", DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124)]

- The validity engine should fail constructs with no compatible origin for the intended propagation host, no selectable marker for the intended selection context, no compatible promoter for the intended expression host/transcript, no downstream terminator/polyadenylation for expression, or an inserted GOI that disrupts an essential backbone function. [Source: Brown, 2002, "Studying DNA", NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK21129/>; Source: Morgan/Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>; Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]
- The design representation should store `topology` separately from sequence text, because the same nucleotide sequence can be represented as circular, linearized, open-circular, or supercoiled physical species in plasmid handling and manufacturing contexts. [Source: Molloy et al., 2004, "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed", DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124); Source: Higgins and Vologodskii, 2015, "Topological Behavior of Plasmid DNA", DOI: [10.1128/microbiolspec.PLAS-0036-2014](https://doi.org/10.1128/microbiolspec.PLAS-0036-2014)]
- Retrieval and generation should treat vector type as a constraint, not just a label, because viral transfer plasmids and standard cloning/expression plasmids have different required elements and helper-system assumptions. [Source: Addgene, 2023, "Viral Vectors 101: Viral Vector Elements", <https://blog.addgene.org/viral-vectors-101-viral-vector-elements>; Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]

2.0.4 Decisions proposed

- Use this minimum "generic expression plasmid" structural expectation for Phase 0/Phase 3 checks: circular annotated sequence; bacterial propagation module with compatible ORI plus bacterial selection marker; expression module with host-compatible `promoter` -> GOI/MCS -> `terminator/polyA`; optional experimental selectable/reporter marker; and vector-type-specific extras for viral or specialized vectors. [Source: Brown, 2002, "Studying DNA", NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK21129/>; Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>; Source: Molloy et al., 2004, "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed", DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124)]
- Store feature coordinates with strand/orientation and validate ordering on the relevant strand, because promoter-upstream and terminator-downstream rules only make sense relative to the transcribed GOI orientation. [Source: Morgan/Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>; Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]

- Treat MCS uniqueness and internal restriction-site conflicts as validation checks parameterized by cloning method, because MCS utility depends on usable restriction sites and pBR322-style vectors were explicitly designed around unique restriction sites. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>; Source: Bolivar et al., 1977, "Construction and characterization of new cloning vehicles. II. A multi-purpose cloning system", DOI: [10.1016/0378-1119\(77\)90000-2](https://doi.org/10.1016/0378-1119(77)90000-2)]
- Represent bacterial marker and experimental target-cell marker as separate concepts in schemas and UI, because Addgene distinguishes the bacterial antibiotic-resistance feature from markers used to select or sort experimental cells. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]

2.0.5 Open questions / conflicts

- "Valid construct" is vector-type-specific: generic plasmid rules do not define all required lentiviral, AAV, CRISPR, shRNA, yeast, plant, or bacterial-expression elements, so later tracks or follow-up research should define per-vector-type validity profiles. [Source: Addgene, 2023, "Viral Vectors 101: Viral Vector Elements", <https://blog.addgene.org/viral-vectors-101-viral-vector-elements>; Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>]
- The minimum acceptable terminator/polyadenylation annotation differs by host and transcript type, because Addgene describes expression vectors generally as requiring a transcription terminator while Molloy et al. describe a mammalian Pol II cassette with polyadenylation. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>; Source: Molloy et al., 2004, "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed", DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124)]
- Size guidance is contextual: Addgene notes many commonly used E. coli-propagated recombinant plasmids are roughly 3,000 to 6,000 bp for easy manipulation, but viral vectors, BACs, complex multi-cassette constructs, and synthesis-provider constraints require separate size rules. [Source: Addgene, 2025, "Molecular Biology Reference", <https://www.addgene.org/mol-bio-reference/>; Source: Brown, 2002, "Studying DNA", NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK21129/>]
- Plasmid topology matters for physical manufacturing and QC, but a sequence generator can only specify circular versus linear sequence topology unless later workflow stages model preparation state such as supercoiled, open-circular, or linear isoform. [Source: Molloy et al., 2004, "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed", DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124); Source: Higgins and Vologodskii, 2015, "Topological Behavior of Plasmid DNA", DOI: [10.1128/microbiolspec.PLAS-0036-2014](https://doi.org/10.1128/microbiolspec.PLAS-0036-2014)]

2.0.6 Sources

1. Addgene. 2025. "Molecular Biology Reference." <https://www.addgene.org/mol-bio-reference/>

2. Bolivar, F., Rodriguez, R. L., Greene, P. J., Betlach, M. C., Heyneker, H. L., Boyer, H. W., Crosa, J. H., and Falkow, S. 1977. "Construction and characterization of new cloning vehicles. II. A multipurpose cloning system." *Gene* 2(2):95-113. DOI: [10.1016/0378-1119\(77\)90000-2](https://doi.org/10.1016/0378-1119(77)90000-2).
3. Brown, T. A. 2002. "Studying DNA." In *Genomes*, 2nd edition. NCBI Bookshelf. <https://www.ncbi.nlm.nih.gov/books/NBK21129/>
4. del Solar, G., Giraldo, R., Ruiz-Echevarria, M. J., Espinosa, M., and Diaz-Orejas, R. 1998. "Replication and Control of Circular Bacterial Plasmids." *Microbiology and Molecular Biology Reviews* 62(2):434-464. DOI: [10.1128/MMBR.62.2.434-464.1998](https://doi.org/10.1128/MMBR.62.2.434-464.1998).
5. Wang, Z., Jin, L., Yuan, Z., Wegrzyn, G., and Wegrzyn, A. 2009. "Classification of plasmid vectors using replication origin, selection marker and promoter as criteria." *Plasmid* 61(1):47-51. DOI: [10.1016/j.plasmid.2008.09.003](https://doi.org/10.1016/j.plasmid.2008.09.003).
6. Molloy, M. J., Hall, V. S., Bailey, S. I., Griffin, K. J., Faulkner, J., and Uden, M. 2004. "Effective and robust plasmid topology analysis and the subsequent characterization of the plasmid isoforms thereby observed." *Nucleic Acids Research* 32(16):e129. DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124).
7. Morgan, K. / Addgene. 2014. "Plasmids 101: The Promoter Region - Let's Go!" <https://blog.addgene.org/plasmids-101-the-promoter-region>
8. Addgene. 2016. "Plasmids 101: Gateway Cloning." <https://blog.addgene.org/plasmids-101-gateway-cloning>
9. Addgene. 2023. "Viral Vectors 101: Viral Vector Elements." <https://blog.addgene.org/viral-vectors-101-viral-vector-elements>
10. Higgins, N. P., and Vologodskii, A. V. 2015. "Topological Behavior of Plasmid DNA." *Microbiology Spectrum* 3(2). DOI: [10.1128/microbiolspec.PLAS-0036-2014](https://doi.org/10.1128/microbiolspec.PLAS-0036-2014).

Chapter 3

Track B: Design & Validity Rules

3.0.1 Scope

This track investigated design rules that determine whether a plasmid construct is likely to function and be synthesis-ready: promoter/host matching, codon optimization, element spacing, restriction-site avoidance, repeat instability, GC-content and synthesis constraints, and common failure modes.

3.0.2 What the literature/sources establish

- Promoters must match both the intended transcript type and host expression machinery: Addgene states that mRNA expression in eukaryotic systems uses RNA polymerase II promoters, small RNAs such as shRNA use RNA polymerase III promoters, bacterial promoters work in prokaryotic cells, and eukaryotic promoter classes have little crossover between mammalian, yeast, and plant contexts. [Source: Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- Promoter choice can change both expression level and biological outcome because promoters control RNA polymerase/transcription-factor binding and determine where and when the gene of interest is expressed; Addgene also notes that promoter leakiness can confound results or harm cells when the payload is toxic. [Source: Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- For lentiviral and other mammalian vectors, ubiquitous promoters such as CMV, EF1-alpha, CAG, and PGK are commonly used, but CMV and other viral promoters can show host/context-dependent variability and methylation-associated silencing. [Source: Haellman and Piras, 2023, "The sound of silence: transgene silencing in mammalian cell engineering", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9880859/>]
- Viral expression cassettes are constrained by both biological element choice and vector capacity; an AAV/lentiviral expression-cassette review describes promoter, ITR/LTR, and polyA as essential cassette elements and gives approximate packaging ranges of 4.1-4.9 kb for AAV and 8-9 kb for lentivirus. [Source: Powell et al., 2015, "Viral Expression Cassette Elements to Enhance Transgene Target Specificity and Expression in Gene Therapy", <https://pmc.ncbi.nlm.nih.gov/articles/PMC4505817/>]
- Mammalian expression vectors require coordinated promoter, terminator/polyA, translation-initiation, bacterial propagation, and selection elements; a mammalian expression-vector review lists constitutive/inducible promoter, transcription terminator, Kozak sequence, stop

codon, cleavage/polyadenylation signal, prokaryotic origin, bacterial marker, and stable-cell-line marker as functional vector elements. [Source: Khan, 2013, "Gene Expression in Mammalian Cells and its Applications", <https://pmc.ncbi.nlm.nih.gov/articles/PMC7147855/>]

- Terminators belong downstream of the gene/transcription unit, define transcription end, release the new RNA from transcription machinery, and influence RNA processing, RNA half-life, and gene expression. [Source: Addgene, 2014, "Plasmids 101: Terminators and PolyA signals", <https://blog.addgene.org/plasmids-101-terminators-and-polya-signals>]
- Common bacterial expression plasmids typically use Rho-independent terminators such as T7, rrnB, or engineered T0; Addgene notes that double terminators can reduce unwanted expression of downstream elements. [Source: Addgene, 2014, "Plasmids 101: Terminators and PolyA signals", <https://blog.addgene.org/plasmids-101-terminators-and-polya-signals>]
- In mammalian pre-mRNA processing, the core polyadenylation signal includes AAUAAA 10-30 nt upstream of the cleavage site, a downstream U-rich or GU-rich element, and the cleavage site. [Source: Zhao et al., 1999, "Formation of mRNA 3' Ends in Eukaryotes: Mechanism, Regulation, and Interrelationships with Other Steps in mRNA Synthesis", <https://pmc.ncbi.nlm.nih.gov/articles/PMC98971/>]
- Kozak found that sequences flanking the AUG initiator codon influence recognition by eukaryotic ribosomes and that CCA/GCCAUGG emerged as a mammalian initiation consensus from hundreds of mRNAs. [Source: Kozak, 1987, "At least six nucleotides preceding the AUG initiator codon enhance translation in mammalian cells", <https://pubmed.ncbi.nlm.nih.gov/3681984/>]
- A cross-eukaryote analysis reports that Kozak's vertebrate GCCGCC(A/G)CCAUGG context enhances translation initiation, while preferred nucleotide contexts differ across eukaryotic lineages. [Source: Nakagawa et al., 2008, "Diversity of preferred nucleotide sequences around the translation initiation codon in eukaryote genomes", <https://pmc.ncbi.nlm.nih.gov/articles/PMC2241899/>]
- In *E. coli*, the Shine-Dalgarno/RBS region and its spacing to the start codon strongly affect translation efficiency; Chen et al. measured optimal aligned Shine-Dalgarno-to-AUG spacing of 5 nt in synthetic RBS constructs lacking complicating secondary structure. [Source: Chen et al., 1994, "Determination of the optimal aligned spacing between the Shine-Dalgarno sequence and the translation initiation codon of *Escherichia coli* mRNAs", <https://academic.oup.com/nar/article/22/23/4953/2400328>]
- *E. coli* sigma-70 promoter architecture commonly uses conserved -35 TTGACA and -10 TATAAT hexamers; Hawley and McClure found that 92% of compiled promoters had 17 +/- 1 bp spacing between those regions and that 75% of defined starts were 7 +/- 1 bases downstream of the -10 region. [Source: Hawley and McClure, 1983, "Compilation and analysis of *Escherichia coli* promoter DNA sequences", <https://pmc.ncbi.nlm.nih.gov/articles/PMC340638/>]
- RegulonDB v12.0 is a curated reference for *E. coli* K-12 transcription-initiation regulation, including promoters, transcription units, transcription-factor binding sites, transcription-factor regulatory sites, terminators, transcription start sites, and condition-linked expression evidence. [Source: Santos-Zavaleta et al., 2024, "RegulonDB v12.0: a comprehensive resource

of transcriptional regulation in E. coli K-12", <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>]

- The Kazusa Codon Usage Database tabulates codon usage from GenBank and provides organism-specific codon-frequency data in downloadable table formats. [Source: Nakamura et al., 2000, "Codon usage tabulated from international DNA sequence databases: status for the year 2000", <https://pmc.ncbi.nlm.nih.gov/articles/PMC102460/>]
- The Codon Adaptation Index uses a reference set of highly expressed genes from a species to score codon-usage bias and can approximate the likely success of heterologous expression. [Source: Sharp and Li, 1987, "The codon adaptation index - a measure of directional synonymous codon usage bias, and its potential applications", <https://doi.org/10.1093/nar/15.3.1281>]
- Codon optimization is not only "use preferred codons": Mauro and Chappell define it as synonymous recoding to increase protein production and warn that synonymous changes can affect protein conformation/function, immunogenicity, and efficacy. [Source: Mauro and Chappell, 2014, "A critical analysis of codon optimization in human therapeutics", <https://doi.org/10.1016/j.molmed.2014.09.003>]
- Synthetic gene design literature describes multi-constraint optimization that filters candidate coding sequences for codon/codon-pair usage, extreme GC content, repetitive sequence, unfavorable mRNA structure, and required inclusion or exclusion of restriction sites. [Source: Parret et al., 2016, "Codon usage bias", <https://pmc.ncbi.nlm.nih.gov/articles/PMC8613526/>]
- NEB advises that restriction sites added to PCR primers need extra 5-prime bases for efficient cutting and that the selected restriction recognition site must not occur inside the gene or DNA fragment being cloned. [Source: New England Biolabs, 2026, "Restriction Enzyme Digestion", <https://www.neb.com/en/applications/cloning-and-synthetic-biology/dna-preparation/restriction-enzyme-digestion>]
- Golden Gate cloning uses type IIS enzymes to produce defined overhangs for ordered assembly, and the original Engler et al. method exploited type IIS cleavage outside the recognition site to obtain highly efficient one-pot restriction-ligation assembly. [Source: Engler et al., 2008, "A one pot, one step, precision cloning method with high throughput capability", <https://pubmed.ncbi.nlm.nih.gov/18985154/>]
- Standardized type IIS assembly systems require domestication of parts by removing internal enzyme recognition sites; the GoldenBraid paper states that internal sites are removed by overlapping PCR, directed mutagenesis, or direct DNA synthesis. [Source: Sarrion-Perdigones et al., 2011, "GoldenBraid: An Iterative Cloning System for Standardized Assembly of Reusable Genetic Modules", <https://pmc.ncbi.nlm.nih.gov/articles/PMC3131274/>]
- Twist's gene-design guidance states that sequence parameters such as GC content, repeat length, repeat density, and homopolymers affect synthesis success, and recommends avoiding repeats ≥ 20 bp or $T_m \geq 60$ C, keeping global GC between 25% and 65%, limiting 50-bp GC-window spread to $\leq 50\%$, avoiding homopolymers longer than 13 nt, avoiding clusters of 8-9 bp repeats, and avoiding direct repeats longer than 12-16 bp. [Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your

Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/D0C-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf]

- Twist's public product guidance says synthesis issues are mainly driven by repetitive structures, extreme GC content, and homopolymers; Twist also states hard rules to avoid homopolymers ≥ 14 bp and CcdB, and says its scoring model combines features such as overall GC percent, homopolymer length, repeat length, sequence length, and repeat density. [Source: Twist Bioscience, 2026, "High Quality Gene Synthesis", <https://www.twistbioscience.com/products/genes/gene-synthesis?tab=fragment>]
- IDT states that gBlocks synthesis can be problematic for sequences with GC content below 25% or above 75%, homopolymeric runs of 10 or more A/T bases or 6 or more G/C bases, and that repeats or hairpins can also affect acceptance in a multifactorial review. [Source: Integrated DNA Technologies, 2026, "What types of sequence motifs should be avoided when ordering gBlocks Gene Fragments?", <https://www3.idtdna.com/pages/support/faqs/what-types-of-sequence-motifs-should-be-avoided-when-ordering-gblocks-gene-fragments->]
- GenScript states that synthesis success is mainly determined by repetitive sequences, GC content, host-cell stability, and toxicity, and flags global GC below 20% or above 80%, variable local GC, homopolymers, repeats ≥ 20 bp, small repeat count/length, and palindromic sequence as analysis factors. [Source: GenScript, 2026, "FLASH Gene & Gene Synthesis - 4-Day Gene to Plasmid", https://www.genscript.com/gene_synthesis.html]
- GenScript identifies repeated sequences as a cloning failure mode because repeated or highly similar regions can cause improper annealing or ligation, and says codon optimization can reduce repetition while preserving amino-acid sequence when the experiment allows synonymous changes. [Source: GenScript, 2026, "Reasons Cloning Fails - and Solutions", https://www.genscript.com/reasons_cloning_fails_solutions.html]
- Lentiviral plasmids can be unstable in bacterial propagation; Yau et al. report structural changes or loss in some E. coli hosts and discuss HIV-1 LTR inverted repeats, G-quadruplex-capable sequences, toxic genes, long inverted repeats, and multiple direct repeats as plasmid-instability factors. [Source: Yau et al., 2013, "Remarkable stability of an instability-prone lentiviral vector plasmid in Escherichia coli Stbl3", <https://pmc.ncbi.nlm.nih.gov/articles/PMC3563744/>]
- AAV ITRs are essential vector elements but are unstable during plasmid propagation in E. coli; a 2024 study reports that among 123 sequence-verified ITR plasmids from Addgene, 69 had unintended internal ITR deletions. [Source: Ling et al., 2024, "Degradation and stable maintenance of adeno-associated virus inverted terminal repeats in E. coli", <https://pmc.ncbi.nlm.nih.gov/articles/PMC11754738/>]
- Twist recommends reviewing full constructs before ordering to confirm that the sequence is inserted correctly, open reading frames are accurate, and protein-coding sequences remain in frame. [Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/D0C-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf]

3.0.3 What this means for our build

- The DesignSpec and validation engine should treat promoter compatibility as a structured rule keyed by (host organism/cell line, transcript type, vector type, promoter class), with hard failures for clear mismatches such as bacterial promoters in mammalian expression and RNA polymerase III promoters for protein-coding mRNA cassettes. [Source: Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- The validator should warn, not automatically fail, when a promoter is biologically plausible but risky in context, such as CMV in settings where silencing/variability is known or where leaky expression of a toxic payload could harm cells. [Source: Haellman and Piras, 2023, "The sound of silence: transgene silencing in mammalian cell engineering", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9880859/>; Source: Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- The component-order validator should require the expression cassette to contain promoter upstream of the transcript, coding sequence in the intended orientation and frame, stop codon where appropriate, and host-appropriate terminator/polyA downstream. [Source: Khan, 2013, "Gene Expression in Mammalian Cells and its Applications", <https://pmc.ncbi.nlm.nih.gov/articles/PMC7147855/>; Source: Addgene, 2014, "Plasmids 101: Terminators and PolyA signals", <https://blog.addgene.org/plasmids-101-terminators-and-polya-signals>]
- The start-context checker should be organism-aware: mammalian coding sequences should be scored for Kozak/start context, while bacterial coding sequences should be scored for RBS/Shine-Dalgarno presence, start-codon spacing, and local secondary-structure risk near the translation-initiation region. [Source: Kozak, 1987, "At least six nucleotides preceding the AUG initiator codon enhance translation in mammalian cells", <https://pubmed.ncbi.nlm.nih.gov/3681984/>; Source: Chen et al., 1994, "Determination of the optimal aligned spacing between the Shine-Dalgarno sequence and the translation initiation codon of Escherichia coli mRNAs", <https://academic.oup.com/nar/article/22/23/4953/2400328>]
- Codon optimization should be implemented as a scorer plus optional rewrite step, not as an unconditional transformation, because CAI/codon-usage matching helps estimate host adaptation but synonymous recoding can change biologically relevant properties. [Source: Sharp and Li, 1987, "The codon adaptation index - a measure of directional synonymous codon usage bias, and its potential applications", <https://doi.org/10.1093/nar/15.3.1281>; Source: Mauro and Chappell, 2014, "A critical analysis of codon optimization in human therapeutics", <https://doi.org/10.1016/j.molmed.2014.09.003>]
- Codon-optimization outputs should satisfy a multi-objective constraint set: host codon usage, avoidance of rare-codon clusters, preservation of protein sequence, absence of unintended stop/frame changes, removal of disallowed cloning sites, avoidance of repeats/hairpins/homopolymers, and provider-specific GC windows. [Source: Parret et al., 2016, "Codon usage bias", <https://pmc.ncbi.nlm.nih.gov/articles/PMC8613526/>; Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/D0C-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf]

- The restriction-site checker should be parameterized by cloning method: classic restriction cloning should require unique intended sites and no internal sites in the insert for the selected enzymes, while Golden Gate/MoClo/GoldenBraid should fail undomesticated internal type IIS sites for the assembly standard. [Source: New England Biolabs, 2026, "Restriction Enzyme Digestion", <https://www.neb.com/en/applications/cloning-and-synthetic-biology/dna-preparation/restriction-enzyme-digestion>; Source: Sarrion-Perdigones et al., 2011, "GoldenBraid: An Iterative Cloning System for Standardized Assembly of Reusable Genetic Modules", <https://pmc.ncbi.nlm.nih.gov/articles/PMC3131274/>]
- Synthesis-readiness should be provider-profile driven, because Twist, IDT, and GenScript publish overlapping but non-identical thresholds for GC content, homopolymers, repeats, palindromes, and other sequence-complexity features. [Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/DOC-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf; Source: Integrated DNA Technologies, 2026, "What types of sequence motifs should be avoided when ordering gBlocks Gene Fragments?", <https://www3.idtdna.com/pages/support/faqs/what-types-of-sequence-motifs-should-be-avoided-when-ordering-gblocks-gene-fragments->; Source: GenScript, 2026, "FLASH Gene & Gene Synthesis - 4-Day Gene to Plasmid", https://www.genscript.com/gene_synthesis.html]
- The repeat/instability checker should distinguish synthesis difficulty from bacterial propagation instability, because provider scoring flags repeats/homopolymers/GC as manufacturing features while lentiviral LTRs and AAV ITRs can also rearrange during E. coli propagation. [Source: Twist Bioscience, 2026, "High Quality Gene Synthesis", <https://www.twistbioscience.com/products/genes/gene-synthesis?tab=fragment>; Source: Ling et al., 2024, "Degradation and stable maintenance of adeno-associated virus inverted terminal repeats in E. coli", <https://pmc.ncbi.nlm.nih.gov/articles/PMC11754738/>]
- The validation report should explain likely construct-failure causes separately: biological incompatibility, translation-initiation mismatch, frame/orientation error, missing terminator/polyA, cloning-site conflict, synthesis-complexity issue, propagation-instability issue, and toxic/leaky expression risk. [Source: Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>; Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/DOC-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf; Source: GenScript, 2026, "FLASH Gene & Gene Synthesis - 4-Day Gene to Plasmid", https://www.genscript.com/gene_synthesis.html]

3.0.4 Decisions proposed

- Encode promoter compatibility as a controlled vocabulary table seeded from common promoter classes and curated databases: mammalian RNA polymerase II promoters for protein-coding mammalian expression, mammalian RNA polymerase III promoters for small RNA expression, bacterial promoters/RBSs for bacterial expression, and organism-specific promoter sets from curated sources such as RegulonDB for E. coli. [Source: Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/>]

[plasmids-101-the-promoter-region](#); Source: Santos-Zavaleta et al., 2024, "RegulonDB v12.0", <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>]

- Implement deterministic checks for cassette structure: promoter before transcript, correct strand, expected start context, in-frame tag/linker/GOI joins, no premature stop in the requested ORF, valid terminal stop when needed, and downstream terminator/polyA appropriate to host. [Source: Khan, 2013, "Gene Expression in Mammalian Cells and its Applications", <https://pmc.ncbi.nlm.nih.gov/articles/PMC7147855/>; Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/D0C-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf]
- Use Kazusa/NCBI-derived codon-usage tables for initial CAI/rare-codon scoring, but expose codon optimization as advisory or user-approved rewrite because synonymous recoding can have functional side effects. [Source: Nakamura et al., 2000, "Codon usage tabulated from international DNA sequence databases", <https://pmc.ncbi.nlm.nih.gov/articles/PMC102460/>; Source: Mauro and Chappell, 2014, "A critical analysis of codon optimization in human therapeutics", <https://doi.org/10.1016/j.molmed.2014.09.003>]
- Create a provider-profile configuration for synthesis constraints with at least `twist_default`, `idt_gblocks`, and `genscript_default` profiles; the default product-agnostic warning profile should use conservative thresholds from the strictest overlapping constraints unless the user selects a synthesis provider. [Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/D0C-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf; Source: Integrated DNA Technologies, 2026, "What types of sequence motifs should be avoided when ordering gBlocks Gene Fragments?", <https://www3.idtdna.com/pages/support/faqs/what-types-of-sequence-motifs-should-be-avoided-when-ordering-gblocks-gene-fragments->; Source: GenScript, 2026, "FLASH Gene & Gene Synthesis", https://www.genscript.com/gene_synthesis.html]
- Add vector-type-specific instability rules for lentiviral LTR-containing plasmids and AAV ITR-containing plasmids, including warnings that stable bacterial strains or provider workflows may be required for propagation-sensitive designs. [Source: Yau et al., 2013, "Remarkable stability of an instability-prone lentiviral vector plasmid in Escherichia coli Stbl3", <https://pmc.ncbi.nlm.nih.gov/articles/PMC3563744/>; Source: Ling et al., 2024, "Degradation and stable maintenance of adeno-associated virus inverted terminal repeats in E. coli", <https://pmc.ncbi.nlm.nih.gov/articles/PMC11754738/>]
- Model cloning checks as a strategy-specific interface rather than a single global restriction-site rule, because restriction cloning, Golden Gate, Gibson/HiFi, and synthesis-only workflows have different forbidden motifs and expected junction constraints. [Source: New England Biolabs, 2026, "Restriction Enzyme Digestion", <https://www.neb.com/en/applications/cloning-and-synthetic-biology/dna-preparation/restriction-enzyme-digestion>; Source: Engler et al., 2008, "A one pot, one step, precision cloning method with high throughput capability", <https://pubmed.ncbi.nlm.nih.gov/18985154/>]

3.0.5 Open questions / conflicts

- Provider constraints conflict at the threshold level: Twist recommends global GC 25-65% in its design PDF, IDT flags gBlocks below 25% or above 75% GC, and GenScript flags global GC below 20% or above 80%; the build should ask whether default validation should be conservative, provider-selected, or both. [Source: Twist Bioscience, 2023, "Twist Tips: How to Design Your Gene", https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/D0C-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf; Source: Integrated DNA Technologies, 2026, "What types of sequence motifs should be avoided when ordering gBlocks Gene Fragments?", <https://www3.idtdna.com/pages/support/faqs/what-types-of-sequence-motifs-should-be-avoided-when-ordering-gblocks-gene-fragments->; Source: GenScript, 2026, "FLASH Gene & Gene Synthesis", https://www.genscript.com/gene_synthesis.html]
- Codon optimization has a measurable tradeoff: CAI can estimate host adaptation, but synonymous recoding can alter protein function or immunogenicity, so the human should decide whether the first product should merely score codon adaptation or automatically rewrite coding sequences with explicit user consent. [Source: Sharp and Li, 1987, "The codon adaptation index", <https://doi.org/10.1093/nar/15.3.1281>; Source: Mauro and Chappell, 2014, "A critical analysis of codon optimization in human therapeutics", <https://doi.org/10.1016/j.molmed.2014.09.003>]
- Promoter "validity" is partly empirical and cell-context specific: Addgene gives broad compatibility rules, while mammalian-expression literature reports promoter silencing and host-context variability; the build needs a curated evidence table before claiming that a promoter is optimal for a named cell line. [Source: Addgene, 2014, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>; Source: Haellman and Piras, 2023, "The sound of silence: transgene silencing in mammalian cell engineering", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9880859/>]
- Some sequence features are required but look "bad" to generic synthesis/instability checks, such as AAV ITRs and viral LTRs; the validator should not auto-remove these required elements and needs vector-type-specific exception handling. [Source: Powell et al., 2015, "Viral Expression Cassette Elements to Enhance Transgene Target Specificity and Expression in Gene Therapy", <https://pmc.ncbi.nlm.nih.gov/articles/PMC4505817/>; Source: Ling et al., 2024, "Degradation and stable maintenance of adeno-associated virus inverted terminal repeats in E. coli", <https://pmc.ncbi.nlm.nih.gov/articles/PMC11754738/>]
- The exact threshold for repeat-induced propagation instability is not universal because repeat class, length, orientation, host strain, culture scale, and vector backbone all matter; the initial validator should report risk and provenance rather than making unsupported hard PASS/FAIL calls for every repeat. [Source: Yau et al., 2013, "Remarkable stability of an instability-prone lentiviral vector plasmid in Escherichia coli Stbl3", <https://pmc.ncbi.nlm.nih.gov/articles/PMC3563744/>; Source: GenScript, 2026, "FLASH Gene & Gene Synthesis", https://www.genscript.com/gene_synthesis.html]

3.0.6 Sources

1. Addgene. 2014. "Plasmids 101: The Promoter Region - Let's Go!" <https://blog.addgene.org/plasmids-101-the-promoter-region>
2. Addgene. 2014. "Plasmids 101: Terminators and PolyA signals." <https://blog.addgene.org/plasmids-101-terminators-and-polya-signals>
3. Chen, H., Bjerknes, M., Kumar, R., and Jay, E. 1994. "Determination of the optimal aligned spacing between the Shine-Dalgarno sequence and the translation initiation codon of Escherichia coli mRNAs." Nucleic Acids Research. <https://academic.oup.com/nar/article/22/23/4953/2400328>
4. Engler, C., Kandzia, R., and Marillonnet, S. 2008. "A one pot, one step, precision cloning method with high throughput capability." PLoS ONE. <https://pubmed.ncbi.nlm.nih.gov/18985154/>
5. GenScript. 2026. "FLASH Gene & Gene Synthesis - 4-Day Gene to Plasmid." https://www.genscript.com/gene_synthesis.html
6. GenScript. 2026. "Reasons Cloning Fails - and Solutions." https://www.genscript.com/reasons_cloning_fails_solutions.html
7. Haellman, V. and Piras, V. 2023. "The sound of silence: transgene silencing in mammalian cell engineering." Biotechnology Advances. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9880859/>
8. Hawley, D. K. and McClure, W. R. 1983. "Compilation and analysis of Escherichia coli promoter DNA sequences." Nucleic Acids Research. <https://pmc.ncbi.nlm.nih.gov/articles/PMC340638/>
9. Integrated DNA Technologies. 2026. "What types of sequence motifs should be avoided when ordering gBlocks Gene Fragments?" <https://www3.idtdna.com/pages/support/faqs/what-types-of-sequence-motifs-should-be-avoided-when-ordering-gblocks-gene-fragments->
10. Khan, K. H. 2013. "Gene Expression in Mammalian Cells and its Applications." Advanced Pharmaceutical Bulletin. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7147855/>
11. Kozak, M. 1987. "At least six nucleotides preceding the AUG initiator codon enhance translation in mammalian cells." Journal of Molecular Biology. <https://pubmed.ncbi.nlm.nih.gov/3681984/>
12. Ling, C., et al. 2024. "Degradation and stable maintenance of adeno-associated virus inverted terminal repeats in E. coli." Molecular Therapy Methods & Clinical Development. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11754738/>
13. Mauro, V. P. and Chappell, S. A. 2014. "A critical analysis of codon optimization in human therapeutics." Trends in Molecular Medicine. <https://doi.org/10.1016/j.molmed.2014.09.003>
14. Nakagawa, S., Niimura, Y., Gojobori, T., Tanaka, H., and Miura, K. 2008. "Diversity of preferred nucleotide sequences around the translation initiation codon in eukaryote genomes." Nucleic Acids Research. <https://pmc.ncbi.nlm.nih.gov/articles/PMC2241899/>

15. Nakamura, Y., Gojobori, T., and Ikemura, T. 2000. "Codon usage tabulated from international DNA sequence databases: status for the year 2000." *Nucleic Acids Research*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC102460/>
16. New England Biolabs. 2026. "Restriction Enzyme Digestion." <https://www.neb.com/en/applications/cloning-and-synthetic-biology/dna-preparation/restriction-enzyme-digestion>
17. Parret, A. H. A., Besir, H., and Meijers, R. 2016. "Codon usage bias." *Essays in Biochemistry*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8613526/>
18. Powell, S. K., Rivera-Soto, R., and Gray, S. J. 2015. "Viral Expression Cassette Elements to Enhance Transgene Target Specificity and Expression in Gene Therapy." *Discovery Medicine*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC4505817/>
19. Santos-Zavaleta, A., et al. 2024. "RegulonDB v12.0: a comprehensive resource of transcriptional regulation in *E. coli* K-12." *Nucleic Acids Research*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>
20. Sarrion-Perdigones, A., et al. 2011. "GoldenBraid: An Iterative Cloning System for Standardized Assembly of Reusable Genetic Modules." *PLoS ONE*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3131274/>
21. Sharp, P. M. and Li, W.-H. 1987. "The codon adaptation index - a measure of directional synonymous codon usage bias, and its potential applications." *Nucleic Acids Research*. <https://doi.org/10.1093/nar/15.3.1281>
22. Twist Bioscience. 2023. "Twist Tips: How to Design Your Gene." https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/D0C-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf
23. Twist Bioscience. 2026. "High Quality Gene Synthesis." <https://www.twistbioscience.com/products/genes/gene-synthesis?tab=fragment>
24. Yau, S. Y., et al. 2013. "Remarkable stability of an instability-prone lentiviral vector plasmid in *Escherichia coli* Stbl3." *3 Biotech*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3563744/>
25. Zhao, J., Hyman, L., and Moore, C. 1999. "Formation of mRNA 3' Ends in Eukaryotes: Mechanism, Regulation, and Interrelationships with Other Steps in mRNA Synthesis." *Microbiology and Molecular Biology Reviews*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC98971/>

Chapter 4

Track C: DNA Language Models & Generation

4.0.1 Scope

This track investigated DNA/genomic language models relevant to natural-language-to-plasmid design, with emphasis on DNABERT/DNABERT-2, Nucleotide Transformer, Evo/Evo 2, GenSLM, and newer relevant genomic LMs; it captured architecture, context length, training corpus, tokenization, capabilities, and commercial-use license status for selecting a base model for whole-plasmid, component-aware generation.

4.0.2 What the literature/sources establish

- DNABERT is an encoder-only BERT-style transformer using overlapping k-mer DNA tokens; the paper states it uses the BERT-base shape of 12 transformer layers, 768 hidden units, and 12 attention heads, accepts 512-token inputs, and was pre-trained on human genome sequences of length 10 to 510 with masked-token objectives [Source: Ji et al., 2021, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome", <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680>].
- DNABERT's reported downstream use cases are promoter, transcription-factor-binding-site, and splice-site prediction after fine-tuning, so its primary value is representation learning and classification, not autoregressive generation of full constructs [Source: Ji et al., 2021, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome", <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680>].
- DNABERT's paper says the source code and pretrained/finetuned models are available on GitHub "for future academic research," while NVIDIA BioNeMo describes DNABERT as ready for commercial and non-commercial use; this is a license conflict that must be resolved before commercial deployment [Source: Ji et al., 2021, "DNABERT", <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680>; Source: NVIDIA BioNeMo DNABERT model page, <https://docs.nvidia.com/bionemo-framework/1.10/models/dnabert.html>].
- DNABERT-2 replaces DNABERT's k-mer tokenization with byte-pair encoding (BPE), uses ALiBi and other efficiency changes, and is described by its official repository as a

transformer-based genome foundation model trained on large-scale multi-species genomes [Source: Zhou et al., 2023/2024, "DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome", <https://arxiv.org/abs/2306.15006>; Source: MAGICSLAB DNABERT_2 GitHub, https://github.com/MAGICSLAB/DNABERT_2].

- DNABERT-2-117M is published as a 117M-parameter Hugging Face model that returns 768-dimensional embeddings, and the official Hugging Face card presents embedding examples rather than text-generation examples [Source: zhihan1996/DNABERT-2-117M model card, <https://huggingface.co/zhihan1996/DNABERT-2-117M>].
- DNABERT-2 is evaluated on the GUE benchmark with input lengths from 70 to 10,000 and achieves comparable performance to state-of-the-art models with 21x fewer parameters and about 92x less pretraining GPU time, according to the paper abstract [Source: Zhou et al., 2023/2024, "DNABERT-2", <https://arxiv.org/abs/2306.15006>].
- The DNABERT-2 GitHub repository is marked Apache-2.0, but the official Hugging Face model card for `zhihan1996/DNABERT-2-117M` does not display a license tag in the fetched page, so the weight license should be verified before commercial use [Source: MAGICSLAB DNABERT_2 GitHub, https://github.com/MAGICSLAB/DNABERT_2; Source: zhihan1996/DNABERT-2-117M model card, <https://huggingface.co/zhihan1996/DNABERT-2-117M>].
- Nucleotide Transformer is an encoder-only transformer family trained with masked language modeling over 6-mer tokens, using a learnable positional encoding with maximum 1,000 tokens, which corresponds to about 6 kb of DNA sequence [Source: Dalla-Torre et al., 2024/2025, "Nucleotide Transformer: building and evaluating robust foundation models for human genomics", <https://www.nature.com/articles/s41592-024-02523-z>].
- The original Nucleotide Transformer family includes 500M and 2.5B models trained on a human reference genome, 3,202 diverse human genomes, and an 850-species multispecies corpus; the paper evaluates embeddings/fine-tuning on 18 genomic prediction tasks and zero-shot variant-effect scoring [Source: Dalla-Torre et al., 2024/2025, "Nucleotide Transformer", <https://www.nature.com/articles/s41592-024-02523-z>].
- The InstaDeepAI/nucleotide-transformer-2.5b-1000g card states that the model is a 2.5B-parameter transformer trained on 3,202 genetically diverse human genomes, tokenized with a 6-mer tokenizer and vocabulary size 4,105, using maximum tokenized sequence length 1,000 and 300B training tokens [Source: InstaDeepAI/nucleotide-transformer-2.5b-1000g model card, <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g>].
- Nucleotide Transformer model cards use the CC-BY-NC-SA-4.0 license, which is non-commercial and therefore unsuitable for commercial production use without separate permission [Source: InstaDeepAI/nucleotide-transformer-2.5b-1000g model card, <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g>].
- Evo 1 is a 7B-parameter autoregressive biological foundation model using the Striped-Hyena architecture, single-nucleotide byte-level tokenization, 131,072-token context for the `evo-1-131k-base` checkpoint, and an OpenGenome prokaryotic whole-genome dataset of about 300B tokens [Source: evo-design/evo GitHub, <https://github.com/evo-design/evo>; Source: Arc Institute Evo blog, <https://arcinstitute.org/news/blog/evo>].

- Evo 1 is directly generative: the repository provides scripts to prompt the model and sample DNA sequences, to score sequence log-likelihoods, and to use the Together API for DNA sequence generation [Source: evo-design/evo GitHub, <https://github.com/evo-design/evo>].
- Evo 1 code and Hugging Face weights are marked Apache-2.0, which is permissive for commercial use subject to normal Apache terms [Source: evo-design/evo GitHub, <https://github.com/evo-design/evo>; Source: togethercomputer/evo-1-131k-base model card, <https://huggingface.co/togethercomputer/evo-1-131k-base/tree/main>].
- Evo 2 is an autoregressive DNA language model using StripedHyena 2, models DNA at single-nucleotide resolution, supports checkpoints up to 1M base-pair context, and was trained on OpenGenome2 containing 8.8T tokens from all domains of life [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>].
- Evo 2 publishes evo2_7b, evo2_7b_262k, evo2_7b_base, evo2_20b, and evo2_40b style checkpoints; the repository notes that 7B models can run without FP8/Transformer Engine while 20B/40B/1B models require FP8-related infrastructure for numerical accuracy [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>].
- Evo 2's NVIDIA BioNeMo page says the 40B model is ready for commercial use, trained on nearly 9T nucleotides, governed by the NVIDIA Open Model License Agreement in the hosted NIM context, and can generate coding-rich sequences up to 1M in length [Source: NVIDIA BioNeMo Evo2 page, <https://docs.nvidia.com/bionemo-framework/2.6/models/evo2/index.html>].
- Evo 2's GitHub repository and Hugging Face model cards are marked Apache-2.0, while NVIDIA-hosted access also adds NVIDIA NIM/software/model terms, so commercial deployments should distinguish local open-weight use from hosted NIM use [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>; Source: arcinstitute/evo2_7b_base model card, https://huggingface.co/arcinstitute/evo2_7b_base; Source: NVIDIA BioNeMo Evo2 page, <https://docs.nvidia.com/bionemo-framework/2.6/models/evo2/index.html>].
- GenSLM uses codon-level tokenization and genome-scale language models trained on nucleotide sequences; the paper reports models from 25M to 25B parameters trained on 110M prokaryotic gene sequences from BV-BRC, with maximum sequence length 2,048 tokens for foundation models and 10,240-token SARS-CoV-2 fine-tuning/evaluation for smaller models [Source: Zvyagin et al., 2022, "GenSLMs: Genome-scale language models reveal SARS-CoV-2 evolutionary dynamics", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9709791/>].
- GenSLM is designed for SARS-CoV-2 evolutionary dynamics and variant-of-concern prediction, not general plasmid generation; the repository is marked MIT, but the PMC-hosted article text is CC-BY-NC-ND 4.0, so model/code/paper licensing must be separated before commercial use [Source: Zvyagin et al., 2022, "GenSLMs", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9709791/>; Source: ramanathanlab/genSLM GitHub, <https://github.com/ramanathanlab/genSLM>].
- HyenaDNA is a Hyena-operator genomic foundation model with single-nucleotide tokenization, sub-quadratic long-range sequence processing, pretrained checkpoints from 1k to 1M tokens, and pretraining on the hg38 human reference genome [Source: Nguyen et al.,

2023, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution", <https://arxiv.org/abs/2306.15794>; Source: HazyResearch/hyena-dna GitHub, <https://github.com/HazyResearch/hyena-dna>].

- HyenaDNA’s repository is Apache-2.0 and offers pretrained weights, but its published workflows emphasize embeddings, fine-tuning, and downstream genomic benchmarks rather than component-conditioned plasmid synthesis [Source: HazyResearch/hyena-dna GitHub, <https://github.com/HazyResearch/hyena-dna>].
- Caduceus is a Mamba-based family that adds bidirectionality and reverse-complement equivariance for DNA, and the paper reports that it outperforms previous long-range models on downstream benchmarks including long-range variant-effect prediction [Source: Schiff et al., 2024, "Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling", <https://arxiv.org/abs/2403.03234>].
- A released Caduceus 131k model card states it was pretrained on the human reference genome with sequence length 131,072 for 50k steps, and Helical’s Caduceus model card lists Apache-2.0 licensing [Source: kuleshov-group/caduceus-ps_seqlen-131k_d_model-256_n_layer-16 model card, https://huggingface.co/kuleshov-group/caduceus-ps_seqlen-131k_d_model-256_n_layer-16/blob/main/README.md; Source: Helical Caduceus model card, https://helical.readthedocs.io/en/stable/model_cards/caduceus/].
- Nucleotide Transformer v3 is described by the official InstaDeep repository as a U-Net-like, single-base-tokenized, multi-species model for contexts up to 1 Mb, trained on 9T base pairs from OpenGenome2 and post-trained on more than 16,000 functional tracks from 24 animal and plant species; it includes controllable sequence-generation work for enhancer design [Source: instadeepai/nucleotide-transformer GitHub, <https://github.com/instadeepai/nucleotide-transformer>].
- Nucleotide Transformer v3’s commercial status is not yet clear from the fetched repository summary, and earlier Nucleotide Transformer Hugging Face checkpoints are CC-BY-NC-SA-4.0, so NTV3 should not be assumed commercially usable until its specific model-card license is verified [Source: instadeepai/nucleotide-transformer GitHub, <https://github.com/instadeepai/nucleotide-transformer>; Source: InstaDeepAI/nucleotide-transformer-2.5b-1000g model card, <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g>].
- GENERator is a 1.2B-parameter long-context generative genomic foundation model with 98k nucleotide context, pretrained on 386B nucleotides of eukaryotic DNA, and evaluated for sequence recovery, variant-effect prediction, and generative applications [Source: Wu et al., 2025, "GENERator: A Long-Context Generative Genomic Foundation Model", <https://arxiv.org/abs/2502.07272>].
- GenomeOcean is a 4B-parameter generative genome foundation model trained on over 600 Gbp of high-quality contigs derived from about 220 TB of metagenomic datasets, using BPE tokenization and reporting BGC discovery/generation capabilities; the fetched PubMed abstract does not establish a commercial-use model license [Source: Zhou et al., 2025, "GenomeOcean: An Efficient Genome Foundation Model Trained on Large-Scale Metagenomic Assemblies", <https://pubmed.ncbi.nlm.nih.gov/39975405/>].

- Carbon-3B is a 3B-parameter decoder-only autoregressive genomic foundation model trained on DNA and RNA with a primary eukaryotic focus, native context of 32,768 6-mer tokens (about 197 kbp), YaRN extension to 65,536 tokens (about 393 kbp), and Apache-2.0 licensing [Source: HuggingFaceBio/Carbon-3B model card, <https://huggingface.co/HuggingFaceBio/Carbon-3B>].
- Carbon-3B uses a hybrid tokenizer with non-overlapping 6-mers for DNA and Qwen3 BPE for English text, requires `<dna>` tags to enter DNA tokenization mode, supports metadata-conditioned generation, and is compatible with vLLM/SGLang-style serving [Source: HuggingFaceBio/Carbon-3B model card, <https://huggingface.co/HuggingFaceBio/Carbon-3B>].
- Carbon’s pretraining corpus card states the released corpus contains 173M DNA/RNA sequences totaling 1.1T nucleotides, with eukaryote, mRNA, mRNA-splice/promoter, and prokaryote subsets whose upstream licenses are MIT or Apache-2.0 [Source: HuggingFaceBio/carbon-pretraining-corpus dataset card, <https://huggingface.co/datasets/HuggingFaceBio/carbon-pretraining-corpus>].

4.0.3 What this means for our build

- Whole-plasmid generation should start from an autoregressive/generative DNA model rather than an encoder-only masked-language model, because the required output is a complete synthesis-ready sequence and Evo/Evo 2/Carbon explicitly support sequence generation while DNABERT, DNABERT-2, Nucleotide Transformer, HyenaDNA, and Caduceus are primarily documented as embedding, masked-LM, or downstream-prediction models [Source: evo-design/evo GitHub, <https://github.com/evo-design/evo>; Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>; Source: HuggingFaceBio/Carbon-3B model card, <https://huggingface.co/HuggingFaceBio/Carbon-3B>; Source: Ji et al., 2021, "DNABERT", <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680>; Source: Dalla-Torre et al., 2024/2025, "Nucleotide Transformer", <https://www.nature.com/articles/s41592-024-02523-z>].
- Evo 2 is the strongest candidate base for whole-plasmid generation because it is autoregressive, single-nucleotide resolution, long-context up to 1M bp, trained across all domains of life, and Apache-2.0 in the open local repository/model-card path [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>; Source: arcinstitute/evo2_7b_base model card, https://huggingface.co/arcinstitute/evo2_7b_base].
- Carbon-3B is a strong near-term baseline or fallback because it is Apache-2.0, decoder-only/autoregressive, much smaller than Evo2-7B, supports about 197-393 kbp contexts, and has explicit metadata-conditioned generation, but it is newer and primarily eukaryote-focused rather than all-domain/plasmid-specific [Source: HuggingFaceBio/Carbon-3B model card, <https://huggingface.co/HuggingFaceBio/Carbon-3B>; Source: HuggingFaceBio/carbon-pretraining-corpus dataset card, <https://huggingface.co/datasets/HuggingFaceBio/carbon-pretraining-corpus>].
- DNABERT-2 should be considered for retrieval embeddings, validation classifiers, or sequence-feature encoders rather than as the primary sequence generator, because the official model card demonstrates embedding extraction and the architecture is BERT-style rather than autoregressive [Source: zhihan1996/DNABERT-2-117M model card,

<https://huggingface.co/zhihan1996/DNABERT-2-117M>; Source: Zhou et al., 2023/2024, "DNABERT-2", <https://arxiv.org/abs/2306.15006>].

- Nucleotide Transformer models should not be used in a commercial build without separate permission because the fetched Hugging Face card is CC-BY-NC-SA-4.0, despite the models being scientifically useful for embeddings and genomic prediction tasks [Source: InstaDeepAI/nucleotide-transformer-2.5b-1000g model card, <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g>].
- Component-aware plasmid generation will require supervised fine-tuning or constrained decoding on annotated plasmid/component data, because none of the reviewed base models is documented as natively taking plasmid feature maps or natural-language component specifications as structured controls [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>; Source: HuggingFaceBio/Carbon-3B model card, <https://huggingface.co/HuggingFaceBio/Carbon-3B>; Source: MAGICSLAB DNABERT_2 GitHub, https://github.com/MAGICSLAB/DNABERT_2].
- The build should expose the base model behind a deterministic generation interface and keep an encoder interface separate, because the best generator candidate and the best embedding/classification candidates are not the same model family [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>; Source: zhihan1996/DNABERT-2-117M model card, <https://huggingface.co/zhihan1996/DNABERT-2-117M>].

4.0.4 Decisions proposed

- Use Evo 2 7B as the default base-model candidate for whole-plasmid generation experiments, with `evo2_7b_base` for 8k-context prototyping and `evo2_7b` or `evo2_7b_262k` for longer-context work when infrastructure permits [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>].
- Benchmark Carbon-3B alongside Evo 2 as the lower-cost Apache-2.0 generative baseline, especially for local development, faster inference, and metadata-conditioning experiments [Source: HuggingFaceBio/Carbon-3B model card, <https://huggingface.co/HuggingFaceBio/Carbon-3B>].
- Do not choose DNABERT, DNABERT-2, Nucleotide Transformer, HyenaDNA, Caduceus, or GenSLM as the primary whole-plasmid generator in Phase 0; evaluate DNABERT-2/HyenaDNA/Caduceus-style encoders separately for retrieval, scoring, and validation if their licenses fit the deployment target [Source: Ji et al., 2021, "DNABERT", <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680>; Source: MAGICSLAB DNABERT_2 GitHub, https://github.com/MAGICSLAB/DNABERT_2; Source: HazyResearch/hyena-dna GitHub, <https://github.com/HazyResearch/hyena-dna>; Source: Schiff et al., 2024, "Caduceus", <https://arxiv.org/abs/2403.03234>; Source: Zvyagin et al., 2022, "GenSLMs", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9709791/>].
- Treat all non-Apache or unclear weight licenses as blockers for commercial production until legal review or author/provider clarification is obtained [Source: InstaDeepAI/nucleotide-transformer-2.5b-1000g model card, <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g>].

[InstaDeepAI/nucleotide-transformer-2.5b-1000g](https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g); Source: zhihan1996/DNABERT-2-117M model card, <https://huggingface.co/zhihan1996/DNABERT-2-117M>; Source: Zvyagin et al., 2022, "GenSLMs", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9709791/>].

4.0.5 Open questions / conflicts

- DNABERT has conflicting commercial-use signals: the paper says the GitHub release is for future academic research, while NVIDIA BioNeMo says the DNABERT model is ready for commercial and non-commercial use [Source: Ji et al., 2021, "DNABERT", <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680>; Source: NVIDIA BioNeMo DNABERT model page, <https://docs.nvidia.com/bionemo-framework/1.10/models/dnabert.html>].
- DNABERT-2's repository is Apache-2.0, but the fetched official Hugging Face model page did not show an explicit license tag for the weights, so commercial use of the exact hosted checkpoint remains unverified [Source: MAGICSLAB DNABERT_2 GitHub, https://github.com/MAGICSLAB/DNABERT_2; Source: zhihan1996/DNABERT-2-117M model card, <https://huggingface.co/zhihan1996/DNABERT-2-117M>].
- Nucleotide Transformer v3 may be technically relevant because it combines 1 Mb context and controllable sequence generation, but the specific checkpoint licenses were not verified in this pass and earlier NT checkpoints are non-commercial CC-BY-NC-SA-4.0 [Source: instadeepai/nucleotide-transformer GitHub, <https://github.com/instadeepai/nucleotide-transformer>; Source: InstaDeepAI/nucleotide-transformer-2.5b-1000g model card, <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g>].
- It is not yet verified how much plasmid-specific sequence, engineered vector sequence, or component-annotation signal exists in Evo 2 OpenGenome2 or Carbon's pretraining data, so component-aware plasmid generation should not be assumed to emerge without fine-tuning and validation [Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>; Source: HuggingFaceBio/carbon-pretraining-corpus dataset card, <https://huggingface.co/datasets/HuggingFaceBio/carbon-pretraining-corpus>].
- GenomeOcean appears relevant for generative microbial/metagenomic work and BGC generation, but the fetched sources did not establish a commercial-use license for model weights or code [Source: Zhou et al., 2025, "GenomeOcean", <https://pubmed.ncbi.nlm.nih.gov/39975405/>].
- Hosted Evo 2 via NVIDIA NIM has NVIDIA-specific terms even though the local Evo 2 repository and Hugging Face model cards are Apache-2.0, so deployment route affects legal review [Source: NVIDIA BioNeMo Evo2 page, <https://docs.nvidia.com/bionemo-framework/2.6/models/evo2/index.html>; Source: ArcInstitute/evo2 GitHub, <https://github.com/ArcInstitute/evo2>].

4.0.6 Sources

1. Yanrong Ji, Zhihan Zhou, Han Liu, Ramana V. Davuluri, 2021, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome", Bioinformatics, <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680>.

2. NVIDIA BioNeMo Framework, "DNABERT" model page, <https://docs.nvidia.com/bionemo-framework/1.10/models/dnabert.html>.
3. Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, Han Liu, 2023/2024, "DNABERT-2: Efficient Foundation Model and Benchmark For Multi-Species Genome", arXiv/ICLR 2024, <https://arxiv.org/abs/2306.15006>.
4. MAGICS-LAB, "DNABERT_2" official GitHub repository, https://github.com/MAGICS-LAB/DNABERT_2.
5. zhihan1996, "DNABERT-2-117M" Hugging Face model card, <https://huggingface.co/zhihan1996/DNABERT-2-117M>.
6. Hugo Dalla-Torre et al., 2024/2025, "Nucleotide Transformer: building and evaluating robust foundation models for human genomics", Nature Methods, <https://www.nature.com/articles/s41592-024-02523-z>.
7. InstaDeepAI, "nucleotide-transformer-2.5b-1000g" Hugging Face model card, <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g>.
8. Eric Nguyen et al., 2024, "Sequence modeling and design from molecular to genome scale with Evo", Science, and evo-design/evo repository, <https://github.com/evo-design/evo>.
9. Arc Institute, 2024, "Evo: DNA foundation modeling from molecular to genome scale", <https://arcinstitute.org/news/blog/evo>.
10. togethercomputer, "evo-1-131k-base" Hugging Face model card, <https://huggingface.co/togethercomputer/evo-1-131k-base/tree/main>.
11. ArcInstitute, "evo2" GitHub repository, <https://github.com/ArcInstitute/evo2>.
12. arcinstitute, "evo2_7b_base" Hugging Face model card, https://huggingface.co/arcinstitute/evo2_7b_base.
13. NVIDIA BioNeMo Framework, "Evo2" model page, <https://docs.nvidia.com/bionemo-framework/2.6/models/evo2/index.html>.
14. Max T. Zvyagin et al., 2022, "GenSLMs: Genome-scale language models reveal SARS-CoV-2 evolutionary dynamics", <https://pmc.ncbi.nlm.nih.gov/articles/PMC9709791/>.
15. ramanathanlab, "genslm" GitHub repository, <https://github.com/ramanathanlab/genslm>.
16. Eric Nguyen et al., 2023, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution", <https://arxiv.org/abs/2306.15794>.
17. HazyResearch, "hyena-dna" GitHub repository, <https://github.com/HazyResearch/hyena-dna>.
18. Yair Schiff et al., 2024, "Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling", <https://arxiv.org/abs/2403.03234>.
19. kuleshov-group, "caduceus-ps_seqlen-131k_d_model-256_n_layer-16" Hugging Face model card, https://huggingface.co/kuleshov-group/caduceus-ps_seqlen-131k_d_model-256_n_layer-16/blob/main/README.md.

20. Helical, "Model Card for Bio-Foundation Model: Caduceus", https://helical.readthedocs.io/en/stable/model_cards/caduceus/.
21. InstaDeepAI, "nucleotide-transformer" GitHub repository, <https://github.com/instadeepai/nucleotide-transformer>.
22. Wei Wu et al., 2025, "GENERator: A Long-Context Generative Genomic Foundation Model", <https://arxiv.org/abs/2502.07272>.
23. Zhihan Zhou et al., 2025, "GenomeOcean: An Efficient Genome Foundation Model Trained on Large-Scale Metagenomic Assemblies", <https://pubmed.ncbi.nlm.nih.gov/39975405/>.
24. HuggingFaceBio, "Carbon-3B" Hugging Face model card, <https://huggingface.co/HuggingFaceBio/Carbon-3B>.
25. HuggingFaceBio, "carbon-pretraining-corpus" dataset card, <https://huggingface.co/datasets/HuggingFaceBio/carbon-pretraining-corpus>.

Chapter 5

Track D: Prior Art & Competing/Related Tools

5.0.1 Scope

This track investigated prior art and competing or related tools for plasmid design, annotation, search, generation, ordering, and automated construction, with emphasis on OriGen, PlasmidGPT, related academic systems, Benchling, SnapGene, VectorBuilder, Asimov Kernel, and Addgene search.

5.0.2 What the literature/sources establish

- OriGen is an autoregressive language model for host-dependent plasmid replicons, where each training/generation example combines host species, Rep protein sequence(s), and oriV nucleotide sequence. [Source: Irvine et al., 2025, "Generating functional plasmid origins with OriGen", <https://doi.org/10.1093/nar/gkaf1198>]
- OriGen expanded a plasmid-replicon dataset by adding 14,424 unique annotated replicons from PLSDB and 12,769 from IMG/PR to 1,023 DoriC examples. [Source: Irvine et al., 2025, "Generating functional plasmid origins with OriGen", <https://doi.org/10.1093/nar/gkaf1198>]
- OriGen can generate new replicon sequences from scratch, generate replicons conditioned on desired host species compatibility, and generate oriVs conditioned on a Rep gene. [Source: Irvine et al., 2025, "Generating functional plasmid origins with OriGen", <https://doi.org/10.1093/nar/gkaf1198>]
- OriGen experimentally validated AI-generated functional origins of replication in E. coli and demonstrated proof-of-principle generation that avoids selected restriction-enzyme recognition motifs. [Source: Irvine et al., 2025, "Generating functional plasmid origins with OriGen", <https://doi.org/10.1093/nar/gkaf1198>]
- OriGen's own discussion states that its validation centered on E. coli, that broader host-specificity testing remains future work, and that extending the approach to whole-plasmid generation still has significant technical hurdles. [Source: Irvine et al., 2025, "Generating functional plasmid origins with OriGen", <https://doi.org/10.1093/nar/gkaf1198>]

- PlasmidGPT is described by its repository as a generative language model pretrained on 153,000 engineered plasmid sequences from Addgene. [Source: Shao, 2024, "PlasmidGPT: a generative framework for plasmid design and annotation", GitHub repository, <https://github.com/lingxusb/PlasmidGPT>]
- PlasmidGPT's repository says the model generates de novo plasmid-like sequences, supports controlled generation from an input sequence or design constraint, and learns embeddings that support prediction of sequence-related attributes. [Source: Shao, 2024, "PlasmidGPT: a generative framework for plasmid design and annotation", GitHub repository, <https://github.com/lingxusb/PlasmidGPT>]
- PlasmidGPT's repository exposes workflows for sequence generation, embeddings, and attribute prediction including lab of origin, species, and vector type. [Source: Shao, 2024, "PlasmidGPT: a generative framework for plasmid design and annotation", GitHub repository, <https://github.com/lingxusb/PlasmidGPT>]
- A Hugging Face compatibility model card lists PlasmidGPT as a GPT-2-style decoder-only transformer with 110M parameters, 12 layers, 768 hidden size, 12 attention heads, 2,048-token context length, and 30,002-token vocabulary. [Source: UCL-CSSB, 2026, "PlasmidGPT (Addgene GPT-2 Compatible Version)", <https://huggingface.co/UCL-CSSB/PlasmidGPT>]
- The original lingxusb/PlasmidGPT Hugging Face page lists license cc-by-nc-4.0, while the GitHub repository displays an MIT license. [Source: Hugging Face, 2026, lingxusb/PlasmidGPT, <https://huggingface.co/lingxusb/PlasmidGPT>; Source: Shao, 2024, GitHub repository, <https://github.com/lingxusb/PlasmidGPT>]
- PlasmidMaker is a Nature Communications system that combines a user-facing plasmid-design frontend, backend processing, PfAgo/artificial-restriction-enzyme assembly, and robotic iBioFAB automation. [Source: Enghiad et al., 2022, "PlasmidMaker is a versatile, automated, and high throughput end-to-end platform for plasmid construction", <https://doi.org/10.1038/s41467-022-30355-y>]
- PlasmidMaker validated construction of 101 plasmids from six species, with sizes from 5 to 18 kb and up to 11 DNA fragments. [Source: Enghiad et al., 2022, "PlasmidMaker is a versatile, automated, and high throughput end-to-end platform for plasmid construction", <https://doi.org/10.1038/s41467-022-30355-y>]
- PlasmidMaker is an automated construction platform, not a natural-language-to-plasmid design system. [Source: Enghiad et al., 2022, "PlasmidMaker is a versatile, automated, and high throughput end-to-end platform for plasmid construction", <https://doi.org/10.1038/s41467-022-30355-y>]
- GenoCAD introduced a formal-language, grammar-based approach to designing and verifying synthetic genetic constructs. [Source: Cai et al., 2010, "GenoCAD: linguistic approaches to synthetic biology", <https://vtechworks.lib.vt.edu/handle/10919/27069>]
- pLannotate accepts FASTA, GenBank, or raw DNA sequence input, queries feature databases, and outputs an interactive plasmid map plus downloadable GenBank or CSV annotations. [Source: McGuffie and Barrick, 2021, "pLannotate: engineered plasmid annotation", <https://doi.org/10.1093/nar/gkab374>]

- pLannotate built a plasmid feature library by cross-referencing 195,426 Addgene GenBank files and deduplicating to 13,240 unique features. [Source: McGuffie and Barrick, 2021, "pLannotate: engineered plasmid annotation", <https://doi.org/10.1093/nar/gkab374>]
- PlasMapper 3.0 accepts pasted DNA, FASTA/raw sequence uploads, existing database plasmids, or examples, and lets users view/edit features, restriction sites, circular/linear maps, sequence text, BLAST results, codon optimization, and PNG/SVG/JSON exports. [Source: Wishart Lab, 2023, "PlasMapper 3.0 help", <https://plasmapper.wishartlab.com/help/>]
- Benchling's Molecular Biology product supports DNA/RNA/amino-acid sequence design and analysis, plasmid or linear maps, annotations, ORFs, primers, restriction enzyme sites, primer design, CRISPR gRNAs, RNA/mRNA design, sequence translation/back-translation, and restriction/Gibson/Golden Gate cloning workflows. [Source: Benchling, 2026, "Molecular Bio Software", <https://www.benchling.com/molecular-biology>]
- SnapGene is an in-silico molecular biology application for planning, simulating, and documenting cloning experiments with annotated plasmid maps, sequence alignment, gel simulation, PCR simulation, and primer design. [Source: SnapGene, 2026, "Discover SnapGene", <https://www.snapgene.com/series/snapgene-tour>]
- SnapGene supports restriction enzyme cloning, Gibson Assembly, Gateway cloning, Golden Gate Assembly, In-Fusion cloning, NEBuilder HiFi Assembly, TOPO cloning, TA/GC cloning, PCR simulation, and primer design. [Source: SnapGene, 2026, "Discover SnapGene", <https://www.snapgene.com/series/snapgene-tour>]
- SnapGene's Gateway workflow can simulate BP/LR reactions with one to four ordered inserts, suggest standard donor vectors, design attB PCR primers, show the predicted expression plasmid, and export primer sequences for ordering. [Source: SnapGene Support, 2025, "Simulate Gateway Cloning with Multiple Inserts", <https://support.snapgene.com/hc/en-us/articles/10384092210708-Simulate-Gateway-Cloning-with-Multiple-Inserts>]
- VectorBuilder provides a web-based vector design and ordering platform where users can customize promoters, ORFs, markers, tags, and other components, perform codon optimization, view annotated maps/sequences, download PDF/GenBank/FASTA/SnapGene reports, and order cloning, virus packaging, or plasmid DNA preparation services. [Source: VectorBuilder, 2026, "Online Vector Design", <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]
- VectorBuilder's design studio covers regular plasmids, lentivirus, AAV, adenovirus, MMLV, piggyBac/PB transposon, and other delivery systems, and it supports applications including overexpression, inducible/conditional expression, shRNA knockdown, CRISPR editing/regulation, recombinant protein expression, in vitro transcription, enhancer/promoter testing, and non-coding RNA expression. [Source: VectorBuilder, 2026, "Online Vector Design", <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]
- VectorBuilder offers free human design support when the user cannot complete a design in the online studio. [Source: VectorBuilder, 2026, "Online Vector Design", <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]

- Asimov Kernel is a genetic design platform combining sequence design tools, collaboration, workflow automation, predictive models, part-based construct design, linear/circular/schematic views, codon optimization, biosecurity sequence analysis, genetic simulation, AI-assisted design support, Addgene repository access, and export tools. [Source: Asimov, 2026, "Welcome to Kernel", <https://docs.kernel.asimov.com/>]
- Asimov Kernel's Compiler automates plasmid design for antibody and protein expression from amino-acid sequences by optimizing codons, predicting and adding signal peptides, selecting vector backbones, assembling required genetic elements, and generating ready-to-order plasmid designs. [Source: Asimov, 2026, "Compiler", <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]
- Asimov's public Kernel page positions the product around model-guided expression optimization, vector-layout recommendations, codon optimization, expression models, reusable vector templates, and data/security controls for commercial teams. [Source: Asimov, 2026, "Kernel", <https://www.asimov.com/kernel>]
- Addgene is a nonprofit repository for plasmids, viral vectors, and antibodies, and its public site exposes gene browsing by official gene name, description, alternate gene name, species, and Entrez Gene ID. [Source: Addgene, 2026, "Browse Genes", <https://www.addgene.org/browse/gene/>]
- Addgene search by sequence performs nucleotide-nucleotide or protein-translated nucleotide BLAST against Addgene plasmid sequence databases, sorts results by E-value, and supports databases such as all plasmids, bacterial expression plasmids, mammalian expression plasmids, plant expression plasmids, yeast expression plasmids, and empty backbones. [Source: Addgene, 2026, "Search by Sequence", <https://www.addgene.org/browse/gene/>]
- Addgene plasmid maps and sequence visualizations are powered by SnapGene Server, and Addgene maps may be based on full lab-provided sequence, Addgene NGS full sequence, partial lab-provided sequence, or partial Sanger sequence. [Source: Addgene Help Center, 2026, "How does Addgene create plasmid maps?", <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]

Capability matrix

OriGen Input style: Host/Rep/oriV-oriented replicon conditioning rather than researcher natural language. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>]

Outputs: Generated oriV/replicon sequences, including E. coli-validated origins. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>]

Generation/automation: Generates plasmid replication components, not full annotated expression plasmids. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>]

Validation/constraints: Demonstrated restriction-site avoidance and experimental E. coli replication validation. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>]

Main stopping point for this build: Does not solve whole-plasmid natural-language design; authors describe whole-plasmid extension as future work with significant hurdles. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>]

PlasmidGPT Input style: DNA start sequence or constraint-oriented prompt, not a full natural-language experimental goal. [Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]

Outputs: Generated FASTA sequences, embeddings, and lab/species/vector-type predictions. [Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]

Generation/automation: Generates de novo plasmid-like sequences pretrained on Addgene plasmids. [Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]

Validation/constraints: Repository emphasizes analysis and prediction; it does not document an end-to-end deterministic synthesis validation report. [Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]

Main stopping point for this build: License status conflicts between GitHub MIT and Hugging Face cc-by-nc-4.0; validated synthesis-ready output is not established by the repository. [Source: Hugging Face, 2026, <https://huggingface.co/lingxusb/PlasmidGPT>; Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]

PlasmidMaker Input style: Web UI where users design plasmids by arranging fragments and selecting/searching templates. [Source: Enghiad et al., 2022, <https://doi.org/10.1038/s41467-022-30355-y>]

Outputs: Constructed physical plasmids verified by restriction digestion/gel electrophoresis. [Source: Enghiad et al., 2022, <https://doi.org/10.1038/s41467-022-30355-y>]

Generation/automation: Automates build/test workflows through PfAgo assembly and robotics. [Source: Enghiad et al., 2022, <https://doi.org/10.1038/s41467-022-30355-y>]

Validation/constraints: Quality checks include guide/primer generation, restriction-enzyme verification planning, and build-stage checks. [Source: Enghiad et al., 2022, <https://doi.org/10.1038/s41467-022-30355-y>]

Main stopping point for this build: Strong automated construction stack, but not a general natural-language design agent. [Source: Enghiad et al., 2022, <https://doi.org/10.1038/s41467-022-30355-y>]

GenoCAD Input style: Rule/grammar-based construct specification. [Source: Cai et al., 2010, <https://vtechworks.lib.vt.edu/handle/10919/27069>]

Outputs: Verifiable synthetic construct designs under formal grammars. [Source: Cai et al., 2010, <https://vtechworks.lib.vt.edu/handle/10919/27069>]

Generation/automation: Rule-based design rather than learned sequence generation. [Source: Cai et al., 2010, <https://vtechworks.lib.vt.edu/handle/10919/27069>]

Validation/constraints: Formal grammar validates allowed part arrangements. [Source: Cai et al., 2010, <https://vtechworks.lib.vt.edu/handle/10919/27069>]

Main stopping point for this build: Useful precedent for constraints, but does not turn free-text experimental goals into complete synthesis-ready plasmids. [Source: Cai et al., 2010, <https://vtechworks.lib.vt.edu/handle/10919/27069>]

pLannotate Input style: FASTA, GenBank, or raw DNA. [Source: McGuffie and Barrick, 2021, <https://doi.org/10.1093/nar/gkab374>]

Outputs: Interactive map plus GenBank/CSV annotations. [Source: McGuffie and Barrick, 2021, <https://doi.org/10.1093/nar/gkab374>]

Generation/automation: Annotates existing plasmids; it does not design new plasmids. [Source: McGuffie and Barrick, 2021, <https://doi.org/10.1093/nar/gkab374>]

Validation/constraints: Detects full and partial feature matches and reports match coverage/identity. [Source: McGuffie and Barrick, 2021, <https://doi.org/10.1093/nar/gkab374>]

Main stopping point for this build: Good annotation component, not a prompt-to-design system. [Source: McGuffie and Barrick, 2021, <https://doi.org/10.1093/nar/gkab374>]

PlasMapper 3.0 Input style: Pasted DNA, FASTA/raw uploads, or chosen database plasmids. [Source: Wishart Lab, 2023, <https://plasmapper.wishartlab.com/help/>]

Outputs: Editable maps, features, restriction sites, BLAST hits, codon optimization, PNG/SVG/JSON exports. [Source: Wishart Lab, 2023, <https://plasmapper.wishartlab.com/help/>]

Generation/automation: Provides editing and annotation utilities, not goal-conditioned design generation. [Source: Wishart Lab, 2023, <https://plasmapper.wishartlab.com/help/>]

Validation/constraints: Supports feature/restriction-site views and limited model-organism codon optimization. [Source: Wishart Lab, 2023, <https://plasmapper.wishartlab.com/help/>]

Main stopping point for this build: Useful visualization/annotation precedent, but no full design synthesis loop. [Source: Wishart Lab, 2023, <https://plasmapper.wishartlab.com/help/>]

Benchling Input style: Manual/team sequence design workflows. [Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>]

Outputs: Plasmid/linear maps, annotations, primers, restriction sites, cloning workflow outputs, registrations, and version history. [Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>]

Generation/automation: Simulates and manages design at scale; public page does not claim natural-language full-plasmid generation. [Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>]

Validation/constraints: Provides in-silico analyses, automated annotation, biochemical property computation, and cloning workflow simulation. [Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>]

Main stopping point for this build: Enterprise sequence workbench/registry, not a plain-English autonomous designer. [Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>]

SnapGene Input style: Manual cloning/sequence files and workflow tools. [Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>]

Outputs: Annotated plasmid maps, cloning simulations, primers, alignments, gels, and history records. [Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>]

Generation/automation: Simulates many cloning methods but does not replace wet-lab work. [Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>]

Validation/constraints: Helps catch design errors before experiments and supports method-specific simulations. [Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>]

Main stopping point for this build: Strong human-operated cloning CAD, not an autonomous NL-to-plasmid system. [Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>]

VectorBuilder Input style: Stepwise web design studio, GOI/component search, pasted sequences, or human design request. [Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]

Outputs: Annotated vector maps/sequences, PDF/GenBank/FASTA/SnapGene reports, price/turnaround, and order options. [Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]

Generation/automation: Semi-automated design and ordering with expert design support. [Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]

Validation/constraints: Provides component choices, codon optimization, vector-system rules, and service-backed QC guarantees. [Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]

Main stopping point for this build: Strongest commercial design/order workflow found, but still component-driven and/or human-assisted rather than full free-text validated generation. [Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]

Asimov Kernel Input style: Part-based construct design and Compiler inputs such as amino acid sequences, molecule format, and expression system. [Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]

Outputs: Complete expression constructs, annotations, linked parts, simulation-ready designs, and ready-to-order plasmid designs. [Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]

Generation/automation: Automates antibody/protein-expression plasmid design with codon optimization, signal peptide prediction, vector selection, and required elements. [Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]

Validation/constraints: Includes sequence validation, reading-frame verification, signal-peptide placement checks, biosecurity sequence analysis, and genetic simulation. [Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>; Source: Asimov, 2026, <https://docs.kernel.asimov.com/>]

Main stopping point for this build: Closest to model-guided production-grade construct design, but public docs frame the Compiler around antibody/protein expression rather than general natural-language plasmid design across arbitrary experimental goals. [Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]

Addgene search Input style: Keyword/gene/category search and BLAST sequence search. [Source: Addgene, 2026, <https://www.addgene.org/browse/gene/>]

Outputs: Existing plasmid records, maps, GenBank/SnapGene downloads where available. [Source: Addgene Help Center, 2026, <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]

Generation/automation: Retrieval only; no custom plasmid generation. [Source: Addgene, 2026, <https://www.addgene.org/browse/gene/>]

Validation/constraints: Search-by-sequence reports BLAST metrics such as E-value, identity, query coverage, bit score, and span. [Source: Addgene, 2026, <https://www.addgene.org/browse/gene/>]

Main stopping point for this build: Excellent corpus/source-of-truth for existing plasmids, but not an adaptive design engine. [Source: Addgene, 2026, <https://www.addgene.org/browse/gene/>]

5.0.3 What this means for our build

- The unsolved gap is a single system that accepts a plain-English experimental goal, retrieves grounded real plasmid evidence, generates or assembles a complete annotated plasmid sequence, runs deterministic validation, designs primers, exports synthesis-ready files, and records provenance; none of the surveyed systems publicly demonstrates that complete end-to-end capability. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>; Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>; Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>; Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>; Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>; Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>; Source: Addgene, 2026, <https://www.addgene.org/browse/gene/>]
- Treat OriGen as evidence that learned generation can produce functional plasmid replication components, but do not assume it can produce complete mammalian, lentiviral, AAV, or multi-component expression vectors without additional retrieval, assembly, validation, and wet-lab evidence. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>]
- Treat PlasmidGPT as relevant prior art for full-plasmid-like sequence modeling and embeddings, but do not depend on it for commercial production until the license conflict and validation status are resolved. [Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>; Source: Hugging Face, 2026, <https://huggingface.co/lingxusb/PlasmidGPT>]
- Borrow from SnapGene, Benchling, PlasMapper, pLannotate, and Addgene for expected user-facing artifacts: annotated maps, feature tables, restriction-site reports, primer records, GenBank/FASTA/SnapGene-compatible exports, and provenance/history. [Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>; Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>; Source: Wishart Lab, 2023, <https://plasmapper.wishartlab.com/help/>; Source: McGuffie and Barrick, 2021, <https://doi.org/10.1093/nar/gkab374>; Source: Addgene Help Center, 2026, <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]
- Borrow from VectorBuilder and Asimov for product expectations around component libraries, codon optimization, vector templates, host/expression-system compatibility, ordering hand-off, and expert rule checks. [Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/>]

[products-services/service/online-vector-design.html/1000](https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000); Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]

- Use Addgene search and maps as retrieval/data-source inspiration, but account for partial or varied sequence provenance because Addgene states maps may come from full sequences, partial Sanger data, lab-provided records, or Addgene NGS. [Source: Addgene Help Center, 2026, <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]

5.0.4 Decisions proposed

- Build a retrieval-first MVP that improves on Addgene keyword/BLAST search by translating natural language into structured design intent and ranking real plasmids with semantic plus structured filters. [Source: Addgene, 2026, <https://www.addgene.org/browse/gene/>]
- Keep generation grounded in retrieved plasmids and explicit component templates because surveyed learned systems either focus on replicons or plasmid-like sequence generation without documented end-to-end deterministic validation. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>; Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]
- Implement a deterministic validation/reporting layer before exposing generated designs as synthesis-ready because existing human-operated CAD tools emphasize in-silico checks, cloning simulation, annotations, and sequence provenance before wet-lab execution. [Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>; Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>; Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>]
- Model the export surface after established tools by supporting GenBank, FASTA, visual maps, feature tables, primer outputs, and order handoff. [Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>; Source: pLannotate, 2021, <https://doi.org/10.1093/nar/gkab374>; Source: Addgene Help Center, 2026, <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]
- Record license status per model and dataset before training or production use because PlasmidGPT source pages currently conflict on license. [Source: Hugging Face, 2026, <https://huggingface.co/lingxusb/PlasmidGPT>; Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]

5.0.5 Open questions / conflicts

- PlasmidGPT licensing is unresolved: the GitHub repository displays MIT license, while the original Hugging Face model page lists cc-by-nc-4.0; commercial use should be blocked until the intended license for weights, tokenizer, data-derived artifacts, and code is confirmed. [Source: Hugging Face, 2026, <https://huggingface.co/lingxusb/PlasmidGPT>; Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>]
- OriGen’s validated scope is E. coli origins/replicons, so whether its architecture or weights help with mammalian expression vectors, lentiviral vectors, AAV vectors, or whole-plasmid

generation remains unresolved. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>]

- Public Asimov Kernel documentation describes a powerful Compiler for antibody/protein expression constructs, but it does not establish whether Kernel accepts unconstrained natural-language experimental goals or supports arbitrary research plasmid classes beyond its documented expression-design workflows. [Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]
- Addgene sequence provenance varies by record, so downstream training/evaluation must distinguish full verified sequences from partial or lab-provided records. [Source: Addgene Help Center, 2026, <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]
- No surveyed source provides a public benchmark for natural-language plasmid design correctness, so this project will need to define its own gold set and acceptance criteria. [Source: Irvine et al., 2025, <https://doi.org/10.1093/nar/gkaf1198>; Source: Shao, 2024, <https://github.com/lingxusb/PlasmidGPT>; Source: Benchling, 2026, <https://www.benchling.com/molecular-biology>; Source: SnapGene, 2026, <https://www.snapgene.com/series/snapgene-tour>; Source: VectorBuilder, 2026, <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>; Source: Asimov, 2026, <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>]

5.0.6 Sources

1. Irvine, J., Martinson, J. N. V., Arora, J., Hasham, S. I., Patel, J. R., Qi, E. T., Cress, B. F., and Rubin, B. E. (2025). "Generating functional plasmid origins with OriGen." *Nucleic Acids Research* 53(22), gkaf1198. <https://doi.org/10.1093/nar/gkaf1198>
2. Shao, B. (2024). "PlasmidGPT: a generative framework for plasmid design and annotation." GitHub repository. <https://github.com/lingxusb/PlasmidGPT>
3. Shao, B. (2024). "PlasmidGPT: a generative framework for plasmid design and annotation." *bioRxiv preprint* DOI 10.1101/2024.09.30.615762. <https://www.biorxiv.org/content/10.1101/2024.09.30.615762v1>
4. UCL-CSSB (2026). "PlasmidGPT (Addgene GPT-2 Compatible Version)." Hugging Face model card. <https://huggingface.co/UCL-CSSB/PlasmidGPT>
5. lingxusb/PlasmidGPT (2026). Hugging Face model page. <https://huggingface.co/lingxusb/PlasmidGPT>
6. Enghiad, B., Xue, P., Singh, N., et al. (2022). "PlasmidMaker is a versatile, automated, and high throughput end-to-end platform for plasmid construction." *Nature Communications* 13, 2697. <https://doi.org/10.1038/s41467-022-30355-y>
7. Cai, Y. (2010). "GenoCAD: linguistic approaches to synthetic biology." Virginia Tech dissertation. <https://vtechworks.lib.vt.edu/handle/10919/27069>
8. McGuffie, M. J. and Barrick, J. E. (2021). "pLannotate: engineered plasmid annotation." *Nucleic Acids Research* 49(W1), W516-W522. <https://doi.org/10.1093/nar/gkab374>

9. Wishart Lab (2023). "PlasMapper 3.0 help." <https://plasmapper.wishartlab.com/help/>
10. Benchling (2026). "Molecular Bio Software: Build, Share & Record DNA Sequences." <https://www.benchling.com/molecular-biology>
11. SnapGene (2026). "Discover SnapGene: Improve Your Molecular Cloning Procedures." <https://www.snapgene.com/series/snapgene-tour>
12. SnapGene Support (2025). "Simulate Gateway Cloning with Multiple Inserts." <https://support.snapgene.com/hc/en-us/articles/10384092210708-Simulate-Gateway-Cloning-with-Multiple-Inserts>
13. VectorBuilder (2026). "Online Vector Design." <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000>
14. Asimov (2026). "Welcome to Kernel." <https://docs.kernel.asimov.com/>
15. Asimov (2026). "Compiler." <https://docs.kernel.asimov.com/compiler-and-simulation/compiler>
16. Asimov (2026). "Kernel." <https://www.asimov.com/kernel>
17. Addgene (2026). "Browse Genes / Search by Sequence." <https://www.addgene.org/browse/gene/>
18. Addgene Help Center (2026). "How does Addgene create plasmid maps?" <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>

Chapter 6

Track E: Data Sources & Access

6.0.1 Scope

This track investigated programmatic access to Addgene and NCBI sequence data, with emphasis on API and bulk-download routes, GenBank/FASTA formats, rate limits, data-use terms for model training, and how each source represents experimental context for plasmid-design retrieval and generation.

6.0.2 What the literature/sources establish

- Addgene’s Developers Portal offers programmatic access through read-only API endpoints and bulk downloads, and its first Catalog endpoint supports plasmids with fields including sequences, expression, vector type, plasmid type, genes, article information, and related data. [Source: Addgene Developers Portal, 2026, "Access Options," <https://developers.addgene.org/access-options/>]
- Addgene bulk downloads are JSON datasets refreshed once per day; Addgene documents JSON properties including UTF-8 encoding, unordered keys, variable whitespace, nullable inapplicable fields, all keys present even when empty/null, and omitted Principal Investigator/Head of Laboratory names. [Source: Addgene Developers Portal, 2026, "Access Options," <https://developers.addgene.org/access-options/>]
- Addgene bulk JSON versions use `majorVersion.minorVersion`; major-version changes can require code changes, while minor-version updates are generally backward-compatible. [Source: Addgene Developers Portal, 2026, "Access Options," <https://developers.addgene.org/access-options/>]
- Addgene access is approval-gated: users request access, specify the requested API scope and intended use, wait for Addgene review, accept a data access license per approved request, then generate a token; Addgene states that token access requires approval and a data access license for each API scope. [Source: Addgene Help Center, 2026, "What is the Developers Portal?," <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal>]
- Addgene states that options are available for both nonprofit and for-profit organizations, that Addgene reviews access requests within five business days, and that generated tokens expire after one year. [Source: Addgene Help Center, 2026,

"What is the Developers Portal?," <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal>]

- Addgene requires users to be logged in to view plasmid sequence data on the website, and directs researchers needing bulk or programmatic sequence access to the Developers Portal. [Source: Addgene Help Center, 2026, "Are there any requirements for viewing plasmid sequence data on the Addgene website?," <https://help.addgene.org/hc/en-us/articles/44210206209549-Are-there-any-requirements-for-viewing-plasmid-sequence-data-on-the-Addgene-website>]
- Addgene says many new deposits since its 2017 seqWell partnership have full plasmid sequence from NGS, while many pre-implementation deposits have only Sanger sequence verifying the insert. [Source: Addgene Help Center, 2026, "Do you have full sequence for my plasmid?," <https://help.addgene.org/hc/en-us/articles/205434259-Do-you-have-full-sequence-for-my-plasmid>]
- Addgene sequence files can vary by source: Addgene identifies full sequences from the creating laboratory, full sequences obtained by Addgene through NGS, partial sequences provided by depositors, and partial sequences from Addgene Sanger sequencing. [Source: Addgene Help Center, 2026, "How does Addgene create plasmid maps?," <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]
- Addgene annotated sequence files are downloadable in GenBank format or SnapGene .dna format from the website, and its maps/tables include restriction enzymes, features, and primers. [Source: Addgene Help Center, 2026, "How does Addgene create plasmid maps?," <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]
- Addgene's API docs define token-authenticated catalog endpoints: `/catalog/plasmid/` lists plasmids and supports query parameters such as `article_pmids`, `genes`, `species`, `experimental_use`, `vector_types`, `expression`, `promoters`, `resistance_marker`, `plasmid_type`, and `is_industry`. [Source: Addgene Developers API, 2026, "catalog_plasmid_list," <https://docs.developers.addgene.org/docs/>]
- Addgene's catalog list response includes fields such as `id`, `name`, `purpose`, `details_url`, `depositor`, `article`, `genes`, `species`, `experimental_use`, `vector_types`, `expression`, `promoters`, `mutations`, and `available_since`. [Source: Addgene Developers API, 2026, "catalog_plasmid_list," <https://docs.developers.addgene.org/docs/>]
- Addgene's single-plasmid catalog response includes `inserts`, `cloning`, `tags`, `bacterial_resistance`, `growth_strain`, `growth_temp`, `growth_notes`, `plasmid_copy`, `resistance_markers`, `origin`, `depositor_comments`, `article`, and `terms`. [Source: Addgene Developers API, 2026, "catalog_plasmid_retrieve," <https://docs.developers.addgene.org/docs/>]
- Addgene's `/download/plasmids/` endpoint requires the `bulk-download:plasmids` scope and returns a JSON object containing a `plasmids` array with plasmid metadata and `terms`. [Source: Addgene Developers API, 2026, "download_plasmids_retrieve," <https://docs.developers.addgene.org/docs/>]
- Addgene's `/download/genbank/{id}/` endpoint downloads a GenBank file for a sequence ID, requires `bulk-download:plasmids-with-sequences` or

`catalog:retrieve-with-sequences`, and returns a 302 redirect to a short-lived pre-signed S3 URL. [Source: Addgene Developers API, 2026, "download_genbank_retrieve," <https://docs.developers.addgene.org/docs/>]

- Addgene's API docs found in this pass document token scopes, pagination, bulk downloads, and S3 redirects but do not publish a numeric request-per-second rate limit. [Source: Addgene Developers API, 2026, "Addgene Developers API," <https://docs.developers.addgene.org/docs/>]
- Addgene's general Terms of Use say site content, including plasmid profiles and sequencing information, is for informational, noncommercial use unless otherwise noted, and content may not be copied, sold, resold, or exploited commercially without express written consent from Addgene or applicable third parties. [Source: Addgene, 2023, "Terms of Use," <https://www.addgene.org/terms-of-use/>]
- Addgene's Developers Portal says Addgene provides access to support research and the development of applications, algorithms, products, synthetic-biology tools, and more, but access still requires the approved scope-specific data access license. [Source: Addgene Developers Portal, 2026, "Access Plasmid and Sequence Data via API," <https://developers.addgene.org/>]
- NCBI E-utilities are a fixed-URL programmatic interface to Entrez, which includes biomedical databases such as nucleotide and protein sequences, gene records, molecular structures, and biomedical literature. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- NCBI says E-utility requests should use URLs beginning with <https://eutils.ncbi.nlm.nih.gov/entrez/eutils/>, because those servers are optimized for E-utilities traffic. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- NCBI recommends no more than three E-utility URL requests per second without an API key, recommends large jobs on weekends or between 9:00 PM and 5:00 AM Eastern time on weekdays, and may block noncompliant IP addresses. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- NCBI states that E-utility software should register `tool` and `email` values with NCBI and include both values in subsequent requests from that software package. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- NCBI API keys allow E-utilities callers to post up to 10 requests per second by default, while higher E-utilities rates are available by request. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- NCBI recommends Entrez History for large retrievals, because many thousands of IDs can be uploaded with one EPost request and several hundred records can be downloaded with one EFetch request. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]

- EFetch supports retrieval of nucleotide FASTA, GenBank flat file (`rettype=gb`), GBSeqXML, and sequence records by accession.version identifiers. [Source: Sayers, 2009/2022, "The E-utilities In-Depth: Parameters, Syntax and More," <https://www.ncbi.nlm.nih.gov/books/NBK25499/>]
- NCBI uses `accession.version` as the primary identifier for nucleotide and protein sequence records, while E-utilities continue to accept GI numbers or accession.version identifiers for sequence databases. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- GenBank is an annotated collection of publicly available DNA sequences, is part of the INSDC with DDBJ and ENA, and the three organizations exchange data daily. [Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>]
- GenBank releases occur every two months and are available from NCBI FTP; ASN.1 and flatfile formats are available through NCBI anonymous FTP. [Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>]
- NCBI also documents anonymous FTP access to GenBank ASN.1 and flatfile data and points users to current GenBank release notes for details on available data. [Source: NCBI, 2017, "FTP access to GenBank data," <https://www.ncbi.nlm.nih.gov/genbank/ftp/>]
- NCBI places no restrictions on use or distribution of GenBank data, but states that some submitters may claim patent, copyright, or other intellectual-property rights in submitted data and that NCBI cannot assess those claims or grant unrestricted permission for them. [Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>]
- NCBI warns that PubMed abstracts may contain copyrighted material and says anyone reproducing, redistributing, or commercially using such information is expected to follow copyright-holder terms. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- NCBI's FASTA submission guidance says a nucleotide FASTA definition line begins with `>`, includes a unique sequence identifier, can include source modifiers in [`modifier=text`] form, and is followed by nucleotide sequence using IUPAC symbols. [Source: NCBI, 2025, "FASTA Format for Nucleotide Sequences," <https://www.ncbi.nlm.nih.gov/genbank/fastafORMAT/>]
- The DDBJ/ENA/GenBank Feature Table is the shared standard for sequence-feature annotation across DDBJ, ENA, and GenBank, and it defines feature vocabulary for regions that perform biological functions, affect expression, interact with molecules, affect replication or recombination, or represent repeats/variation. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- The INSDC Feature Table's artificial cloning-vector example represents host/source context with `source` features, `/organism`, `/lab_host`, `/mol_type`, and `/focus`, and represents functional elements with features such as `CDS` for marker or insert genes. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]

- The INSDC Feature Table defines **regulatory** as a feature for regions regulating transcription, translation, replication, recombination, or chromatin structure, and requires `/regulatory_class="TYPE"` for that feature. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- The INSDC Feature Table defines **rep_origin** as the origin of replication feature and allows qualifiers such as `/direction`, `/experiment`, `/function`, `/gene`, and `/note`. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- The INSDC Feature Table defines `/plasmid` as the name of a naturally occurring plasmid source for a sequence, where plasmid is an independently replicating genetic unit not described by `/chromosome` or `/segment`. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- NCBI Datasets CLI can download genome data packages that include genomic FASTA, GenBank flat files (**gbff**), GFF3/GTF annotations, protein/CDS/RNA sequence files, sequence reports, and JSONL assembly metadata, but this is documented as a genome/assembly package route rather than an Entrez Nucleotide plasmid-record route. [Source: NCBI Datasets, 2026, "datasets download genome reference," <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/command-line/datasets/download/genome/>]

6.0.3 What this means for our build

- Treat Addgene as the best contextual plasmid source for retrieval because its API exposes plasmid purpose, experimental use, genes, species, vector types, expression, promoters, resistance markers, article metadata, cloning metadata, depositor comments, and terms. [Source: Addgene Developers API, 2026, "catalog_plasmid_list" and "catalog_plasmid_retrieve," <https://docs.developers.addgene.org/docs/>]
- Implement Addgene ingestion as an approval-gated connector that can run only when `ADDGENE_API_TOKEN` is present and the accepted data access license allows the intended use, because access requires approved scopes and a scope-specific license. [Source: Addgene Help Center, 2026, "What is the Developers Portal?," <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal>]
- Do not assume Addgene data can be used for commercial model training merely because the data is technically accessible, because Addgene's general Terms of Use restrict commercial exploitation without consent and the Developers Portal requires a data access license for each scope. [Source: Addgene, 2023, "Terms of Use," <https://www.addgene.org/terms-of-use/>; Source: Addgene Help Center, 2026, "What is the Developers Portal?," <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal>]
- Store Addgene **terms** and API scope/provenance per plasmid record, because the API exposes a **terms** field and Addgene access is license-scoped. [Source: Addgene Developers API, 2026, "download_plasmids_retrieve" and "catalog_plasmid_retrieve," <https://docs.developers.addgene.org/docs/>]
- Expect Addgene sequence completeness to be heterogeneous and store sequence provenance and completeness flags, because Addgene distinguishes NGS full sequences, depositor full sequences, depositor partial sequences, and Sanger insert-

verification sequences. [Source: Addgene Help Center, 2026, "Do you have full sequence for my plasmid?," <https://help.addgene.org/hc/en-us/articles/205434259-Do-you-have-full-sequence-for-my-plasmid>; Source: Addgene Help Center, 2026, "How does Addgene create plasmid maps?," <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]

- Normalize Addgene GenBank downloads into our `AnnotatedSequence` schema but preserve raw GenBank files, because Addgene's sequence download endpoint returns GenBank files and Addgene maps include features, restriction enzymes, and primers. [Source: Addgene Developers API, 2026, "download_genbank_retrieve," <https://docs.developers.addgene.org/docs/>; Source: Addgene Help Center, 2026, "How does Addgene create plasmid maps?," <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>]
- Implement NCBI ingestion with ESearch/EPost/EFetch and Entrez History batching, because NCBI documents those pipelines for retrieval and recommends History batching for large downloads. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- Configure NCBI clients with registered `tool` and `email` parameters, respect three requests per second without an API key and 10 requests per second with an API key, and schedule large jobs in NCBI's recommended off-peak windows. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- Use `rettype=gb` for annotated ingestion and `rettype=fasta` only when raw sequence is sufficient, because EFetch supports both GenBank flat files and FASTA for nucleotide records. [Source: Sayers, 2009/2022, "The E-utilities In-Depth: Parameters, Syntax and More," <https://www.ncbi.nlm.nih.gov/books/NBK25499/>]
- Use NCBI GenBank as the broad sequence corpus and Addgene as the stronger experimental-context corpus, because GenBank is a sequence/annotation repository while Addgene exposes explicit plasmid purpose, experimental-use, catalog, cloning, and article fields. [Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>; Source: Addgene Developers API, 2026, "catalog_plasmid_list" and "catalog_plasmid_retrieve," <https://docs.developers.addgene.org/docs/>]
- Parse INSDC features into canonical component candidates: `regulatory` with `/regulatory_class`, `rep_origin`, `CDS`, `repeat_region`, and `source` qualifiers should become first-pass signals for promoter/RBS/terminator, ORI, GOI/marker, repeat, and host/source context. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- Record GenBank legal/provenance fields separately from model-training eligibility, because NCBI places no restrictions on GenBank data but flags possible submitter IP claims. [Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>]
- Avoid using PubMed abstracts as unconstrained training text, because NCBI warns that abstracts may contain copyrighted material and large PubMed data mining should use the appropriate local-copy route. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]

6.0.4 Decisions proposed

- Build two separate ingestion adapters: `AddgeneClient` for approved token-scoped Addgene API/bulk data and `NcbiEntrezClient` for public Entrez/E-utilities access with registered `tool`, `email`, `api_key`, `retry`, and `rate-limit` configuration. [Source: Addgene Help Center, 2026, "What is the Developers Portal?," <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal>; Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- Make Addgene ingestion disabled by default in CI and local setup unless `ADDGENE_API_TOKEN` and an explicit `accepted-license` marker are configured, because Addgene access is approval-gated and license-scoped. [Source: Addgene Help Center, 2026, "What is the Developers Portal?," <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal>]
- Make NCBI E-utilities available by default with conservative unauthenticated throttling and optional `NCBI_API_KEY`, because NCBI permits public E-utilities access but rate-limits unauthenticated callers more strictly than API-key callers. [Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]
- Persist raw source artifacts for both sources before parsing: Addgene JSON/GenBank and NCBI GenBank/FASTA/GBSeqXML, because Addgene and NCBI both expose structured source formats that may need re-parsing as schemas improve. [Source: Addgene Developers API, 2026, "download_plasmids_retrieve" and "download_genbank_retrieve," <https://docs.developers.addgene.org/docs/>; Source: Sayers, 2009/2022, "The E-utilities In-Depth: Parameters, Syntax and More," <https://www.ncbi.nlm.nih.gov/books/NBK25499/>]
- Use GenBank flat files as the canonical annotated-sequence ingest format and FASTA as a secondary raw-sequence fallback, because GenBank/INSDC carries feature coordinates and qualifiers while FASTA carries only a definition line plus sequence. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>; Source: NCBI, 2025, "FASTA Format for Nucleotide Sequences," <https://www.ncbi.nlm.nih.gov/genbank/fastafORMAT/>]
- Add source-level license fields to the Phase 0 schema now: `license_status`, `terms_ref`, `training_use_allowed`, `commercial_use_allowed`, and `review_required`, because Addgene license eligibility depends on the approved data-access license and GenBank has possible submitter IP caveats. [Source: Addgene, 2023, "Terms of Use," <https://www.addgene.org/terms-of-use/>; Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>]
- Use Addgene fields to construct retrieval documents with explicit experimental context, and use NCBI fields primarily for sequence annotations plus linked publications/context extraction. [Source: Addgene Developers API, 2026, "catalog_plasmid_list" and "catalog_plasmid_retrieve," <https://docs.developers.addgene.org/docs/>; Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>]

6.0.5 Open questions / conflicts

- Addgene’s public developer/API docs found in this pass do not publish a numeric API rate limit, so the build needs either Addgene guidance from the approved data access license or a conservative internal throttle until Addgene confirms limits. [Source: Addgene Developers API, 2026, "Addgene Developers API," <https://docs.developers.addgene.org/docs/>]
- Addgene’s public Terms of Use restrict noncommercial/general site content use, while the Developers Portal is explicitly intended to support applications, algorithms, products, and synthetic-biology tools under scope-specific licenses; the exact training/commercial-use permission must be read from the accepted data access license before any Addgene-derived training set is built. [Source: Addgene, 2023, "Terms of Use," <https://www.addgene.org/terms-of-use/>; Source: Addgene Developers Portal, 2026, "Access Plasmid and Sequence Data via API," <https://developers.addgene.org/>]
- Addgene sequence records are heterogeneous across NGS full plasmid sequence, depositor full sequence, depositor partial sequence, and Sanger insert verification, so the parser needs a source-completeness model before treating an Addgene record as synthesis-ready training target. [Source: Addgene Help Center, 2026, "Do you have full sequence for my plasmid?," <https://help.addgene.org/hc/en-us/articles/205434259-Do-you-have-full-sequence-for-my-plasmid>]
- NCBI places no restrictions on GenBank data but flags possible submitter patent/copyright/IP claims, so commercial model-training clearance for GenBank-derived records remains a legal review question rather than a purely technical decision. [Source: NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>]
- NCBI Datasets can download `gbff` and FASTA for genome assemblies, but it is not documented as a complete replacement for Entrez Nucleotide retrieval of individual plasmid records, so Phase 0 should use E-utilities for record-level plasmid retrieval and evaluate Datasets only for assembly-level bulk needs. [Source: NCBI Datasets, 2026, "datasets download genome reference," <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/command-line/datasets/download/genome/>; Source: Sayers, 2009/2022, "A General Introduction to the E-utilities," <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>]

6.0.6 Sources

1. Addgene, 2026, "Access Plasmid and Sequence Data via API," <https://developers.addgene.org/>
2. Addgene Developers Portal, 2026, "Access Options," <https://developers.addgene.org/access-options/>
3. Addgene Developers API, 2026, "Addgene Developers API," <https://docs.developers.addgene.org/docs/>
4. Addgene Help Center, 2026, "What is the Developers Portal?," <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal>

5. Addgene Help Center, 2026, "Are there any requirements for viewing plasmid sequence data on the Addgene website?," <https://help.addgene.org/hc/en-us/articles/44210206209549-Are-there-any-requirements-for-viewing-plasmid-sequence-data-on-the-Addgene-website>
6. Addgene Help Center, 2026, "Do you have full sequence for my plasmid?," <https://help.addgene.org/hc/en-us/articles/205434259-Do-you-have-full-sequence-for-my-plasmid>
7. Addgene Help Center, 2026, "How does Addgene create plasmid maps?," <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps>
8. Addgene, 2023, "Terms of Use," <https://www.addgene.org/terms-of-use/>
9. Eric Sayers, 2009, updated 2022, "A General Introduction to the E-utilities," NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/>
10. Eric Sayers, 2009, updated 2022, "The E-utilities In-Depth: Parameters, Syntax and More," NCBI Bookshelf, <https://www.ncbi.nlm.nih.gov/books/NBK25499/>
11. NCBI, 2026, "GenBank Overview," <https://www.ncbi.nlm.nih.gov/genbank/genbank/>
12. NCBI, 2017, "FTP access to GenBank data," <https://www.ncbi.nlm.nih.gov/genbank/ftp/>
13. NCBI, 2025, "FASTA Format for Nucleotide Sequences," <https://www.ncbi.nlm.nih.gov/genbank/fastafmt/>
14. DDBJ/ENA/GenBank, 2026, "The DDBJ/ENA/GenBank Feature Table Definition," <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>
15. NCBI Datasets, 2026, "datasets download genome reference," <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/command-line/datasets/download/genome/>

Chapter 7

Track F: Sequence Representation & Tokenization

7.0.1 Scope

This track investigated DNA tokenization and embedding choices for machine learning, annotation and feature-coordinate representation, circular-topology handling for linear sequence models, and the representation strategy needed by retrieval, generation, and validation for a prompt-to-plasmid design system. [Source: Ji et al., 2021, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome", <https://doi.org/10.1093/bioinformatics/btab083>; Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>; DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]

7.0.2 What the literature/sources establish

- DNABERT tokenizes DNA as overlapping k-mers, evaluates k values of 3, 4, 5, and 6, and defines each DNABERT-k vocabulary as all 4^k nucleotide k-mers plus five special tokens. [Source: Ji et al., 2021, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome", <https://doi.org/10.1093/bioinformatics/btab083>]
- Overlapping k-mer tokenization gives adjacent tokens k - 1 shared bases, which DNABERT-2 identifies as information leakage for masked language modeling. [Source: Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>]
- Non-overlapping k-mer tokenization reduces sequence length by a factor of k, but DNABERT-2 reports that a one-base shift can produce a very different token sequence for a near-identical genomic segment. [Source: Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>]
- DNABERT-2 replaces k-mer tokenization with Byte Pair Encoding (BPE), constructs variable-length genome tokens by merging frequent nucleotide or segment pairs, and selects a 4096-token vocabulary as its best balance of performance and computational efficiency

among tested sizes. [Source: Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>]

- The Nucleotide Transformer uses 6-mer tokens, an encoder-only transformer architecture, masked language modeling, and a learned positional encoding layer for models accepting up to 1000 tokens, with NT-v2 using rotary embeddings and accepting up to 2048 tokens. [Source: Dalla-Torre et al., 2024, "Nucleotide Transformer: building and evaluating robust foundation models for human genomics", <https://doi.org/10.1038/s41592-024-02523-z>]
- HyenaDNA uses single-nucleotide tokens, reports context lengths up to 1 million tokens, and argues that fixed k-mer tokenizers can lose single-nucleotide resolution relevant to SNP-scale variation. [Source: Nguyen et al., 2023, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution", <https://doi.org/10.48550/arXiv.2306.15794>]
- Evo uses a byte-level, single-nucleotide tokenizer and is trained on a prokaryotic whole-genome dataset of 300 billion nucleotides. [Source: Nguyen et al., 2024, "Sequence modeling and design from molecular to genome scale with Evo", <https://doi.org/10.1126/science.ad9336>]
- GROVER applies BPE to the human genome and selects its vocabulary through an intrinsic next-k-mer prediction task rather than a single downstream biological task. [Source: Sanabria et al., 2024, "DNA language model GROVER learns sequence context in the human genome", <https://doi.org/10.1038/s42256-024-00872-0>]
- The 2026 EvoLen preprint argues that DNA lacks natural token boundaries and that tokenization imposes an inductive bias; it proposes evolutionary-signal-guided tokenization to preserve motif-scale functional sequence units better than plain BPE in its experiments. [Source: Huang et al., 2026, "EvoLen: Evolution-Guided Tokenization for DNA Language Model", <https://doi.org/10.48550/arXiv.2604.08698>]
- DNATokenizer, a 2026 systems preprint, reports that tokenization choice interacts with model architecture and task requirements, and that high-throughput DNA tokenization can become a production bottleneck when inputs reach billions of bases. [Source: Niktab and Patel, 2026, "DNATokenizer: A GPU-First Byte-to-Identifier Tokenizer for High-Throughput DNA Language Models", <https://doi.org/10.48550/arXiv.2601.05531>]
- The INSDC feature table used by DDBJ, ENA, and GenBank represents sequence annotations as feature keys with locations and qualifiers, and its location syntax uses one-based numbering where base 1 is the first base of the presented sequence. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- INSDC location syntax represents continuous ranges, fuzzy bounds, between-base sites, joined segments, reverse-complement features, and remote-entry references using operators such as `join(...)`, `complement(...)`, and `order(...)`. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- Biopython's `FeatureLocation` stores simple features with Python-style zero-based coordinates, converts a GenBank one-based inclusive range such as 123..150 into [122:150], and expects `start <= end` even for reverse-strand features. [Source: Biopython, 2026, "Bio.SeqFeature module documentation", <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html>]

- Biopython's `CompoundLocation` is used for non-continuous features such as a coding sequence made of exons, and Biopython documents feature length so that gaps are excluded and origin-wrapping compound locations work correctly. [Source: Biopython, 2026, "Bio.SeqFeature module documentation", <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html>]
- NCBI GFF3 uses one-based start and end coordinates, uses column 7 for strand, uses column 8 for CDS phase, and represents multi-row features by repeating the same ID. [Source: NCBI, 2026, "GFF3 format", <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>]
- NCBI GFF3 extends column 5 into virtual space when an interval spans the origin of a circular sequence. [Source: NCBI, 2026, "GFF3 format", <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>]
- Plasmids are commonly circular DNA molecules that replicate independently from chromosomal DNA. [Source: Addgene, 2026, "Plasmids 101 topic overview", <https://info.addgene.org/plasmids-101-topic-page>]
- Benchling supports circular DNA sequence alignment workflows, including whole-plasmid alignment and cross-origin short-read alignment against a circular DNA template. [Source: Benchling, 2025, "Creating consensus and template alignments on DNA/RNA sequences", <https://help.benchling.com/hc/en-us/articles/9684273213325-Creating-consensus-and-template-alignments-on-DNA-RNA-sequences>]
- Reverse-complement symmetry matters for DNA sequence models because predictive models mapping double-stranded DNA to regulatory activity should produce analogous predictions for a sequence and its reverse complement, but standard neural networks can diverge across strands. [Source: Zhou et al., 2022, "Towards a Better Understanding of Reverse-Complement Equivariance for Deep Learning Models in Genomics", <https://proceedings.mlr.press/v165/zhou22a.html>]
- Zhou et al. recommend post-hoc conjoined reverse-complement models as a reliable baseline before exploring reverse-complement parameter sharing, because their benchmarks found post-hoc conjoined models consistently best or second-best. [Source: Zhou et al., 2022, "Towards a Better Understanding of Reverse-Complement Equivariance for Deep Learning Models in Genomics", <https://proceedings.mlr.press/v165/zhou22a.html>]
- Caduceus builds on Mamba with bidirectional and reverse-complement-equivariant components, and presents these properties as responses to long-range context and reverse-complementarity challenges in genomic sequence modeling. [Source: Schiff et al., 2024, "Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling", <https://arxiv.org/abs/2403.03234>]

7.0.3 What this means for our build

- Retrieval should store more than one representation per plasmid: full-sequence embeddings for global similarity, component-level chunks for promoter/ORI/marker/GOI retrieval, and structured metadata filters for host, topology, vector class, organism, feature type, and source provenance. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition",

<https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>; Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>]

- Retrieval embeddings should preserve deterministic links from every embedded window or component back to source accession, topology, strand, zero-based half-open coordinates, and original GenBank/GFF3 location text so validation and visualization can audit each match. [Source: Biopython, 2026, "Bio.SeqFeature module documentation", <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html>; DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- Generation should maintain a canonical internal feature model separate from the raw DNA string, because GenBank/INSDC annotations encode feature type, strand, joins, complements, fuzzy bounds, and qualifiers that a sequence-only representation does not preserve. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- The internal coordinate system should be zero-based half-open with explicit conversion functions for GenBank and GFF3, because Biopython uses zero-based Python-style locations while INSDC and NCBI GFF3 files expose one-based coordinates. [Source: Biopython, 2026, "Bio.SeqFeature module documentation", <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html>; NCBI, 2026, "GFF3 format", <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>]
- Circular plasmids should be represented with an explicit `topology = circular` flag and origin-aware coordinate utilities, because circular features can wrap the origin and NCBI GFF3 handles origin-spanning intervals by extending the end coordinate into virtual space. [Source: Addgene, 2026, "Plasmids 101 topic overview", <https://info.addgene.org/plasmids-101-topic-page>; NCBI, 2026, "GFF3 format", <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>]
- Linear model inputs for circular plasmids should be augmented by origin-rotated windows or sequence duplication around the origin for cross-origin context, because circular plasmids have no biologically privileged sequence break and Benchling's cross-origin alignment feature reflects the practical need to match across that boundary. [Source: Addgene, 2026, "Plasmids 101 topic overview", <https://info.addgene.org/plasmids-101-topic-page>; Benchling, 2025, "Creating consensus and template alignments on DNA/RNA sequences", <https://help.benchling.com/hc/en-us/articles/9684273213325-Creating-consensus-and-template-alignments-on-DNA-RNA-sequences>]
- Validation should run on exact bases and exact feature coordinates rather than only on model tokens, because BPE and non-overlapping k-mer representations can compress or shift token boundaries while restriction sites, primer sites, CDS frame, and feature overlaps are coordinate-level properties. [Source: Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>; DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]

- If the base generator is a BPE or k-mer model, the build still needs a base-level postprocessor and validator, because HyenaDNA and Evo show that single-nucleotide resolution is an explicit design objective for some genomic foundation models and coordinate-level edits cannot be delegated to token boundaries. [Source: Nguyen et al., 2023, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution", <https://doi.org/10.48550/arXiv.2306.15794>; Nguyen et al., 2024, "Sequence modeling and design from molecular to genome scale with Evo", <https://doi.org/10.1126/science.ad09336>]
- Reverse-complement handling should be an explicit representation concern for embeddings and validators, because standard models can produce divergent outputs for reverse-complement inputs and Caduceus treats reverse-complement equivariance as an architectural requirement for DNA sequence modeling. [Source: Zhou et al., 2022, "Towards a Better Understanding of Reverse-Complement Equivariance for Deep Learning Models in Genomics", <https://proceedings.mlr.press/v165/zhou22a.html>; Schiff et al., 2024, "Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling", <https://arxiv.org/abs/2403.03234>]

7.0.4 Decisions proposed

- Use a canonical `PlasmidRecord` representation with `sequence`, `topology`, `features`, `source`, and `provenance`, where feature intervals are zero-based half-open internally and exporters convert to GenBank/GFF3 one-based conventions. [Source: Biopython, 2026, "Bio.SeqFeature module documentation", <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html>; NCBI, 2026, "GFF3 format", <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>]
- Store each feature with `type`, `label`, `strand`, `parts`, `qualifiers`, `source_location`, and `evidence/provenance`, because INSDC feature tables combine feature keys, locations, operators, and qualifiers to describe biological sequence elements. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>]
- Represent multi-part and origin-wrapping features as ordered interval lists instead of forcing one start/end pair, because INSDC supports `join(...)` and Biopython uses `CompoundLocation` for non-continuous features and origin wrapping. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>; Biopython, 2026, "Bio.SeqFeature module documentation", <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html>]
- For retrieval, initially embed both BPE-tokenized sequence windows and structured feature-text documents, then keep the tokenizer behind an interface so later experiments can compare DNABERT-2-style BPE, 6-mer models, and single-nucleotide models. [Source: Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>; Dalla-Torre et al., 2024, "Nucleotide Transformer: building and evaluating robust foundation models for human genomics", <https://doi.org/10.1038/s41592-024-02523-z>; Nguyen et al., 2023, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution", <https://doi.org/10.48550/arXiv.2306.15794>]

- For generation, prefer a component-aware sequence plan plus base-level sequence assembly over an unconstrained one-shot token stream, because the target output needs exact DNA plus exact feature annotations and validation must inspect base-level sites and coordinates. [Source: DDBJ/ENA/GenBank, 2026, "Feature Table Definition", <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>; Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>]
- For circular topology, implement deterministic utilities for rotation, cross-origin slicing, coordinate normalization, and origin-preserving export before training or evaluating any learned circular-sequence behavior. [Source: NCBI, 2026, "GFF3 format", <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>; Benchling, 2025, "Creating consensus and template alignments on DNA/RNA sequences", <https://help.benchling.com/hc/en-us/articles/9684273213325-Creating-consensus-and-template-alignments-on-DNA-RNA-sequences>]
- For reverse complements, canonicalize exact-match retrieval keys with a strand-aware policy while keeping display/export strand unchanged, and evaluate embedding models with forward/reverse-complement pairs before using them for biological similarity. [Source: Zhou et al., 2022, "Towards a Better Understanding of Reverse-Complement Equivariance for Deep Learning Models in Genomics", <https://proceedings.mlr.press/v165/zhou22a.html>]

7.0.5 Open questions / conflicts

- DNABERT-2 and GROVER support BPE for efficient genomic tokenization, while HyenaDNA and Evo favor single-nucleotide/byte-level resolution for long-context modeling, so the best tokenizer for whole-plasmid generation remains a model-selection question rather than a settled biological fact. [Source: Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>; Sanabria et al., 2024, "DNA language model GROVER learns sequence context in the human genome", <https://doi.org/10.1038/s42256-024-00872-0>; Nguyen et al., 2023, "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution", <https://doi.org/10.48550/arXiv.2306.15794>; Nguyen et al., 2024, "Sequence modeling and design from molecular to genome scale with Evo", <https://doi.org/10.1126/science.ado9336>]
- EvoLen is a recent preprint that reports biologically guided tokenization benefits, but it has not yet established itself as a standard for plasmid-scale generation or retrieval. [Source: Huang et al., 2026, "EvoLen: Evolution-Guided Tokenization for DNA Language Model", <https://doi.org/10.48550/arXiv.2604.08698>]
- The canonical origin for a designed circular plasmid is a product decision: sources establish that circular topology can wrap across the origin, but they do not prescribe whether this build should place base 1 at the ORI, a cloning site, an imported source origin, or a synthesis-vendor-preferred point. [Source: NCBI, 2026, "GFF3 format", <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>; Addgene, 2026, "Plasmids 101 topic overview", <https://info.addgene.org/plasmids-101-topic-page>]
- The retrieval embedding strategy needs empirical evaluation on plasmid tasks, because the cited tokenization papers benchmark genomic tasks and do not directly answer which embedding granularity best retrieves functionally interchangeable plasmid components. [Source:

Ji et al., 2021, "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome", <https://doi.org/10.1093/bioinformatics/btab083>; Zhou et al., 2024, "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome", <https://arxiv.org/abs/2306.15006>; Dalla-Torre et al., 2024, "Nucleotide Transformer: building and evaluating robust foundation models for human genomics", <https://doi.org/10.1038/s41592-024-02523-z>

- Reverse-complement equivariance has multiple implementation strategies, and Zhou et al. report strong performance for post-hoc conjoined models while Caduceus builds equivariance into the architecture, so this build should benchmark rather than assume one approach. [Source: Zhou et al., 2022, "Towards a Better Understanding of Reverse-Complement Equivariance for Deep Learning Models in Genomics", <https://proceedings.mlr.press/v165/zhou22a.html>; Schiff et al., 2024, "Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling", <https://arxiv.org/abs/2403.03234>]

7.0.6 Sources

1. Ji, Yanrong, Zhihan Zhou, Han Liu, and Ramana V. Davuluri. 2021. "DNABERT: pre-trained Bidirectional Encoder Representations from Transformers model for DNA-language in genome." *Bioinformatics*. <https://doi.org/10.1093/bioinformatics/btab083>
2. Zhou, Zhihan, Yuyang Ji, Weiyang Wang, Pratik Dutta, Ramana V. Davuluri, and Han Liu. 2024. "DNABERT-2: Efficient Foundation Model and Benchmark for Multi-Species Genome." *ICLR 2024 / arXiv*. <https://arxiv.org/abs/2306.15006>
3. Dalla-Torre, Hugo, Liam Gonzalez, Javier Mendoza-Revilla, et al. 2024. "Nucleotide Transformer: building and evaluating robust foundation models for human genomics." *Nature Methods*. <https://doi.org/10.1038/s41592-024-02523-z>
4. Nguyen, Eric, Michael Poli, Marjan Faizi, et al. 2023. "HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution." *NeurIPS 2023 / arXiv*. <https://doi.org/10.48550/arXiv.2306.15794>
5. Nguyen, Eric, Michael Poli, Matthew G. Durrant, et al. 2024. "Sequence modeling and design from molecular to genome scale with Evo." *Science*. <https://doi.org/10.1126/science.ado9336>
6. Sanabria, Melissa, Jonas Hirsch, Pierre M. Joubert, et al. 2024. "DNA language model GROVER learns sequence context in the human genome." *Nature Machine Intelligence*. <https://doi.org/10.1038/s42256-024-00872-0>
7. Huang, Nan, Xiaoxiao Zhou, Junxia Cui, Mario Tapia-Pacheco, Tiffany Amariuta, Yang Li, and Jingbo Shang. 2026. "EvoLen: Evolution-Guided Tokenization for DNA Language Model." *arXiv*. <https://doi.org/10.48550/arXiv.2604.08698>
8. Niktab, Eliatan, and Hardip Patel. 2026. "DNATokenizer: A GPU-First Byte-to-Identifier Tokenizer for High-Throughput DNA Language Models." *arXiv*. <https://doi.org/10.48550/arXiv.2601.05531>
9. DDBJ/ENA/GenBank. 2026. "The DDBJ/ENA/GenBank Feature Table Definition." Version 11.4 page / Version 11.3 text. <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html>

10. Biopython. 2026. "Bio.SeqFeature module documentation." <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html>
11. NCBI. 2026. "GFF3 format." <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/>
12. Addgene. 2026. "Plasmids 101 topic overview." <https://info.addgene.org/plasmids-101-topic-page>
13. Benchling. 2025. "Creating consensus and template alignments on DNA/RNA sequences." <https://help.benchling.com/hc/en-us/articles/9684273213325-Creating-consensus-and-template-alignments-on-DNA-RNA-sequences>
14. Zhou, Hannah, Avanti Shrikumar, and Anshul Kundaje. 2022. "Towards a Better Understanding of Reverse-Complement Equivariance for Deep Learning Models in Genomics." Proceedings of Machine Learning Research. <https://proceedings.mlr.press/v165/zhou22a.html>
15. Schiff, Yair, Chia-Hsiang Kao, Aaron Gokaslan, Tri Dao, Albert Gu, and Volodymyr Kuleshov. 2024. "Caduceus: Bi-Directional Equivariant Long-Range DNA Sequence Modeling." arXiv / ICML 2024. <https://arxiv.org/abs/2403.03234>

Chapter 8

Track G: Validation Tooling

8.0.1 Scope

This track investigated concrete, codifiable validation tools and reference data for plasmid-design checks: restriction-site conflicts, codon-usage scoring, repeat and synthesis-complexity detection, regulatory-element compatibility, and implementation patterns for deterministic PASS/WARN/FAIL validation reports. The focus was on sources that can feed Phase 3 directly: official software docs, curated biological databases, peer-reviewed database papers, and DNA-synthesis provider requirements.

8.0.2 What the literature/sources establish

- Restriction-enzyme validation needs both recognition-site/cut-site data and enzyme meta-data such as methylation sensitivity and commercial availability; REBASE is a curated, fully referenced restriction-modification database covering recognition and cleavage sites for restriction enzymes and DNA methyltransferases, commercial availability, methylation sensitivity, crystal data, sequence data, web browsing, FTP downloads, and monthly updates. [Source: Roberts et al., 2023, "REBASE: a database for DNA restriction and modification: enzymes, genes and genomes", <https://pubmed.ncbi.nlm.nih.gov/36318248/>, DOI: [10.1093/nar/gkac975](https://doi.org/10.1093/nar/gkac975)]
- Biopython provides a `Bio.Restriction` package that is scriptable inside Python, and the current API documentation exposes the package as part of Biopython 1.87. [Source: Biopython contributors, 2026, "Bio.Restriction package - Biopython 1.87 documentation", <https://biopython.org/docs/latest/api/Bio.Restriction.html>]
- Biopython provides `Bio.SeqUtils.CodonAdaptationIndex`, which computes codon adaptiveness from coding DNA sequences, implements the Sharp and Li CAI method, calculates CAI for an input sequence, and can produce codon-optimized DNA using preferred codons from the CAI table. [Source: Biopython contributors, 2026, "Bio.SeqUtils.CodonAdaptationIndex - Biopython 1.87 documentation", <https://biopython.org/docs/latest/api/Bio.SeqUtils.html>]
- The original Codon Adaptation Index method scores a gene by comparing its synonymous codon usage to a reference set of highly expressed genes in the species, so CAI is appropriate as a host-adaptation score but depends on the chosen reference set. [Source: Sharp and Li, 1987, "The codon adaptation index-a measure of directional synonymous codon usage bias, and its potential applications", <https://doi.org/10.1093/nar/15.3.1281>]

- The Kazusa Codon Usage Database reports codon-usage tables derived from NCBI GenBank flat file release 160.0 from June 15, 2007, covering 35,799 organisms and 3,027,973 complete protein-coding genes, and provides a web search interface plus FTP distribution of Codon Usage Tabulated from GenBank. [Source: Kazusa DNA Research Institute, "Codon Usage Database", <https://www.kazusa.or.jp/codon/>]
- The HIVE-CUTs database was built from GenBank and RefSeq coding sequences to provide codon-usage tables for organisms with public sequencing data, and its paper reports that older Kazusa tables can be too sparse for some organisms and can change optimization conclusions compared with HIVE-CUTs. [Source: Athey et al., 2017, "A new and updated resource for codon usage tables", <https://doi.org/10.1186/s12859-017-1793-7>]
- HIVE-CUTs provides plain-text codon usage tables with codon frequency per 1000 codons and raw codon counts, which are directly ingestible as local validation reference data. [Source: Athey et al., 2017, "A new and updated resource for codon usage tables", <https://doi.org/10.1186/s12859-017-1793-7>]
- The Kazusa Countcodon tool can compile a pasted sequence into a codon-usage table and ignores spaces, returns, asterisks, and non-G/A/T/U/C bases; this is useful as a spot-checking reference but not as a runtime dependency. [Source: Kazusa DNA Research Institute, "Count-codon program version 4", <https://www.kazusa.or.jp/codon/countcodon.html>]
- EMBOSS **etandem** identifies tandem repeats in nucleotide sequences, reports locations and scores, and supports minimum repeat size, maximum repeat size, and score threshold parameters. [Source: EMBOSS, "etandem manual", <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/etandem>]
- EMBOSS **einverted** finds inverted repeats, can be made more sensitive by lowering the score threshold, and does not report overlapping matches. [Source: EMBOSS, "einverted manual", <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/einverted>]
- Twist reports that sequence complexity issues are driven mainly by repetitive structures and extreme GC content, and its Express Genes guidance recommends avoiding repeats of at least 20 bp or Tm at least 60 C, keeping global GC between 25% and 65%, avoiding large 50 bp local-GC differences, minimizing homopolymers, and minimizing small scattered repeats. [Source: Twist Bioscience, "Express Genes", <https://www.twistbioscience.com/products/genes/express-genes>]
- Twist states that its gene-synthesis scoring model considers overall GC percent, maximum homopolymer length, maximum repeat length, sequence length, repeat density, and related sequence parameters. [Source: Twist Bioscience, "Express Genes", <https://www.twistbioscience.com/products/genes/express-genes>]
- IDT states that problematic motifs for gBlocks Gene Fragment synthesis include GC content below 25% or above 75%, homopolymeric runs of 10 or more A/T bases or 6 or more G/C bases, and other structural motifs such as repeats or hairpins. [Source: Integrated DNA Technologies, "gBlocks and gBlocks HiFi Gene Fragments", <https://www.eu.idtdna.com/pages/products/genes-and-gene-fragments/double-stranded-dna-fragments/gblocks-gene-fragments>]

- GenScript states that synthesis success is mainly affected by DNA structures including repeats and GC content plus host-cell stability/toxicity, and lists factors including global GC below 20% or above 80%, variable GC content, homopolymers, repeats of at least 20 bp, small-repeat number/length, and palindromic sequence. [Source: GenScript, "FLASH Gene & Gene Synthesis", https://www.genscript.com/gene_synthesis.html]
- EPD/EPDnew are experimentally validated eukaryotic promoter resources: EPD is an annotated non-redundant collection of eukaryotic RNA polymerase II promoters with experimentally determined transcription start sites, and EPDnew uses high-throughput TSS mapping such as CAGE and Oligocapping for selected model organisms. [Source: SIB/ExPASy, "EPD The Eukaryotic Promoter Database", <https://epd.expasy.org/epd/>]
- JASPAR CORE is a curated, non-redundant, open-access database of eukaryotic transcription-factor binding profiles derived from experimentally defined binding sites; JASPAR 2024 exposes downloads, a REST API, and pyJASPAR. [Source: Rauluseviciute et al., 2024, "JASPAR 2024: 20th anniversary of the open-access database of transcription factor binding profiles", <https://jaspar2024.eelixir.no/>, DOI: 10.1093/nar/gkad1059]
- RegulonDB v12.0 is a curated reference for transcription initiation regulation in *Escherichia coli* K-12, integrating classical molecular-biology evidence and high-throughput methodologies. [Source: Santos-Zavaleta et al., 2024, "RegulonDB v12.0: a comprehensive resource of transcriptional regulation in *E. coli* K-12", <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>, DOI: 10.1093/nar/gkad1072]
- Addgene's promoter reference states that promoters must be compatible with the host organism and transcript type, gives common bacterial and mammalian promoters, and notes that RNAP II promoters are used for mRNA while RNAP III promoters are used for small RNAs such as shRNA. [Source: Addgene, "Plasmids 101: The Promoter Region - Let's Go!", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- Addgene's plasmid overview identifies core plasmid elements including origin of replication, antibiotic resistance/selectable marker, multiple cloning site, insert, and promoter region, which are the element classes the validation engine must reason over. [Source: Addgene, "Plasmids 101: What is a plasmid?", <https://blog.addgene.org/plasmids-101-what-is-a-plasmid>]
- The Sequence Ontology provides a structured controlled vocabulary for biological sequence features, so it is a suitable normalization target for feature types used in validation messages and component annotations. [Source: Sequence Ontology Consortium, "Sequence Ontology", <https://www.sequenceontology.org/>]

8.0.3 What this means for our build

- The restriction-site checker should use Biopython `Bio.Restriction` for local deterministic analysis, with enzyme metadata pinned from a versioned REBASE-derived table and optional NEB/REBASE cross-checks during data refresh, because runtime validation must not depend on a web tool. [Source: Biopython contributors, 2026, "Bio.Restriction package - Biopython 1.87 documentation", <https://biopython.org/docs/latest/api/Bio.Restriction.html>; Source: Roberts et al., 2023, "REBASE", <https://pubmed.ncbi.nlm.nih.gov/36318248/>]

- Restriction checks should be context-aware: a site inside a GOI is a FAIL only when it conflicts with the requested cloning strategy or required diagnostic digest, while extra non-required sites should be WARN unless they break an explicitly requested "unique site" or "no internal site" constraint. [Source: Roberts et al., 2023, "REBASE", <https://pubmed.ncbi.nlm.nih.gov/36318248/>; Source: Addgene, "Plasmids 101: What is a plasmid?", <https://blog.addgene.org/plasmids-101-what-is-a-plasmid>]
- Restriction validation must handle circular topology by checking wraparound recognition sites and reporting coordinates in the canonical `AnnotatedSequence` coordinate system, because plasmids are usually circular molecules and the system design requires feature-coordinate annotations. [Source: Addgene, "Plasmids 101: What is a plasmid?", <https://blog.addgene.org/plasmids-101-what-is-a-plasmid>; Source: `SYSTEM_DESIGN.md` Section 12.3, `AnnotatedSequence` schema]
- Codon validation should compute CAI for the GOI against the target host table, report rare-codon clusters in sliding windows, and avoid changing the sequence inside validation because Section 8.2 says codon optimization scoring is separate from sequence optimization. [Source: Biopython contributors, 2026, "Bio.SeqUtils.CodonAdaptationIndex", <https://biopython.org/docs/latest/api/Bio.SeqUtils.html>; Source: Sharp and Li, 1987, CAI, <https://doi.org/10.1093/nar/15.3.1281>; Source: `SYSTEM_DESIGN.md` Section 8.2]
- HIVE-CUTs should be preferred for host codon tables when available, because it incorporates GenBank and RefSeq at larger scale than Kazusa; Kazusa remains a useful fallback and legacy compatibility source when HIVE-CUTs lacks a requested organism or cell-system proxy. [Source: Athey et al., 2017, HIVE-CUTs, <https://doi.org/10.1186/s12859-017-1793-7>; Source: Kazusa DNA Research Institute, "Codon Usage Database", <https://www.kazusa.or.jp/codon/>]
- Codon-table records should store provenance fields including source database, source release/date, organism/taxon, number of CDSs or codons used, genetic code, and retrieval date, because CAI and rare-codon calls are only interpretable relative to their reference table. [Source: Sharp and Li, 1987, CAI, <https://doi.org/10.1093/nar/15.3.1281>; Source: Athey et al., 2017, HIVE-CUTs, <https://doi.org/10.1186/s12859-017-1793-7>]
- Repeat validation should be implemented as a local checker for homopolymers, exact direct repeats, approximate tandem repeats, inverted repeats/palindromes, repeat density, and local GC windows, with `EMBOSS etandem` and `einverted` used as benchmark references rather than required production dependencies. [Source: EMBOSS, "etandem manual", <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/etandem>; Source: EMBOSS, "einverted manual", <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/einverted>]
- Synthesis-readiness checks should use provider-calibrated WARN/FAIL thresholds: Twist-like default thresholds for global GC 25-65%, direct repeats at least 20 bp, and local 50 bp GC variation; IDT-specific stricter homopolymer thresholds for gBlocks-like fragments; and provider profiles should be configurable rather than hard-coded globally. [Source: Twist Bioscience, "Express Genes", <https://www.twistbioscience.com/products/genes/express-genes>; Source: IDT, "gBlocks and gBlocks HiFi Gene Fragments", <https://www.eu.idtdna.com/pages/products/genes-and-gene-fragments/double-stranded-dna-fragments/gblocks-gene-fragments>; Source: GenScript, "FLASH Gene & Gene Synthesis", https://www.genscript.com/gene_synthesis.html]

- Regulatory compatibility should be a rule table over normalized element types, host/vector type, transcript type, and application: e.g. mammalian mRNA expression requires a mammalian/RNAP II-compatible promoter, shRNA requires an RNAP III-compatible promoter, bacterial expression requires bacterial promoter/RBS assumptions, and E. coli promoter lookups should use RegulonDB rather than eukaryotic promoter databases. [Source: Addgene, "Plasmids 101: The Promoter Region", <https://blog.addgene.org/plasmids-101-the-promoter-region>; Source: Santos-Zavaleta et al., 2024, RegulonDB v12.0, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>]
- EPD and JASPAR are useful for validating or annotating natural eukaryotic regulatory elements, but common synthetic plasmid promoters and viral promoters still need a curated local catalog because EPD is focused on experimentally mapped eukaryotic Pol II promoters and JASPAR is focused on transcription-factor binding profiles. [Source: SIB/ExPASy, "EPD", <https://epd.expasy.org/epd/>; Source: JASPAR 2024, <https://jaspar2024.elixir.no/>; Source: Addgene, "Plasmids 101: The Promoter Region", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- Validation output should include check id, status, message, severity, source rule id, offending coordinates, affected feature ids, and remediation text, because the system design requires PASS/WARN/FAIL entries with actionable messages and relevant coordinates. [Source: SYSTEM_DESIGN.md Section 8.1 and Section 12.5]

8.0.4 Decisions proposed

- Use `Bio.Restriction` as the first implementation for restriction analysis and maintain a version-pinned local enzyme catalog derived from REBASE metadata. [Source: Biopython contributors, 2026, "Bio.Restriction package", <https://biopython.org/docs/latest/api/Bio.Restriction.html>; Source: Roberts et al., 2023, "REBASE", <https://pubmed.ncbi.nlm.nih.gov/36318248/>]
- Add a `RestrictionSiteConflictCheck` that accepts `AnnotatedSequence`, `DesignSpec.cloning_method`, and a cloning-strategy rule profile, then emits FAIL for forbidden internal sites, WARN for unexpected extra sites, and PASS when requested sites are unique and in the expected region. [Source: SYSTEM_DESIGN.md Section 8.2; Source: Roberts et al., 2023, "REBASE", <https://pubmed.ncbi.nlm.nih.gov/36318248/>]
- Add a `CodonUsageCheck` that loads codon tables from a local registry, computes CAI with Biopython, calculates per-window rare-codon density, and reports scores without rewriting the coding sequence. [Source: Biopython contributors, 2026, "CodonAdaptationIndex", <https://biopython.org/docs/latest/api/Bio.SeqUtils.html>; Source: Sharp and Li, 1987, CAI, <https://doi.org/10.1093/nar/15.3.1281>]
- Store HIVE-CUTs codon tables as preferred reference data and Kazusa tables as fallback/reference comparators, with provenance and release metadata required for every table. [Source: Athey et al., 2017, HIVE-CUTs, <https://doi.org/10.1186/s12859-017-1793-7>; Source: Kazusa DNA Research Institute, "Codon Usage Database", <http://www.kazusa.or.jp/codon/>]
- Add a `RepeatInstabilityCheck` that reports homopolymers, direct repeats, tandem repeats, inverted repeats, palindromic segments, local GC windows, and repeat density, with thresh-

olds selected by provider profile and vector context. [Source: EMBOSS, "etandem manual", <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/etandem>; Source: EMBOSS, "einverted manual", <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/einverted>; Source: Twist Bioscience, "Express Genes", <https://www.twistbioscience.com/products/genes/express-genes>]

- Add provider-specific synthesis profiles named `twist_default`, `idt_gblocks`, and `genscript_default`; report provider-specific failures as synthesis-readiness WARN/FAIL while keeping the biological construct validation separate. [Source: Twist Bioscience, "Express Genes", <https://www.twistbioscience.com/products/genes/express-genes>; Source: IDT, "gBlocks and gBlocks HiFi Gene Fragments", <https://www.eu.idtdna.com/pages/products/genes-and-gene-fragments/double-stranded-dna-fragments/gblocks-gene-fragments>; Source: GenScript, "FLASH Gene & Gene Synthesis", https://www.genscript.com/gene_synthesis.html]
- Add a local `regulatory_elements.yml` catalog keyed by normalized element name, aliases, feature type, host scope, transcript class, inducibility, required cofactors, source, and confidence, seeded from Addgene references plus EPD/JASPAR/RegulonDB where applicable. [Source: Addgene, "Plasmids 101: The Promoter Region", <https://blog.addgene.org/plasmids-101-the-promoter-region>; Source: SIB/ExPASy, "EPD", <https://epd.expasy.org/epd/>; Source: JASPAR 2024, <https://jaspar2024.elixir.no/>; Source: Santos-Zavaleta et al., 2024, RegulonDB v12.0, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>]
- Normalize feature terms to Sequence Ontology-compatible names where possible, while preserving plasmid-design domain aliases such as ORI, MCS, GOI, and selectable marker in user-facing messages. [Source: Sequence Ontology Consortium, "Sequence Ontology", <https://www.sequenceontology.org/>; Source: Addgene, "Plasmids 101: What is a plasmid?", <https://blog.addgene.org/plasmids-101-what-is-a-plasmid>]
- Implement validation checks as pure functions with deterministic fixtures: each check should run on short synthetic sequences for unit tests and on curated known-good/known-bad plasmids for the Phase 3 gate. [Source: SYSTEM_DESIGN.md Section 8.3 and Section 14]

8.0.5 Open questions / conflicts

- Provider thresholds differ: Twist recommends global GC 25-65% and avoiding repeats at least 20 bp, IDT flags gBlocks synthesis issues below 25% or above 75% GC plus stricter base-specific homopolymers, and GenScript lists broader global GC limits below 20% or above 80%; the build should ask the human which provider profile is the default for "synthesis-ready" exports. [Source: Twist Bioscience, "Express Genes", <https://www.twistbioscience.com/products/genes/express-genes>; Source: IDT, "gBlocks and gBlocks HiFi Gene Fragments", <https://www.eu.idtdna.com/pages/products/genes-and-gene-fragments/double-stranded-dna-fragments/gblocks-gene-fragments>; Source: GenScript, "FLASH Gene & Gene Synthesis", https://www.genscript.com/gene_synthesis.html]
- HIVE-CUTs is more current and broader than Kazusa, but Kazusa is widely recognized and easy to access; the human should decide whether Phase 0 may cache HIVE-CUTs data directly for commercial use after license/terms review. [Source: Athey et al., 2017, HIVE-CUTs,

<https://doi.org/10.1186/s12859-017-1793-7>; Source: Kazusa DNA Research Institute, "Codon Usage Database", <https://www.kazusa.or.jp/codon/>]

- Codon usage for mammalian cell lines is not identical to organism-level codon usage, and none of the reviewed validation sources provides a complete, standard cell-line-specific codon table strategy for HEK293, CHO, or similar production hosts. [Source: Athey et al., 2017, HIVE-CUTs, <https://doi.org/10.1186/s12859-017-1793-7>; Source: Sharp and Li, 1987, CAI, <https://doi.org/10.1093/nar/15.3.1281>]
- Regulatory compatibility for synthetic promoters is partly catalog knowledge rather than database lookup: EPD validates natural eukaryotic Pol II promoters, JASPAR validates TF binding profiles, RegulonDB covers E. coli K-12 regulation, and Addgene provides practical plasmid promoter guidance, but none of these alone is a complete synthetic-vector promoter compatibility database. [Source: SIB/ExPASy, "EPD", <https://epd.expasy.org/epd/>; Source: JASPAR 2024, <https://jaspar2024.elixir.no/>; Source: Santos-Zavaleta et al., 2024, RegulonDB v12.0, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>; Source: Addgene, "Plasmids 101: The Promoter Region", <https://blog.addgene.org/plasmids-101-the-promoter-region>]
- The exact PASS/WARN/FAIL severity policy for therapeutic-compliance or gene/cell-therapy vector elements is out of scope for this validation-tooling pass and should be aligned with Track J before Phase 3 implementation. [Source: SYSTEM_DESIGN.md Section 8.2 and Section 11.6]

8.0.6 Sources

1. Roberts RJ, Vincze T, Posfai J, Macelis D. 2023. "REBASE: a database for DNA restriction and modification: enzymes, genes and genomes." *Nucleic Acids Research* 51(D1):D629-D630. <https://pubmed.ncbi.nlm.nih.gov/36318248/>. DOI: 10.1093/nar/gkac975.
2. Biopython contributors. 2026. "Bio.Restriction package - Biopython 1.87 documentation." <https://biopython.org/docs/latest/api/Bio.Restriction.html>.
3. Biopython contributors. 2026. "Bio.SeqUtils.CodonAdaptationIndex - Biopython 1.87 documentation." <https://biopython.org/docs/latest/api/Bio.SeqUtils.html>.
4. Sharp PM, Li WH. 1987. "The codon adaptation index-a measure of directional synonymous codon usage bias, and its potential applications." *Nucleic Acids Research* 15(3):1281-1295. <https://doi.org/10.1093/nar/15.3.1281>.
5. Kazusa DNA Research Institute. "Codon Usage Database." <https://www.kazusa.or.jp/codon/>.
6. Kazusa DNA Research Institute. "Countcodon program version 4." <https://www.kazusa.or.jp/codon/countcodon.html>.
7. Athey J, Alexaki A, Osipova E, Rostovtsev A, Santana-Quintero LV, Katneni U, Simonyan V, Kimchi-Sarfaty C. 2017. "A new and updated resource for codon usage tables." *BMC Bioinformatics* 18:391. <https://doi.org/10.1186/s12859-017-1793-7>.
8. EMBOSS. "etandem manual." <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/etandem>.

9. EMBOSS. "einverted manual." <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/einverted>.
10. Twist Bioscience. "Express Genes." <https://www.twistbioscience.com/products/genes/express-genes>.
11. Integrated DNA Technologies. "gBlocks and gBlocks HiFi Gene Fragments." <https://www.idtdna.com/pages/products/genes-and-gene-fragments/double-stranded-dna-fragments/gblocks-gene-fragments>.
12. GenScript. "FLASH Gene & Gene Synthesis." https://www.genscript.com/gene_synthesis.html.
13. SIB/ExPASy. "EPD The Eukaryotic Promoter Database." <https://epd.expasy.org/epd/>.
14. Rauluseviciute I, Riudavets-Puig R, Blanc-Mathieu R, Castro-Mondragon JA, Ferenc K, Kumar V, Lemma RB, Lucas J, Cheneby J, Baranasic D, Khan A, Fornes O, Gundersen S, Johansen M, Hovig E, Lenhard B, Sandelin A, Wasserman WW, Parcy F, Mathelier A. 2024. "JASPAR 2024: 20th anniversary of the open-access database of transcription factor binding profiles." Nucleic Acids Research 52(D1):D174-D182. <https://jaspar2024.elixir.no/>. DOI: [10.1093/nar/gkad1059](https://doi.org/10.1093/nar/gkad1059).
15. Santos-Zavaleta A, et al. 2024. "RegulonDB v12.0: a comprehensive resource of transcriptional regulation in E. coli K-12." Nucleic Acids Research 52(D1):D255-D264. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/>. DOI: [10.1093/nar/gkad1072](https://doi.org/10.1093/nar/gkad1072).
16. Addgene. "Plasmids 101: The Promoter Region - Let's Go!" <https://blog.addgene.org/plasmids-101-the-promoter-region>.
17. Addgene. "Plasmids 101: What is a plasmid?" <https://blog.addgene.org/plasmids-101-what-is-a-plasmid>.
18. Sequence Ontology Consortium. "Sequence Ontology." <https://www.sequenceontology.org/>.
19. SYSTEM_DESIGN.md. Local source-of-truth repository specification, Sections 8.1, 8.2, 8.3, 11.6, 12.3, 12.5, and 14.

Chapter 9

Track H: Visualization

9.0.1 Scope

This track investigated browser-ready plasmid visualization libraries, centered on `seqviz` and practical alternatives, for rendering circular and linear annotated plasmid maps from this system's canonical `AnnotatedSequence` shape. It focused on required input data shape, feature annotations, circular/linear rendering behavior, licensing, and frontend integration implications for a Next.js application.

9.0.2 What the literature/sources establish

- `seqviz` is a JavaScript DNA, RNA, and protein sequence viewer distributed through npm, and its documented install path is `npm install seqviz`; the npm package page lists version 3.10.9, TypeScript declarations, 3 dependencies, and MIT licensing metadata. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- `seqviz` can be used as a React component via `SeqViz` or as a plain JavaScript viewer via `seqviz.Viewer()`, so it can be integrated into a React/Next.js frontend while still leaving a non-React embedding route available. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- `seqviz` supports viewer modes named `linear`, `circular`, `both`, and `both_flip`; the package documentation states that `both` renders circular and linear viewers side by side, and `both_flip` reverses their order. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- `seqviz` renders annotations, primers, highlights, amino-acid translations, enzyme cut sites, and sequence search results, and user selection/click events can be returned through an `onSelection()` callback. [Source: Lattice Automation, 2025, `seqviz` GitHub README, <https://github.com/Lattice-Automation/seqviz>]
- The minimum `seqviz` data shape is close to `{ name, seq, annotations }`; `seq` is the sequence string, and each annotation uses 0-based start-inclusive/end-exclusive coordinates plus fields such as `name`, `direction`, `color`, and optional `type`. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- `seqviz` documentation marks `direct file` and `accession` props as deprecated and recommends parsing sequence files outside `seqviz` with `seqparse`; this means our API should

provide normalized sequence JSON instead of making the frontend load GenBank/SnapGene files directly. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]

- `seqparse` parses GenBank, FASTA, JBEI, SnapGene, SBOL, and accession IDs to a common shape containing `name`, `type`, `seq`, and `annotations`; its annotation fields include `name`, `start`, `end`, optional `direction`, optional `color`, and optional `type`, and npm lists it as MIT licensed. [Source: Lattice Automation, 2024, `seqparse` npm package, <https://www.npmjs.com/package/seqparse>]
- Open Vector Editor is an open-source vector/plasmid editor component built with React and Redux, has an MIT license in the GitHub repository, and its original repository says it has moved to TeselaGen's `tg-oss` monorepo. [Source: TeselaGen, 2024, `openVectorEditor` GitHub repository, <https://github.com/TeselaGen/openVectorEditor>]
- Open Vector Editor exposes a `SimpleCircularOrLinearView` example with `sequenceData` containing `circular`, `sequence`, and `features`, and its documented feature coordinates are 0-based inclusive; this differs from `seqviz`'s 0-based start-inclusive/end-exclusive annotation convention. [Source: TeselaGen, 2024, `openVectorEditor` README, <https://github.com/TeselaGen/openVectorEditor>]
- Open Vector Editor's universal build can be embedded in non-React apps, but the documentation says individual components like `CircularView` and `EnzymeViewer` require React, so using it as a fine-grained map component would couple the frontend more tightly to its React/Redux model than `seqviz`. [Source: TeselaGen, 2024, `openVectorEditor` README, <https://github.com/TeselaGen/openVectorEditor>]
- `CGView.js` is a browser JavaScript circular genome viewing tool adapted from the Java `CGView` program, and its home page documents circular and linear genome views, smooth zooming, features/plots from sequence, PNG/SVG export, and npm installation via `npm install cgview`. [Source: Jason R. Grant and `CGView.js` team, 2025, `CGView.js` documentation, <https://js.cgview.ca/>]
- `CGView.js` documentation states that it can draw genomes up to 10 Mbp with thousands of features and hundreds of contigs, which makes it more genome-browser-like than the plasmid-focused `seqviz` need for ordinary synthetic plasmids. [Source: Jason R. Grant and `CGView.js` team, 2025, `CGView.js` documentation, <https://js.cgview.ca/>]
- `CGView.js` API documentation shows an initialization shape with a target element, `height`, `width`, and a `sequence` object containing either `length` or `seq`; the docs also point users to `CGParse.js` for converting GenBank and EMBL files into maps. [Source: Jason R. Grant and `CGView.js` team, 2025, `CGView.js` API documentation, <https://js.cgview.ca/api/index.html>]
- `CGView.js` is distributed under the Apache License 2.0 according to its API documentation. [Source: Jason R. Grant and `CGView.js` team, 2025, `CGView.js` API documentation, <https://js.cgview.ca/api/index.html>]
- `JBrowse 2` provides embeddable React components, including `@jbrowse/react-linear-genome-view2` and `@jbrowse/react-circular-genome-view2`, and its embedded-components documentation describes these as reusable components for

placing JBrowse views on a webpage. [Source: GMOD/JBrowse, 2026, JBrowse 2 embedded components documentation, https://jbrowse.org/jb2/docs/embedded_components/]

- The npm page for `@jbrowse/react-circular-genome-view` describes it as a single JBrowse 2 circular genome view React component, shows an integration shape based on `createViewState({ assembly, tracks })`, lists 22 dependencies, and lists MIT licensing. [Source: GMOD/JBrowse, 2025, `@jbrowse/react-circular-genome-view` npm package, <https://www.npmjs.com/package/%40jbrowse/react-circular-genome-view>]
- JBrowse 2 documentation says embedded components differ from the full JBrowse Web app because an embedded component has access to one view type, while full JBrowse Web can load all session view types and plugins; this matters if the product later needs full genome-browser workflows rather than a plasmid map widget. [Source: GMOD/JBrowse, 2026, JBrowse 2 FAQ, <https://jbrowse.org/jb2/docs/faq/>]
- The JBrowse 2 paper describes JBrowse 2 as a modular genome browser with embedded linear and circular React components available on npm, and states that JBrowse 2 source code is distributed under Apache License 2.0. [Source: Diesh et al., 2023, "JBrowse 2: a modular genome browser with views of synteny and structural variation", Genome Biology, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10108523/>]
- `igv.js` is an embeddable JavaScript genome visualization component developed by the IGV team, runs in modern browsers, and is MIT licensed according to its official documentation. [Source: IGV Team, 2026, `igv.js` documentation, <https://igv.org/doc/igvjs/>]
- The `igv.js` quickstart creates a browser from a container div plus a configuration object defining a reference genome, locus, tracks, and related state, which is a genome-browser track model rather than a direct plasmid-map component model. [Source: IGV Team, 2026, `igv.js` quickstart, <https://igv.org/doc/igvjs/QuickStart/>]
- The Bioinformatics paper for `igv.js` states that the component is embeddable, runs entirely in the web browser with no backend server or preprocessing required, is installable from npm, and has MIT-licensed source code. [Source: Robinson et al., 2023, "igv.js: an embeddable JavaScript implementation of the Integrative Genomics Viewer (IGV)", Bioinformatics, <https://academic.oup.com/bioinformatics/article/39/1/btac830/6958554>]

9.0.3 What this means for our build

- Use `seqviz` as the default Phase 4 plasmid-map renderer because Section 9.4 already names it, it supports the exact circular-plus-linear rendering requirement, and its prop model can be derived directly from our canonical `AnnotatedSequence` without introducing a genome-browser assembly/track abstraction. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- Keep the backend/frontend contract normalized around our own `AnnotatedSequence` schema and add a small adapter from `{ sequence, topology, features[] }` to `seqviz` props `{ name, seq, annotations, viewer }`, because `seqviz` itself recommends parsing files outside the viewer and receiving already-parsed sequence data. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]

- Map feature `strand` to `seqviz direction` where forward is 1, reverse is -1, and absent/unknown is omitted; map feature `name`, `type`, and deterministic component colors to annotation fields because `seqviz` annotations accept those fields. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- Preserve the internal coordinate convention from Section 12.3 and convert explicitly at adapter boundaries, because `seqviz` uses 0-based start-inclusive/end-exclusive coordinates while Open Vector Editor documents 0-based inclusive start/end coordinates. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>; Source: TeselaGen, 2024, `openVectorEditor` README, <https://github.com/TeselaGen/openVectorEditor>]
- Treat Open Vector Editor as an option for a future editable plasmid design surface rather than the initial read-only map, because it is editor-oriented and uses a larger React/Redux state model. [Source: TeselaGen, 2024, `openVectorEditor` README, <https://github.com/TeselaGen/openVectorEditor>]
- Treat `CGView.js` as a fallback or future high-density/export renderer when the system needs larger genome maps, rich plot tracks, or very high-resolution SVG/PNG output; for the MVP plasmid experience it is less directly aligned than `seqviz` because its documented input model centers on a `CGView` viewer/map API rather than the simple sequence-plus-annotations React prop shape. [Source: Jason R. Grant and `CGView.js` team, 2025, `CGView.js` documentation, <https://js.cgview.ca/>]
- Avoid `JBrowse` and `igv.js` for the first plasmid-map UI unless we later add genomic tracks, read alignments, variants, synteny, or large reference-genome browsing; both are strong genome-browser components, but their documented configuration models are heavier than needed for showing one annotated plasmid design. [Source: GMOD/`JBrowse`, 2026, `JBrowse 2` embedded components documentation, https://jbrowse.org/jb2/docs/embedded_components/; Source: IGV Team, 2026, `igv.js` quickstart, <https://igv.org/doc/igvjs/QuickStart/>]
- Implement map rendering behind a frontend adapter component such as `PlasmidMapView` so the rest of the app depends on `AnnotatedSequence` rather than on `seqviz` prop names; this also leaves room to swap in `CGView.js`, `JBrowse`, or Open Vector Editor later without changing API responses. [Source: `SYSTEM_DESIGN.md` Section 4.3 interface-contract rule; Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- Add visual regression or browser smoke tests once the frontend exists, because the selected renderer is an interactive browser component and the Phase 4 acceptance criteria depend on the rendered map being visible and correct. [Source: `SYSTEM_DESIGN.md` Section 9.4; Source: Lattice Automation, 2025, `seqviz` GitHub README, <https://github.com/Lattice-Automation/seqviz>]

9.0.4 Decisions proposed

- Default renderer: `seqviz` for Phase 4 circular and linear plasmid maps. [Source: `SYSTEM_DESIGN.md` Section 9.4; Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]

- Default viewer mode: use `viewer="both"` for desktop when space allows, and switch to a tabbed or stacked `viewer="circular"` / `viewer="linear"` layout on narrow screens because `seqviz` exposes those viewer modes. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- Required API-to-view adapter shape: `name`, `seq`, `annotations`, `viewer`, optional `style`, and `onSelection`; compute `annotations` from `AnnotatedSequence.features` with 0-based start-inclusive/end-exclusive coordinates. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- Color policy: assign deterministic colors by feature type (`ORI`, `promoter`, `G0I`, `marker`, `MCS`, `terminator`, `other`) in the adapter layer, while allowing a feature-level override only if validated data later supplies one. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- File import/export policy: keep GenBank/FASTA/SnapGene parsing and export outside the visualizer; if frontend-side parsing is needed for user uploads, evaluate `seqparse`, but generated designs should arrive from the backend as validated `AnnotatedSequence` JSON. [Source: Lattice Automation, 2024, `seqparse` npm package, <https://www.npmjs.com/package/seqparse>]
- Alternative ranking for future use: Open Vector Editor for editing workflows, `CGView.js` for dense circular genome/plot/export workflows, `JBrowse` for genomic tracks and structural-variation views, and `igv.js` for alignment/variant track inspection. [Source: TeselaGen, 2024, `openVectorEditor` README, <https://github.com/TeselaGen/openVectorEditor>; Source: Jason R. Grant and `CGView.js` team, 2025, `CGView.js` documentation, <https://js.cgview.ca/>; Source: GMOD/`JBrowse`, 2026, `JBrowse 2` embedded components documentation, https://jbrowse.org/jb2/docs/embedded_components/; Source: IGV Team, 2026, `igv.js` documentation, <https://igv.org/doc/igvjs/>]

9.0.5 Open questions / conflicts

- `seqviz` documentation clearly defines simple annotation coordinates, but this pass did not verify how `seqviz` handles a circular feature that crosses the origin; before implementing the adapter, test an origin-spanning feature and decide whether to split it into two annotations. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- The latest public package metadata reviewed here shows `seqviz` 3.10.9 and the GitHub README, but package health, maintenance cadence, and compatibility with the exact future Next.js version should be rechecked at Phase 4 implementation time. [Source: Lattice Automation, 2025, `seqviz` npm package, <https://www.npmjs.com/package/seqviz>]
- `JBrowse` package metadata can show MIT licensing for individual npm packages while the `JBrowse 2` paper states Apache License 2.0 for `JBrowse 2` source code; this is not blocking for an MVP that does not use `JBrowse`, but it should be reviewed if `JBrowse` becomes a dependency. [Source: GMOD/`JBrowse`, 2025, `@jbrowse/react-circular-genome-view` npm package, <https://www.npmjs.com/package/%40jbrowse/react-circular-genome-view>; Source: Diesh et al., 2023, "JBrowse 2: a modular genome browser with views of synteny and structural variation", *Genome Biology*, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10108523/>]

- Need a product/design decision on whether map interactivity should be read-only in MVP or include editing; `seqviz` fits read-only visualization, while Open Vector Editor is more appropriate if users must directly edit sequence/features in the browser. [Source: Lattice Automation, 2025, `seqviz` GitHub README, <https://github.com/Lattice-Automation/seqviz>; Source: TeselaGen, 2024, `openVectorEditor` README, <https://github.com/TeselaGen/openVectorEditor>]

9.0.6 Sources

1. Lattice Automation. 2025. "seqviz npm package." <https://www.npmjs.com/package/seqviz>
2. Lattice Automation. 2025. "seqviz: a JavaScript DNA, RNA, and protein sequence viewer." GitHub README. <https://github.com/Lattice-Automation/seqviz>
3. Lattice Automation. 2024. "secparse npm package." <https://www.npmjs.com/package/secparse>
4. TeselaGen. 2024. "openVectorEditor: Open Source Vector/Plasmid Editor Component." GitHub README. <https://github.com/TeselaGen/openVectorEditor>
5. Jason R. Grant and CGView.js team. 2025. "CGView.js documentation." <https://js.cgview.ca/>
6. Jason R. Grant and CGView.js team. 2025. "CGView.js API documentation." <https://js.cgview.ca/api/index.html>
7. Stothard P., Grant J.R., Van Domselaar G. 2019. "Visualizing and comparing circular genomes using the CGView family of tools." Briefings in Bioinformatics. <https://pubmed.ncbi.nlm.nih.gov/29939276/>
8. GMOD/JBrowse. 2026. "JBrowse 2 embedded components documentation." https://jbrowse.org/jb2/docs/embedded_components/
9. GMOD/JBrowse. 2026. "JBrowse 2 FAQ." <https://jbrowse.org/jb2/docs/faq/>
10. GMOD/JBrowse. 2025. "@jbrowse/react-circular-genome-view npm package." <https://www.npmjs.com/package/%40jbrowse/react-circular-genome-view>
11. Diesh C., Stevens G.J., Xie P., De Jesus Martinez T., Hershberg E.A., Leung A., Guo E., Dider S., Zhang J., Bridge C., Hogue G., Duncan A., Morgan M., Flores J.R., Bimber B.N., Haw R., Cain S., Buels R.M., Stein L.D., Holmes I.H., Elisk C.G., Lewis S.E., Mungall C.J., Yao E. 2023. "JBrowse 2: a modular genome browser with views of syntenic and structural variation." Genome Biology. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10108523/>
12. IGV Team. 2026. "igv.js documentation." <https://igv.org/doc/igvjs/>
13. IGV Team. 2026. "igv.js quickstart." <https://igv.org/doc/igvjs/QuickStart/>
14. Robinson J.T., Thorvaldsdottir H., Turner D., Mesirov J.P. 2023. "igv.js: an embeddable JavaScript implementation of the Integrative Genomics Viewer (IGV)." Bioinformatics. <https://academic.oup.com/bioinformatics/article/39/1/btac830/6958554>

Chapter 10

Track I: System Architecture & ML in Production

10.0.1 Scope

This track investigated reference architecture patterns for retrieval-augmented generation over scientific corpora and for production serving of fine-tuned generative models, including inference servers, quantization, evaluation gates, model registry and rollout controls, observability, and failure modes relevant to a natural-language-to-synthesis-ready-plasmid system.

10.0.2 What the literature/sources establish

- The canonical RAG formulation combines a parametric seq2seq model with a non-parametric dense vector index accessed through a neural retriever; Lewis et al. specifically frame RAG as a way to improve knowledge-intensive generation, provenance, and updateability relative to parametric-only models [Source: Lewis et al., 2020, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", <https://arxiv.org/abs/2005.11401>].
- RAG is not a correctness guarantee: the original RAG paper reports more specific, diverse, and factual generations than a parametric-only baseline, but the architecture still depends on retrieved passages and the generator's faithful use of them [Source: Lewis et al., 2020, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", <https://arxiv.org/abs/2005.11401>].
- Scientific corpora benefit from domain-specific document embeddings: SPECTER was trained with citation-graph supervision and is designed to produce document-level embeddings for scientific-paper classification, recommendation, and citation-related tasks [Source: Cohan et al., 2020, "SPECTER: Document-level Representation Learning using Citation-informed Transformers", <https://aclanthology.org/2020.acl-main.207/>].
- Scientific claim verification is a distinct retrieval-and-reasoning problem: SciFact introduced 1.4K expert-written scientific claims paired with evidence-containing abstracts labeled as supporting or refuting the claim, and the authors describe scientific corpora as specialized-domain retrieval/reasoning testbeds [Source: Wadden et al., 2020, "Fact or Fiction: Verifying Scientific Claims", <https://aclanthology.org/2020.emnlp-main.609/>].
- Biomedical RAG work increasingly adds structured knowledge, not only plain vector search: MedGraphRAG constructs a graph that links user documents to credible medical sources and

controlled vocabularies, then uses graph-aware retrieval to combine global context with precise indexing [Source: Wu et al., 2024, "Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation", <https://arxiv.org/abs/2408.04187>].

- A separate MedRAG paper reports a four-tier hierarchical diagnostic knowledge graph, retrieval from an EHR database, and LLM reasoning over the retrieved graph/context; its authors present this as a response to inadequate accuracy and specificity in heuristic medical RAG [Source: Zhao et al., 2025, "MedRAG: Enhancing Retrieval-augmented Generation with Knowledge Graph-Elicited Reasoning for Healthcare Copilot", <https://arxiv.org/abs/2502.04413>].
- RAG evaluation should measure retrieval and generation separately: RAGAS defines reference-free metrics for context relevance, whether the LLM faithfully uses context, and generation quality because RAG systems contain both retrieval and generation modules [Source: Es et al., 2023/2025, "Ragas: Automated Evaluation of Retrieval Augmented Generation", <https://arxiv.org/abs/2309.15217>].
- ARES evaluates RAG systems along context relevance, answer faithfulness, and answer relevance; it uses synthetic training data plus a small human-annotated set for prediction-powered inference, and reports effectiveness across domain shifts [Source: Saad-Falcon et al., 2023/2024, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems", <https://arxiv.org/abs/2311.09476>].
- vLLM's PagedAttention addresses LLM serving bottlenecks caused by dynamically growing KV-cache memory; the paper reports 2-4x throughput improvements at similar latency versus FasterTransformer and Orca in evaluated workloads [Source: Kwon et al., 2023, "Efficient Memory Management for Large Language Model Serving with PagedAttention", <https://arxiv.org/abs/2309.06180>].
- Hugging Face Text Generation Inference is a production-oriented LLM serving toolkit with OpenTelemetry tracing, Prometheus metrics, tensor parallelism, token streaming, continuous batching, Flash Attention, and PagedAttention support, while its own docs state TGI is now in maintenance mode and recommend vLLM/SGLang/local interoperable engines for future optimized serving [Source: Hugging Face, "Text Generation Inference", <https://huggingface.co/docs/text-generation-inference/index>].
- NVIDIA Triton serves models from a model repository, routes HTTP/gRPC requests to per-model schedulers and backends, supports configurable batching/scheduling, exposes model management APIs, and can serve models from local paths or cloud object storage including S3 [Source: NVIDIA, "Triton Inference Server", <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html>; Source: NVIDIA, "Model Repository", https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_repository.html].
- Triton model repositories are versioned by numeric subdirectories, and model configuration can specify version policies controlling which versions are available; this makes model versioning an explicit serving concern rather than only metadata in a training system [Source: NVIDIA, "Model Repository", https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_repository.html].

- Quantization is a deployment lever, not only a training choice: TGI supports GPTQ, AWQ, bitsandbytes, EETQ, Marlin, EXL2, and fp8 quantization; GPTQ/AWQ/Marlin/EXL2 require pre-quantized weights, while bitsandbytes/EETQ/fp8 can quantize on load [Source: Hugging Face, "TGI Quantization", <https://huggingface.co/docs/text-generation-inference/conceptual/quantization>].
- bitsandbytes 8-bit quantization can reduce hardware requirements for multi-billion-parameter inference, but the TGI docs note bitsandbytes inference is slower than GPTQ or FP16 precision [Source: Hugging Face, "TGI Quantization", <https://huggingface.co/docs/text-generation-inference/conceptual/quantization>].
- LLM.int8() cuts transformer inference memory roughly in half for feed-forward and attention projection layers while preserving full-precision performance in the evaluated models up to 175B parameters by using vector-wise quantization plus mixed-precision treatment of outlier features [Source: Dettmers et al., 2022, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale", <https://arxiv.org/abs/2208.07339>].
- MLflow Model Registry supports registered models with versions, aliases, tags, signatures, source runs, and descriptions; its API can register a model during a run or later from a run URI [Source: MLflow, "Model Registry Workflows", <https://mlflow.org/docs/latest/ml/model-registry/workflow/>].
- KServe supports canary rollout for inference services, including routing a configured percentage of traffic to a new revision, tracking the last good fully rolled-out revision, avoiding traffic to unhealthy revisions, and pinning traffic back to a previous revision for rollback [Source: KServe, "Canary Rollout Strategy", <https://kserve.github.io/archive/0.13/model-serving/v1beta1/rollout/canary/>].
- OWASP's LLM application risks include prompt injection, insecure output handling, training data poisoning, model denial of service, supply-chain vulnerabilities, sensitive information disclosure, excessive agency, overreliance, and model theft, all of which apply to a RAG-plus-generation system exposed through an API [Source: OWASP Foundation, "OWASP Top 10 for Large Language Model Applications", <https://owasp.org/www-project-top-10-for-large-language-model-applications/>].
- NIST's Generative AI Profile states that generative-AI risks can arise from design, training, operation, inputs, outputs, human misuse, and human-AI interaction, and that immature measurement science and limited visibility into training data make some risks hard to scope or estimate [Source: NIST, 2024, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile", <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>].

10.0.3 What this means for our build

- The system should use RAG as a grounding layer over curated plasmid records, annotations, papers, and validation references, but the final plasmid must be accepted only after deterministic biological validation because RAG improves provenance/updateability without proving the generated DNA is valid [Source: Lewis et al., 2020, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", <https://arxiv.org/abs/2005.11401>; Source: Es et al., 2023/2025, "Ragas", <https://arxiv.org/abs/2309.15217>].

- Retrieval should combine vector search with structured filters and biological metadata, because scientific and biomedical RAG papers show value in domain-specific embeddings, evidence retrieval, and structured knowledge beyond undifferentiated text chunks [Source: Cohan et al., 2020, "SPECTER", <https://aclanthology.org/2020.acl-main.207/>; Source: Wadden et al., 2020, "Fact or Fiction", <https://aclanthology.org/2020.emnlp-main.609/>; Source: Wu et al., 2024, "MedGraphRAG", <https://arxiv.org/abs/2408.04187>].
- The production pipeline should separate at least four planes: data/indexing, retrieval/reranking, generation, and validation/export, because RAG evaluation frameworks treat retrieval context quality, answer faithfulness, and answer relevance as separable failure surfaces [Source: Es et al., 2023/2025, "Ragas", <https://arxiv.org/abs/2309.15217>; Source: Saad-Falcon et al., 2023/2024, "ARES", <https://arxiv.org/abs/2311.09476>].
- The generation service should be behind the `SequenceGenerator` interface and initially support a deterministic fake plus a real serving adapter, because `SYSTEM_DESIGN` requires deterministic fakes and the cited serving systems expose deployable model endpoints independently from application code [Source: `SYSTEM_DESIGN.md` Section 4.3; Source: Hugging Face, "Text Generation Inference", <https://huggingface.co/docs/text-generation-inference/index>; Source: NVIDIA, "Triton Inference Server", <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html>].
- vLLM should be the default first real LLM/DNA-LM serving target for autoregressive models when compatible with the selected checkpoint, because it directly addresses KV-cache memory pressure for batched long-sequence generation and is now recommended by Hugging Face's TGI documentation for optimized inference engines [Source: Kwon et al., 2023, "PageldAttention", <https://arxiv.org/abs/2309.06180>; Source: Hugging Face, "Text Generation Inference", <https://huggingface.co/docs/text-generation-inference/index>].
- Triton is a strong later target for heterogeneous production serving and model-repository governance, especially if the build serves multiple backends such as embedding models, rerankers, validation classifiers, and sequence generators from a shared inference platform [Source: NVIDIA, "Triton Inference Server", <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html>; Source: NVIDIA, "Model Repository", https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_repository.html].
- Quantized serving should be treated as a candidate optimization that must pass sequence-quality, validation-pass-rate, and latency/cost gates, because low-bit methods reduce memory but can alter model behavior and different quantization schemes have different speed and calibration requirements [Source: Hugging Face, "TGI Quantization", <https://huggingface.co/docs/text-generation-inference/conceptual/quantization>; Source: Dettmers et al., 2022, "LLM.int8()", <https://arxiv.org/abs/2208.07339>].
- Rollout must be gated by offline evals, shadow runs, and canary traffic rather than direct promotion, because KServe exposes percentage-based canaries and rollback semantics, and NIST emphasizes that GAI risks vary by model/system architecture, training data, operation, access, and use case [Source: KServe, "Canary Rollout Strategy", <https://kserve.github.io/archive/0.13/model-serving/v1beta1/rollout/canary/>; Source: NIST, 2024, "GAI Profile", <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>].

- The system’s eval gate must include biological validity metrics in addition to general RAG metrics, because RAGAS/ARES cover context relevance and faithfulness but do not measure plasmid-specific constraints such as component ordering, restriction conflicts, host compatibility, or synthesis constraints [Source: Es et al., 2023/2025, "Ragas", <https://arxiv.org/abs/2309.15217>; Source: Saad-Falcon et al., 2023/2024, "ARES", <https://arxiv.org/abs/2311.09476>].
- Security and safety controls should be implemented outside prompts alone, because OWASP identifies prompt injection, output handling, poisoning, disclosure, excessive agency, and overreliance as application risks, and this build produces orderable biological sequences where deterministic policy checks are mandatory [Source: OWASP Foundation, "OWASP Top 10 for Large Language Model Applications", <https://owasp.org/www-project-top-10-for-large-language-model-applications/>; Source: NIST, 2024, "GAI Profile", <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>].

10.0.4 Decisions proposed

- Use a layered architecture for Phase 0-4: source ingestion and normalization, rebuildable retrieval indexes, `IntentParser`, `Retriever`, `SequenceGenerator`, `ConstraintEngine`, primer/export services, and a job-oriented API; keep each ML/provider dependency behind the `SYSTEM_DESIGN` interfaces with deterministic fake implementations [Source: `SYSTEM_DESIGN.md` Section 4.3; Source: Lewis et al., 2020, "RAG", <https://arxiv.org/abs/2005.11401>].
- Implement retrieval as hybrid evidence retrieval over structured plasmid/component metadata plus text embeddings over composed records and literature snippets, then store enough provenance to cite retrieved sources in validation and recommendation outputs [Source: Cohan et al., 2020, "SPECTER", <https://aclanthology.org/2020.acl-main.207/>; Source: Wadden et al., 2020, "Fact or Fiction", <https://aclanthology.org/2020.emnlp-main.609/>; Source: Lewis et al., 2020, "RAG", <https://arxiv.org/abs/2005.11401>].
- Stand up a local fake generator first, then add a vLLM-compatible adapter for the selected open DNA generator, and evaluate Triton only when the serving portfolio includes enough heterogeneous models to justify a model repository and backend abstraction [Source: Kwon et al., 2023, "PagedAttention", <https://arxiv.org/abs/2309.06180>; Source: Hugging Face, "Text Generation Inference", <https://huggingface.co/docs/text-generation-inference/index>; Source: NVIDIA, "Triton Inference Server", <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html>].
- Track every model, adapter, prompt, dataset snapshot, quantization mode, and validation-suite version as a versioned artifact with registry metadata and aliases such as `candidate`, `shadow`, and `production` [Source: MLflow, "Model Registry Workflows", <https://mlflow.org/docs/latest/ml/model-registry/workflow/>; Source: NVIDIA, "Model Repository", https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_repository.html].
- Gate promotion on an evaluation bundle that includes retrieval metrics, answer/source faithfulness, deterministic plasmid validation pass rate, synthesis-screen pass rate, latency, cost,

and regression checks against curated design prompts [Source: Es et al., 2023/2025, "Ragas", <https://arxiv.org/abs/2309.15217>; Source: Saad-Falcon et al., 2023/2024, "ARES", <https://arxiv.org/abs/2311.09476>; Source: KServe, "Canary Rollout Strategy", <https://kserve.github.io/archive/0.13/model-serving/v1beta1/rollout/canary/>].

- Use quantization only after a baseline fp16/bf16 model has known validation behavior; require quantized variants to pass the same biological and retrieval/generation eval gates before they can serve user-visible designs [Source: Hugging Face, "TGI Quantization", <https://huggingface.co/docs/text-generation-inference/conceptual/quantization>; Source: Dettmers et al., 2022, "LLM.int8()", <https://arxiv.org/abs/2208.07339>].
- Treat the vector index as rebuildable derived state rather than the source of truth; source records, parsed annotations, citations, and eval datasets should remain canonical in relational/object storage so embeddings can be regenerated when models or chunking rules change [Source: Lewis et al., 2020, "RAG", <https://arxiv.org/abs/2005.11401>; Source: SPECTER, Cohan et al., 2020, <https://aclanthology.org/2020.acl-main.207/>].

10.0.5 Open questions / conflicts

- TGI remains documented as production-ready but is also described by Hugging Face as being in maintenance mode with vLLM/SGLang recommended going forward; the build should prefer vLLM for new generative serving unless a selected DNA model is unsupported [Source: Hugging Face, "Text Generation Inference", <https://huggingface.co/docs/text-generation-inference/index>; Source: Kwon et al., 2023, "PagedAttention", <https://arxiv.org/abs/2309.06180>].
- The best serving stack for Evo 2 or Carbon-style DNA models depends on checkpoint architecture, tokenizer support, context length, and GPU memory behavior; this track did not verify exact compatibility for the selected Track C model candidate [Source: Hugging Face, "Text Generation Inference", <https://huggingface.co/docs/text-generation-inference/index>; Source: NVIDIA, "Triton Inference Server", <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html>].
- RAGAS and ARES are useful architecture-evaluation patterns, but neither validates biological correctness; Phase 3 must define plasmid-specific metrics and acceptance thresholds before production rollout [Source: Es et al., 2023/2025, "Ragas", <https://arxiv.org/abs/2309.15217>; Source: Saad-Falcon et al., 2023/2024, "ARES", <https://arxiv.org/abs/2311.09476>].
- Quantization quality for genomic sequence generation is unverified in the cited sources; do not assume LLM.int8() findings on general transformer language models transfer unchanged to DNA language models without plasmid-specific evals [Source: Dettmers et al., 2022, "LLM.int8()", <https://arxiv.org/abs/2208.07339>].
- Scientific RAG literature supports evidence retrieval and structured knowledge, but there is not yet a cited reference architecture specifically for natural-language-to-complete-plasmid-design; this build should expect to create its own biological eval suite and rollout policy [Source: Wadden et al., 2020, "Fact or Fiction", <https://aclanthology.org/2020.emnlp-main.609/>; Source: Wu et al., 2024, "MedGraphRAG", <https://arxiv.org/abs/>].

2408.04187; Source: NIST, 2024, "GAI Profile", <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>].

10.0.6 Sources

1. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuettler, Mike Lewis, Wen-tau Yih, Tim Rocktaeschel, Sebastian Riedel, Douwe Kiela, 2020, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS/arXiv, <https://arxiv.org/abs/2005.11401>.
2. Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, Daniel S. Weld, 2020, "SPECTER: Document-level Representation Learning using Citation-informed Transformers", ACL, <https://aclanthology.org/2020.acl-main.207/>.
3. David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, Hannaneh Hajishirzi, 2020, "Fact or Fiction: Verifying Scientific Claims", EMNLP, <https://aclanthology.org/2020.emnlp-main.609/>.
4. Junde Wu, Jiayuan Zhu, Yunli Qi, Jingkun Chen, Min Xu, Filippo Menolascina, Vicente Grau, 2024, "Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation", arXiv, <https://arxiv.org/abs/2408.04187>.
5. Xuejiao Zhao, Siyan Liu, Su-Yin Yang, Chunyan Miao, 2025, "MedRAG: Enhancing Retrieval-augmented Generation with Knowledge Graph-Elicited Reasoning for Healthcare Copilot", arXiv, <https://arxiv.org/abs/2502.04413>.
6. Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert, 2023/2025, "Ragas: Automated Evaluation of Retrieval Augmented Generation", arXiv, <https://arxiv.org/abs/2309.15217>.
7. Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia, 2023/2024, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems", NAACL/arXiv, <https://arxiv.org/abs/2311.09476>.
8. Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica, 2023, "Efficient Memory Management for Large Language Model Serving with PagedAttention", SOSP/arXiv, <https://arxiv.org/abs/2309.06180>.
9. Hugging Face, "Text Generation Inference" documentation, <https://huggingface.co/docs/text-generation-inference/index>.
10. Hugging Face, "Text Generation Inference: Quantization" documentation, <https://huggingface.co/docs/text-generation-inference/conceptual/quantization>.
11. NVIDIA, "Triton Inference Server" documentation, <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html>.
12. NVIDIA, "Triton Inference Server: Model Repository" documentation, https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_repository.html.

13. Tim Dettmers, Mike Lewis, Younes Belkada, Luke Zettlemoyer, 2022, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale", NeurIPS/arXiv, <https://arxiv.org/abs/2208.07339>.
14. MLflow, "Model Registry Workflows" documentation, <https://mlflow.org/docs/latest/ml/model-registry/workflow/>.
15. KServe, "Canary Rollout Strategy" documentation, <https://kserve.github.io/archive/0.13/model-serving/v1beta1/rollout/canary/>.
16. OWASP Foundation, "OWASP Top 10 for Large Language Model Applications", <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
17. National Institute of Standards and Technology, 2024, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile", NIST AI 600-1, <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

Chapter 11

Track J: Biosecurity & Compliance

11.0.1 Scope

This track investigated sequence-of-concern screening duties and best practices for a software system that generates orderable DNA designs from natural-language prompts, with emphasis on U.S. government guidance, gene-synthesis industry practice, standards work, and implications for AI-enabled design tools. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf] [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]

11.0.2 What the literature/sources establish

- The 2024 U.S. Framework treats nucleic acid synthesis as a biosecurity control point and directs providers and benchtop-equipment manufacturers to screen purchase orders for sequences of concern, assess customer legitimacy, retain relevant records, report unresolved illegitimate orders, and protect screening/customer information. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 101-138, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- The 2024 U.S. Framework says federal life-science funding agencies will require federally funded procurement of synthetic nucleic acids and benchtop synthesis equipment to use providers and manufacturers that adhere to the framework, with the stated effective date of April 26, 2025. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 121-144, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- A May 5, 2025 Executive Order directs OSTP to revise or replace the 2024 Framework within 90 days, requires federally funded synthetic nucleic acid procurement to use adherent providers or manufacturers, and calls for actions toward comprehensive, scalable, and verifiable screening in non-federally funded settings. [Source: White House, 2025, Improving the Safety and Security of Biological Research, lines 162-166, <https://www.whitehouse.gov/presidential-actions/2025/05/improving-the-safety-and-security-of-biological-research/>]

- The 2023 HHS guidance is broader than the older 2010 dsDNA guidance: it covers DNA and RNA, single- and double-stranded forms, providers, manufacturers, third-party vendors, institutions, principal users, and end users. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 63-65 and 71-77, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]
- The 2023 HHS guidance defines a Sequence of Concern initially as a best match to federally regulated agents on the Biological Select Agents and Toxins list or Commerce Control List, excluding matches also found in unregulated organisms or toxins, and recommends expanding SOC treatment to sequences known to contribute to pathogenicity or toxicity even when not derived from regulated agents. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 377-385, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]
- The 2024 U.S. Framework operationalizes SOC screening with 200-nucleotide windows at issuance, requires translation-aware screening across applicable reading frames for protein-coding nucleic acids, and says that on and after October 13, 2026 providers should reduce the screening window to 50 nucleotides and detect attempts to assemble SOC from shorter ordered fragments across bulk or repeated orders. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 297-320, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- The 2024 U.S. Framework says expanded SOC criteria can include scientific evidence that a sequence contributes to pathogenicity or toxicity, whether the function is recognized as a misuse candidate, and the ease of misuse by de novo assembly or insertion into a backbone. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 329-335, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- The Federal Select Agent Program list covers biological agents and toxins determined to pose severe threats to human, animal, or plant health, and it specifically notes that recombinant or synthetic nucleic acids encoding expressible toxic forms of select toxins may have regulatory status. [Source: Federal Select Agent Program, 2025, Select Agents and Toxins List, lines 39-43 and 131-135, <https://www.selectagents.gov/sat/list.htm>]
- The Commerce Control List is relevant because U.S. screening frameworks use CCL-controlled agents for international orders, and ECCN 1C353 covers genetic elements and genetically modified organisms for listed biological agents and toxins under the Export Administration Regulations. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 272-278, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
[Source: eCFR, current 15 CFR Part 774 Supplement No. 1, ECCN 1C353, <https://www.ecfr.gov/current/title-15/subtitle-B/chapter-VII/subchapter-C/part-774/appendix-Supplement%20No.%201%20to%20Part%20774>]
- The 2023 HHS guidance recommends that providers and third-party vendors know and document recipients, know whether products contain SOC, notify customers/principal users when orders contain SOC, protect customer identity and intellectual property, avoid fulfilling unresolved suspicious orders, report unresolved concerns to the FBI, and archive

SOC-order information for at least three years. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 503-525, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]

- The 2023 HHS guidance recommends that customers, principal users, and end users who know an order contains SOC's preemptively provide legitimacy information, transfer SOC-containing synthetic nucleic acids only to verified legitimate recipients, record transfers, communicate with biosafety officers or equivalents, and retain SOC-transfer records for at least three years. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 526-553, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]
- The IGSC Harmonized Screening Protocol v3.0 describes a voluntary industry protocol in which IGSC provider and manufacturer members screen ordered nucleic acid sequences and vet customers to reduce misuse risk, and IGSC states that its members represent a majority of global commercial gene-length nucleic acid synthesis capacity. [Source: IGSC, 2024, Harmonized Screening Protocol v3.0, lines 0-13, <https://genesynthesisconsortium.org/wp-content/uploads/IGSC-Harmonized-Screening-Protocol-v3.0-1.pdf>]
- The IBBIS Common Mechanism is a free, open-source screening tool intended to help providers screen DNA and RNA orders while supporting both sequence-risk recognition and customer-legitimacy review. [Source: IBBIS, Common Mechanism, lines 28-35 and 67-72, <https://ibbis.bio/our-work/common-mechanism/>]
- IBBIS states that the Common Mechanism screens nucleic acid sequences of 50 base pairs or more with biorisk search, taxonomy search, and low-concern search steps, but it does not directly screen protein sequences and has limitations for sequences below 50 bp, ambiguous codons, and specialized oligo-order handling. [Source: IBBIS, Common Mechanism FAQ, lines 78-95, <https://ibbis.bio/our-work/common-mechanism/faq/>]
- NIST frames AI-enabled nucleic acid design as an emerging screening problem because AI may generate novel DNA sequences that current screening tools do not detect, and NIST is working with stakeholders on standards, benchmark datasets, and risk-mitigation practices. [Source: NIST, 2025 update, Biosecurity for Synthetic Nucleic Acid Sequences, lines 150-184, <https://www.nist.gov/programs-projects/biosecurity-synthetic-nucleic-acid-sequences>]
- A NIST-hosted publication reports that, on a NIST test dataset, six screening tool developers showed greater than 95% sensitivity and 97% accuracy overall, while disagreements were attributed to differences in SOC definitions and screening algorithms. [Source: NIST, 2025, Inter-tool analysis of a NIST dataset for assessing baseline nucleic acid sequence screening, lines 147-151, <https://www.nist.gov/publications/inter-tool-analysis-nist-dataset-assessing-baseline-nucleic-acid-sequence-screening>]
- A 2025 Nature Biotechnology correspondence hosted by NIST argues that generative AI tools for protein engineering, genome editing, and molecular synthesis can be misused to enhance viral virulence, design toxins, or modify embryos, and calls for built-in safeguards such as watermarking, alignment, anti-jailbreak methods, and unlearning to complement governance. [Source: Wang et al., 2025, A Call for Built-In Biosecurity Safeguards for Generative AI Tools, Nature Biotechnology, <https://doi.org/10.1038/s41587-025-02650-8>]

- The 2024 U.S. Framework explicitly warns that SOC databases and databases of pathogenicity/toxicity sequences can themselves pose biosecurity risks and recommends cybersecurity safeguards for SOC databases, customer identity, proprietary information, supply chains, and benchtop synthesis equipment. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 438-461, <https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid-Synthesis-Screening-Framework-Sep2024-Final.pdf>]

11.0.3 What this means for our build

- Treat every design request and every generated/exported sequence as subject to biosecurity pre-screening before synthesis-ready export because the product intentionally produces orderable nucleic-acid designs and the authoritative frameworks use procurement/orderability as the risk control point. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 106-112 and 121-138, <https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid-Synthesis-Screening-Framework-Sep2024-Final.pdf>]
- Add a deterministic `BiosecurityScreening` interface before final export and synthesis handoff, with a fake implementation for tests and a swappable production implementation that can call an approved screening engine, because Section 4.3 of the system design requires model/infrastructure dependencies behind interfaces and the U.S. Framework allows commercial, open-source, or in-house screening systems when adequate standards are met. [Source: `SYSTEM_DESIGN.md`, Section 4.3, interface/fake requirement] [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 297-301 and 322-325, <https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid-Synthesis-Screening-Framework-Sep2024-Final.pdf>]
- Screen both user prompts and generated DNA, but treat prompt screening as a triage layer rather than a substitute for sequence screening, because the frameworks define screening around ordered synthetic nucleic acid sequences and customer/order legitimacy. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 299-328 and 503-525, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]
- Do not automatically export or hand off any design with an SOC flag; require human review, customer/end-user legitimacy checks, audit logging, and provider-side screening before fulfillment, because HHS and OSTP guidance require follow-up legitimacy review and non-fulfillment/reporting when concerns remain unresolved. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 312-328 and 517-525, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>] [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 366-382 and 408-424, <https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid-Synthesis-Screening-Framework-Sep2024-Final.pdf>]
- Persist enough screening metadata to support auditability without storing unnecessary hazardous details in ordinary application logs, because the 2024 U.S. Framework recommends at least three years of screening records while also warning that SOC databases and customer/proprietary data require cybersecurity and information-security safeguards. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 425-449, <https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid-Synthesis-Screening-Framework-Sep2024-Final.pdf>]

- Keep generated sequence exports unavailable for unauthenticated users and for users who have not provided verified affiliation/use-context fields, because customer legitimacy assessment in the U.S. Framework includes identity, institution, legitimate need, end-user status, and red-flag review. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 358-382 and 389-397, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Prefer an integration path with synthesis providers that already attest to the current U.S. Framework or equivalent standards, because federally funded procurement is tied to adherent providers/manufacturers and provider-side screening remains the established order-fulfillment control. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 121-144 and 253-271, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Use IBBIS Common Mechanism or an equivalent screened-provider API as the first production candidate for baseline local screening, but treat its documented limitations as product requirements for fallback review rather than as clearance, because IBBIS describes known limitations for short sequences, proteins, specialized oligo orders, and ambiguous codons. [Source: IBBIS, Common Mechanism FAQ, lines 78-105, <https://ibbis.bio/our-work/common-mechanism/faq/>]
- Add explicit security controls for screening artifacts, hazardous-match details, provider attestations, customer identity, and export/handoff events, because OSTP identifies SOC databases, customer identities, proprietary sequence information, and synthesis supply chains as information-security concerns. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 438-461, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Document that the system’s compliance output is advisory and not a substitute for institutional biosafety, export-control, select-agent, or provider compliance review, because HHS guidance positions institutional policies, BMBL, WHO guidance, and existing regulations as complementary controls. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 329-344, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]

11.0.4 Decisions proposed

- Implement a mandatory biosecurity gate with states `CLEAR`, `REVIEW_REQUIRED`, `BLOCKED`, and `SCREENING_UNAVAILABLE`; no synthesis-ready export or order handoff should proceed unless the state is `CLEAR` and the provider-side screening path is recorded. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 312-328 and 503-525, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]
- Define a `BiosecurityScreeningResult` schema with fields for screening engine/version, screening-window policy, sequence hash, screening timestamp, matched policy category, decision, reviewer status, provider attestation reference, and retention class; do not store raw hazardous-match alignments in general application tables or logs.

[Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 425-449, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]

- Require the production screening adapter to support current 200-nt window screening immediately and be configurable for 50-nt windows and multi-fragment assembly detection before October 13, 2026. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 297-320, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Screen sequence translations in all applicable reading frames when a generated nucleic acid may encode protein, because the 2024 U.S. Framework explicitly calls for translation-aware screening across applicable reading frames. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 279-281 and 311-316, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Add an abuse-prevention layer for prompts and iterative refinements that flags requests involving pathogens, toxins, virulence, immune evasion, host-range expansion, synthesis-screening evasion, or instructions to bypass provider review; route such sessions to refusal or human review before generation. [Source: NIST, 2025 update, Biosecurity for Synthetic Nucleic Acid Sequences, lines 150-161, <https://www.nist.gov/programs-projects/biosecurity-synthetic-nucleic-acid-sequences>] [Source: Wang et al., 2025, A Call for Built-In Biosecurity Safeguards for Generative AI Tools, Nature Biotechnology, <https://doi.org/10.1038/s41587-025-02650-8>]
- Restrict synthesis handoff to providers that publish or provide a current attestation to the applicable U.S. Framework or equivalent jurisdictional screening standard, and store the provider attestation reference with the handoff record. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 253-271, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Separate ordinary validation failures from biosecurity decisions in the API and UI, because a biologically valid construct can still require biosecurity review and a biosecurity flag should not be exposed with operational match detail. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 438-449, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Treat therapeutic, gene-therapy, pathogen, toxin, and select-agent-adjacent requests as requiring institutional-review metadata before export, because HHS guidance recommends using institutional biosafety/biosecurity practices and principal/end-user responsibility for possession, use, and transfer of SOC-containing materials. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 329-344 and 526-553, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>]

11.0.5 Open questions / conflicts

- The May 5, 2025 Executive Order required OSTP to revise or replace the 2024 Framework within 90 days, but this pass found the directive and implementation references,

not a clearly identified replacement framework; before implementation, counsel or the human should verify the currently controlling U.S. framework and agency-specific procurement terms. [Source: White House, 2025, Improving the Safety and Security of Biological Research, lines 162-166, <https://www.whitehouse.gov/presidential-actions/2025/05/improving-the-safety-and-security-of-biological-research/>]

- The product is a design tool rather than a synthesis provider, third-party vendor, or benchtop synthesizer manufacturer; the strongest direct duties in HHS/OSTP guidance attach to procurement, synthesis, transfer, and equipment access, so the human should decide whether to operate the platform as a regulated-like checkpoint even before provider handoff. [Source: HHS ASPR, 2023, Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids, lines 351-376 and 476-486, <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>] [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 187-196 and 219-226, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]
- Screening tools vary in SOC definitions and algorithms, and NIST reported disagreements on specific sequences even when overall sensitivity and accuracy were high; product requirements need a defined review path for borderline or conflicting screening results. [Source: NIST, 2025, Inter-tool analysis of a NIST dataset for assessing baseline nucleic acid sequence screening, lines 147-151, <https://www.nist.gov/publications/inter-tool-analysis-nist-dataset-assessing-baseline-nucleic-acid-sequence-screening>]
- IBBIS Common Mechanism is attractive as a baseline implementation, but it documents limits for sequences below 50 bp, direct protein screening, specialized oligo orders, and ambiguous codons; those gaps need either provider-side handling, another screening engine, or a manual-review rule. [Source: IBBIS, Common Mechanism FAQ, lines 88-105, <https://ibbis.bio/our-work/common-mechanism/faq/>]
- International launch will require jurisdiction-specific review because the U.S. Framework, UK guidance, IGSC protocol, ISO standards, export controls, and provider policies are overlapping but not identical. [Source: IBBIS, Common Mechanism FAQ, lines 135-137, <https://ibbis.bio/our-work/common-mechanism/faq/>] [Source: GOV.UK, 2024, UK screening guidance on synthetic nucleic acids, lines 77-99, <https://www.gov.uk/government/publications/uk-screening-guidance-on-synthetic-nucleic-acids>]
- The platform should avoid displaying hazardous match details to ordinary users, but reviewers need enough information to make auditable decisions; the exact data-minimization boundary should be set with biosafety counsel and security review. [Source: OSTP, 2024, Framework for Nucleic Acid Synthesis Screening, lines 438-461, https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf]

11.0.6 Sources

1. Office of Science and Technology Policy. 2024. "Framework for Nucleic Acid Synthesis Screening." https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf

2. The White House. 2025. "Improving the Safety and Security of Biological Research." <https://www.whitehouse.gov/presidential-actions/2025/05/improving-the-safety-and-security-of-biological-research/>
3. HHS ASPR. 2023. "Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids." Federal Register notice mirrored at <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html>
4. Federal Select Agent Program. 2025. "Select Agents and Toxins List." <https://www.selectagents.gov/sat/list.htm>
5. Electronic Code of Federal Regulations. Current. "15 CFR Part 774, Supplement No. 1 to Part 774 - The Commerce Control List." <https://www.ecfr.gov/current/title-15/subtitle-B/chapter-VII/subchapter-C/part-774/appendix-Supplement%20No.%201%20to%20Part%20774>
6. International Gene Synthesis Consortium. 2024. "Harmonized Screening Protocol v3.0." <https://genesynthesisconsortium.org/wp-content/uploads/IGSC-Harmonized-Screening-Protocol-v3.0-1.pdf>
7. International Biosecurity and Biosafety Initiative for Science. Current. "Common Mechanism." <https://ibbis.bio/our-work/common-mechanism/>
8. International Biosecurity and Biosafety Initiative for Science. Current. "Common Mechanism FAQ." <https://ibbis.bio/our-work/common-mechanism/faq/>
9. National Institute of Standards and Technology. 2025 update. "Biosecurity for Synthetic Nucleic Acid Sequences." <https://www.nist.gov/programs-projects/biosecurity-synthetic-nucleic-acid-sequences>
10. Laird, T., Flyangolts, K., Bartling, C., Gemler, B., Beal, J., Mitchell, T., Murphy, S. T., Berlips, J., Foner, L., Doughty, R., Quintana, F., Nute, M., Treangen, T. J., Godbold, G. D., Ternus, K., Alexanian, T., Wheeler, N., and Forry, S. 2025. "Inter-tool analysis of a NIST dataset for assessing baseline nucleic acid sequence screening." NIST publication page: <https://www.nist.gov/publications/inter-tool-analysis-nist-dataset-assessing-baseline-nucleic-acid-sequence-screening>
11. Wang, M., Zhang, Z., Bedi, A. S., Velasquez, A., Guerra, S., Lin-Gibson, S., Cong, L., Blewett, M., Qu, Y., Ma, J., Xing, E., Church, G., and Chakraborty, S. 2025. "A Call for Built-In Biosecurity Safeguards for Generative AI Tools." Nature Biotechnology. <https://doi.org/10.1038/s41587-025-02650-8>
12. Department for Science, Innovation and Technology. 2024. "UK screening guidance on synthetic nucleic acids." <https://www.gov.uk/government/publications/uk-screening-guidance-on-synthetic-nucleic-acids>

Chapter 12

Annotated Bibliography

This bibliography is organized by Phase R track. Each source listed here was cited in the corresponding findings file and includes the takeaway that affects the build.

12.0.1 Track A — Plasmid Biology & Structure

1. Addgene. 2025. "Molecular Biology Reference." <https://www.addgene.org/mol-bio-reference/> — Practical summary of common plasmid features used to seed canonical component types.
2. Bolivar et al. 1977. "Construction and characterization of new cloning vehicles. II. A multi-purpose cloning system." DOI: [10.1016/0378-1119\(77\)90000-2](https://doi.org/10.1016/0378-1119(77)90000-2) — Classic pBR322 example showing unique sites, markers, and backbone design logic.
3. Brown, T. A. 2002. "Studying DNA." NCBI Bookshelf. <https://www.ncbi.nlm.nih.gov/books/NBK21129/> — Establishes plasmids as replicating vectors and explains origins and selectable markers.
4. del Solar et al. 1998. "Replication and Control of Circular Bacterial Plasmids." DOI: [10.1128/MMBR.62.2.434-464.1998](https://doi.org/10.1128/MMBR.62.2.434-464.1998) — Supports host/ORI/replication-control specificity.
5. Wang et al. 2009. "Classification of plasmid vectors using replication origin, selection marker and promoter as criteria." DOI: [10.1016/j.plasmid.2008.09.003](https://doi.org/10.1016/j.plasmid.2008.09.003) — Supports treating ORI, marker, and promoter as first-class structured fields.
6. Molloy et al. 2004. "Effective and robust plasmid topology analysis..." DOI: [10.1093/nar/gnh124](https://doi.org/10.1093/nar/gnh124) — Shows topology and isoforms matter for plasmid QC.
7. Morgan/Addgene. 2014. "Plasmids 101: The Promoter Region - Let's Go!" <https://blog.addgene.org/plasmids-101-the-promoter-region> — Practical promoter/host/transcript compatibility guidance.
8. Addgene. 2016. "Plasmids 101: Gateway Cloning." <https://blog.addgene.org/plasmids-101-gateway-cloning> — Reinforces reading-frame and fusion-tag constraints.
9. Addgene. 2023. "Viral Vectors 101: Viral Vector Elements." <https://blog.addgene.org/viral-vectors-101-viral-vector-elements> — Establishes viral vector designs as vector-type-specific.

10. Higgins and Vologodskii. 2015. "Topological Behavior of Plasmid DNA." DOI: [10.1128/microbiolspec.PLAS-0036-2014](https://doi.org/10.1128/microbiolspec.PLAS-0036-2014) — Supports explicit topology representation.

12.0.2 Track B — Design & Validity Rules

1. Addgene. 2014. "Plasmids 101: The Promoter Region - Let's Go!" <https://blog.addgene.org/plasmids-101-the-promoter-region> — Host and transcript class must drive promoter validation.
2. Addgene. 2014. "Plasmids 101: Terminators and PolyA signals." <https://blog.addgene.org/plasmids-101-terminators-and-polya-signals> — Terminators/polyA elements are required downstream cassette components.
3. Chen et al. 1994. "Determination of the optimal aligned spacing between the Shine-Dalgarno sequence and the translation initiation codon..." <https://academic.oup.com/nar/article/22/23/4953/2400328> — Bacterial translation initiation should check RBS/start spacing.
4. Engler et al. 2008. "A one pot, one step, precision cloning method with high throughput capability." <https://pubmed.ncbi.nlm.nih.gov/18985154/> — Type IIS assembly requires strategy-specific restriction-site checks.
5. GenScript. 2026. "FLASH Gene & Gene Synthesis." https://www.genscript.com/gene_synthesis.html — Provider-specific synthesis constraints differ from Twist/IDT.
6. GenScript. 2026. "Reasons Cloning Fails - and Solutions." https://www.genscript.com/reasons_cloning_fails_solutions.html — Repeats and cloning design errors are common failure modes.
7. Haellman and Piras. 2023. "The sound of silence: transgene silencing in mammalian cell engineering." <https://pmc.ncbi.nlm.nih.gov/articles/PMC9880859/> — Promoter behavior is context-dependent and can warrant WARN rather than PASS/FAIL.
8. Hawley and McClure. 1983. "Compilation and analysis of Escherichia coli promoter DNA sequences." <https://pmc.ncbi.nlm.nih.gov/articles/PMC340638/> — E. coli promoter spacing is codifiable.
9. IDT. 2026. "What types of sequence motifs should be avoided when ordering gBlocks Gene Fragments?" <https://www3.idtdna.com/pages/support/faqs/what-types-of-sequence-motifs-should-be-avoided-when-ordering-gblocks-gene-fragments-> — gBlocks has strict motif/GC/homopolymer constraints.
10. Khan. 2013. "Gene Expression in Mammalian Cells and its Applications." <https://pmc.ncbi.nlm.nih.gov/articles/PMC7147855/> — Mammalian expression vectors require coordinated regulatory, translation, propagation, and selection elements.
11. Kozak. 1987. "At least six nucleotides preceding the AUG initiator codon enhance translation in mammalian cells." <https://pubmed.ncbi.nlm.nih.gov/3681984/> — Mammalian coding designs should score Kozak/start context.
12. Ling et al. 2024. "Degradation and stable maintenance of adeno-associated virus inverted terminal repeats in E. coli." <https://pmc.ncbi.nlm.nih.gov/articles/PMC11754738/> — AAV ITRs require vector-specific instability handling.

13. Mauro and Chappell. 2014. "A critical analysis of codon optimization in human therapeutics." DOI: [10.1016/j.molmed.2014.09.003](https://doi.org/10.1016/j.molmed.2014.09.003) — Codon optimization can affect biology and should not be automatic by default.
14. Nakagawa et al. 2008. "Diversity of preferred nucleotide sequences around the translation initiation codon..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC2241899/> — Start-context preferences vary across eukaryotes.
15. Nakamura et al. 2000. "Codon usage tabulated from international DNA sequence databases..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC102460/> — Kazusa provides codon usage reference data.
16. NEB. 2026. "Restriction Enzyme Digestion." <https://www.neb.com/en/applications/cloning-and-synthetic-biology/dna-preparation/restriction-enzyme-digestion> — Restriction cloning needs no internal selected enzyme sites and proper flanking bases.
17. Parret et al. 2016. "Codon usage bias." <https://pmc.ncbi.nlm.nih.gov/articles/PMC8613526/> — Gene design is multi-constraint, not just CAI maximization.
18. Powell et al. 2015. "Viral Expression Cassette Elements..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC4505817/> — Viral vectors have cassette and packaging constraints.
19. Santos-Zavaleta et al. 2024. "RegulonDB v12.0..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC10767902/> — Curated E. coli regulatory reference.
20. Sarrion-Perdigones et al. 2011. "GoldenBraid..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC3131274/> — Type IIS systems require part domestication.
21. Sharp and Li. 1987. "The codon adaptation index..." DOI: [10.1093/nar/15.3.1281](https://doi.org/10.1093/nar/15.3.1281) — CAI is the baseline codon-adaptation scoring method.
22. Twist Bioscience. 2023. "Twist Tips: How to Design Your Gene." https://www.twistbioscience.com/content/dam/twistbioscience/resources/2023-06/DOC-001081_TechNote_TwistTipVectorDesign-REV4-singles.pdf — Provides concrete synthesis-readiness heuristics.
23. Twist Bioscience. 2026. "High Quality Gene Synthesis." <https://www.twistbioscience.com/products/genes/gene-synthesis?tab=fragment> — Confirms provider scoring factors for gene synthesis.
24. Yau et al. 2013. "Remarkable stability of an instability-prone lentiviral vector plasmid in Escherichia coli Stbl3." <https://pmc.ncbi.nlm.nih.gov/articles/PMC3563744/> — Lentiviral plasmids need propagation-instability rules.
25. Zhao et al. 1999. "Formation of mRNA 3' Ends in Eukaryotes..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC98971/> — Polyadenylation signals have concrete sequence architecture.

12.0.3 Track C — DNA Language Models & Generation

1. Ji et al. 2021. "DNABERT..." <https://academic.oup.com/bioinformatics/article/37/15/2112/6128680> — Encoder model useful for classification/representation, not full plasmid generation.

2. NVIDIA BioNeMo. "DNABERT." <https://docs.nvidia.com/bionemo-framework/1.10/models/dnabert.html> — Creates a license-signal conflict for DNABERT commercial use.
3. Zhou et al. 2023/2024. "DNABERT-2..." <https://arxiv.org/abs/2306.15006> — Efficient BPE genome encoder, likely useful for embeddings/scoring.
4. MAGICS-LAB. "DNABERT_2." https://github.com/MAGICS-LAB/DNABERT_2 — Repository license appears permissive, but weight license still needs verification.
5. zhihan1996. "DNABERT-2-117M." <https://huggingface.co/zhihan1996/DNABERT-2-117M> — Model-card behavior focuses on embeddings.
6. Dalla-Torre et al. 2024/2025. "Nucleotide Transformer..." <https://www.nature.com/articles/s41592-024-02523-z> — Strong encoder family but license constraints limit commercial use.
7. InstaDeepAI. "nucleotide-transformer-2.5b-1000g." <https://huggingface.co/InstaDeepAI/nucleotide-transformer-2.5b-1000g> — Non-commercial model-card license blocks production without permission.
8. Nguyen et al. 2024. "Sequence modeling and design from molecular to genome scale with Evo." <https://github.com/evo-design/evo> — Autoregressive DNA generator with long context.
9. Arc Institute. 2024. "Evo: DNA foundation modeling..." <https://arcinstitute.org/news/blog/evo> — Context for Evo capabilities.
10. togethercomputer. "evo-1-131k-base." <https://huggingface.co/togethercomputer/evo-1-131k-base/tree/main> — Evo 1 weight availability and license context.
11. ArcInstitute. "evo2." <https://github.com/ArcInstitute/evo2> — Strong first candidate for whole-plasmid generation experiments.
12. arcinstitute. "evo2_7b_base." https://huggingface.co/arcinstitute/evo2_7b_base — Candidate open checkpoint.
13. NVIDIA BioNeMo. "Evo2." <https://docs.nvidia.com/bionemo-framework/2.6/models/evo2/index.html> — Hosted deployment has additional NVIDIA terms.
14. Zvyagin et al. 2022. "GenSLMs..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC9709791/> — GenSLM is more relevant to viral/prokaryotic sequence modeling than plasmid design.
15. ramanathanlab. "genslm." <https://github.com/ramanathanlab/genslm> — Code license and paper license must be separated.
16. Nguyen et al. 2023. "HyenaDNA..." <https://arxiv.org/abs/2306.15794> — Long-context single-nucleotide model useful for representation comparisons.
17. HazyResearch. "hyena-dna." <https://github.com/HazyResearch/hyena-dna> — Apache-2.0 code/weights but not a primary plasmid generator.
18. Schiff et al. 2024. "Caduceus..." <https://arxiv.org/abs/2403.03234> — Reverse-complement equivariant model relevant for embeddings/scoring.

19. kuleshov-group. "caduceus..." https://huggingface.co/kuleshov-group/caduceus-ps_seqlen-131k_d_model-256_n_layer-16/blob/main/README.md — Released Caduceus checkpoint context.
20. Helical. "Caduceus model card." https://helical.readthedocs.io/en/stable/model_cards/caduceus/ — License and model-card context.
21. InstaDeepAI. "nucleotide-transformer." <https://github.com/instadeepai/nucleotide-transformer> — NTV3 may be relevant but needs license verification.
22. Wu et al. 2025. "GENERator..." <https://arxiv.org/abs/2502.07272> — Emerging long-context genomic generator to monitor.
23. Zhou et al. 2025. "GenomeOcean..." <https://pubmed.ncbi.nlm.nih.gov/39975405/> — Emerging metagenomic generator with unclear production license.
24. HuggingFaceBio. "Carbon-3B." <https://huggingface.co/HuggingFaceBio/Carbon-3B> — Lower-cost Apache-2.0 generative baseline candidate.
25. HuggingFaceBio. "carbon-pretraining-corpus." <https://huggingface.co/datasets/HuggingFaceBio/carbon-pretraining-corpus> — Training corpus license context for Carbon.

12.0.4 Track D — Prior Art & Related Tools

1. Irvine et al. 2025. "Generating functional plasmid origins with OriGen." DOI: [10.1093/nar/gkaf1198](https://doi.org/10.1093/nar/gkaf1198) — Functional origin generation is proven for a narrow scope, not full plasmids.
2. Shao. 2024. "PlasmidGPT." <https://github.com/lingxusb/PlasmidGPT> — Plasmid-like sequence generation prior art, with validation/licensing caveats.
3. Shao. 2024. "PlasmidGPT" bioRxiv. DOI: [10.1101/2024.09.30.615762](https://doi.org/10.1101/2024.09.30.615762) — Preprint context for PlasmidGPT.
4. UCL-CSSB. 2026. "PlasmidGPT compatible version." <https://huggingface.co/UCL-CSSB/PlasmidGPT> — Architecture/context for model compatibility.
5. lingxusb/PlasmidGPT. 2026. <https://huggingface.co/lingxusb/PlasmidGPT> — Conflicting non-commercial model license signal.
6. Enghiad et al. 2022. "PlasmidMaker..." DOI: [10.1038/s41467-022-30355-y](https://doi.org/10.1038/s41467-022-30355-y) — Automated construction platform, not NL-to-design.
7. Cai. 2010. "GenoCAD..." <https://vtechworks.lib.vt.edu/handle/10919/27069> — Formal grammars are useful for construct constraints.
8. McGuffie and Barrick. 2021. "pLannotate..." DOI: [10.1093/nar/gkab374](https://doi.org/10.1093/nar/gkab374) — Strong precedent for engineered plasmid annotation.
9. Wishart Lab. 2023. "PlasMapper 3.0 help." <https://plasmapper.wishartlab.com/help/> — Visualization/annotation/export expectations.
10. Benchling. 2026. "Molecular Bio Software." <https://www.benchling.com/molecular-biology> — Enterprise CAD/registry feature baseline.

11. SnapGene. 2026. "Discover SnapGene." <https://www.snapgene.com/series/snapgene-tour> — Human-operated cloning simulation and map expectations.
12. SnapGene Support. 2025. "Simulate Gateway Cloning with Multiple Inserts." <https://support.snapgene.com/hc/en-us/articles/10384092210708-Simulate-Gateway-Cloning-with-Multiple-Inserts> — Method-specific cloning simulation precedent.
13. VectorBuilder. 2026. "Online Vector Design." <https://en.vectorbuilder.com/products-services/service/online-vector-design.html/1000> — Strong component-driven design/order workflow reference.
14. Asimov. 2026. "Welcome to Kernel." <https://docs.kernel.asimov.com/> — Product reference for model-guided construct design.
15. Asimov. 2026. "Compiler." <https://docs.kernel.asimov.com/compiler-and-simulation/compiler> — Closest compiler-style plasmid design precedent.
16. Asimov. 2026. "Kernel." <https://www.asimov.com/kernel> — Commercial positioning and capability context.
17. Addgene. 2026. "Browse Genes / Search by Sequence." <https://www.addgene.org/browse/gene/> — Retrieval source and user expectation baseline.
18. Addgene Help Center. 2026. "How does Addgene create plasmid maps?" <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps> — Sequence provenance and map-generation caveats.

12.0.5 Track E — Data Sources & Access

1. Addgene. 2026. "Access Plasmid and Sequence Data via API." <https://developers.addgene.org/> — Programmatic Addgene data requires approved access.
2. Addgene Developers Portal. 2026. "Access Options." <https://developers.addgene.org/access-options/> — Bulk JSON and API versioning/access details.
3. Addgene Developers API. 2026. "Addgene Developers API." <https://docs.developers.addgene.org/docs/> — Endpoint and field contract for ingestion.
4. Addgene Help Center. 2026. "What is the Developers Portal?" <https://help.addgene.org/hc/en-us/articles/38241923181453-What-is-the-Developers-Portal> — Access approval and license workflow.
5. Addgene Help Center. 2026. "Requirements for viewing plasmid sequence data." <https://help.addgene.org/hc/en-us/articles/44210206209549-Are-there-any-requirements-for-viewing-plasmid-sequence-data-on-the-Addgene> — Sequence access is controlled.
6. Addgene Help Center. 2026. "Do you have full sequence for my plasmid?" <https://help.addgene.org/hc/en-us/articles/>

[205434259-Do-you-have-full-sequence-for-my-plasmid](#) — Sequence completeness varies.

7. Addgene Help Center. 2026. "How does Addgene create plasmid maps?" <https://help.addgene.org/hc/en-us/articles/115005662726-How-does-Addgene-create-plasmid-maps> — Sequence provenance categories.
8. Addgene. 2023. "Terms of Use." <https://www.addgene.org/terms-of-use/> — Commercial/data-use restrictions require review.
9. Sayers. 2009/2022. "A General Introduction to the E-utilities." <https://www.ncbi.nlm.nih.gov/sites/books/NBK25497/> — NCBI access, throttling, and client identification rules.
10. Sayers. 2009/2022. "The E-utilities In-Depth." <https://www.ncbi.nlm.nih.gov/books/NBK25499/> — EFetch formats and retrieval details.
11. NCBI. 2026. "GenBank Overview." <https://www.ncbi.nlm.nih.gov/genbank/genbank/> — GenBank scope, releases, and use caveats.
12. NCBI. 2017. "FTP access to GenBank data." <https://www.ncbi.nlm.nih.gov/genbank/ftp/> — Bulk data access route.
13. NCBI. 2025. "FASTA Format for Nucleotide Sequences." <https://www.ncbi.nlm.nih.gov/genbank/fastafmt/> — FASTA import/export rules.
14. DDBJ/ENA/GenBank. 2026. "Feature Table Definition." <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html> — Canonical feature vocabulary and location syntax.
15. NCBI Datasets. 2026. "datasets download genome reference." <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/command-line/datasets/download/genome/> — Useful but not a replacement for Entrez plasmid record retrieval.

12.0.6 Track F — Sequence Representation & Tokenization

1. Ji et al. 2021. "DNABERT." DOI: [10.1093/bioinformatics/btab083](https://doi.org/10.1093/bioinformatics/btab083) — Overlapping k-mer tokenization baseline.
2. Zhou et al. 2024. "DNABERT-2." <https://arxiv.org/abs/2306.15006> — BPE tokenization motivation and efficiency tradeoffs.
3. Dalla-Torre et al. 2024. "Nucleotide Transformer." DOI: [10.1038/s41592-024-02523-z](https://doi.org/10.1038/s41592-024-02523-z) — 6-mer encoder representation baseline.
4. Nguyen et al. 2023. "HyenaDNA." DOI: [10.48550/arXiv.2306.15794](https://doi.org/10.48550/arXiv.2306.15794) — Single-nucleotide long-context representation baseline.
5. Nguyen et al. 2024. "Evo." DOI: [10.1126/science.ado9336](https://doi.org/10.1126/science.ado9336) — Byte/single-nucleotide generative sequence modeling.
6. Sanabria et al. 2024. "GROVER..." DOI: [10.1038/s42256-024-00872-0](https://doi.org/10.1038/s42256-024-00872-0) — BPE over genome sequence context.

7. Huang et al. 2026. "EvoLen..." DOI: [10.48550/arXiv.2604.08698](https://doi.org/10.48550/arXiv.2604.08698) — Emerging biology-guided tokenization idea to monitor.
8. Niktab and Patel. 2026. "DNATokenizer..." DOI: [10.48550/arXiv.2601.05531](https://doi.org/10.48550/arXiv.2601.05531) — Tokenization throughput can be a production bottleneck.
9. DDBJ/ENA/GenBank. 2026. "Feature Table Definition." <https://www.ddbj.nig.ac.jp/ddbj/feature-table.html> — Feature coordinate and qualifier standard.
10. Biopython. 2026. "Bio.SeqFeature module documentation." <https://biopython.org/docs/1.76/api/Bio.SeqFeature.html> — Python coordinate conversion and compound-feature behavior.
11. NCBI. 2026. "GFF3 format." <https://www.ncbi.nlm.nih.gov/datasets/docs/v2/reference-docs/file-formats/annotation-files/about-ncbi-gff3/> — GFF3 coordinate/origin-spanning behavior.
12. Addgene. 2026. "Plasmids 101 topic overview." <https://info.addgene.org/plasmids-101-topic-page> — Plasmids are commonly circular molecules.
13. Benchling. 2025. "Creating consensus and template alignments..." <https://help.benchling.com/hc/en-us/articles/9684273213325-Creating-consensus-and-template-alignments-on-DNA-RNA-sequences> — Circular/cross-origin handling matters in practice.
14. Zhou, Shrikumar, and Kundaje. 2022. "Reverse-Complement Equivariance..." <https://proceedings.mlr.press/v165/zhou22a.html> — Reverse-complement behavior must be evaluated.
15. Schiff et al. 2024. "Caduceus..." <https://arxiv.org/abs/2403.03234> — Architecture-level reverse-complement equivariance reference.

12.0.7 Track G — Validation Tooling

1. Roberts et al. 2023. "REBASE..." DOI: [10.1093/nar/gkac975](https://doi.org/10.1093/nar/gkac975) — Restriction-enzyme meta-data source.
2. Biopython contributors. 2026. "Bio.Restriction package." <https://biopython.org/docs/latest/api/Bio.Restriction.html> — Local deterministic restriction analysis implementation.
3. Biopython contributors. 2026. "Bio.SeqUtils.CodonAdaptationIndex." <https://biopython.org/docs/latest/api/Bio.SeqUtils.html> — CAI implementation and codon optimization utility.
4. Sharp and Li. 1987. "The codon adaptation index..." DOI: [10.1093/nar/15.3.1281](https://doi.org/10.1093/nar/15.3.1281) — Codon adaptation scoring basis.
5. Kazusa DNA Research Institute. "Codon Usage Database." <https://www.kazusa.or.jp/codon/> — Legacy codon table reference.
6. Kazusa DNA Research Institute. "Countcodon." <https://www.kazusa.or.jp/codon/countcodon.html> — Spot-check codon counting reference.

7. Athey et al. 2017. "A new and updated resource for codon usage tables." DOI: [10.1186/s12859-017-1793-7](https://doi.org/10.1186/s12859-017-1793-7) — HIVE-CUTs should be preferred where available.
8. EMBOSS. "etandem manual." <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/etandem> — Tandem-repeat detection reference.
9. EMBOSS. "einverted manual." <https://emboss.bioinformatics.nl/cgi-bin/emboss/help/einverted> — Inverted-repeat detection reference.
10. Twist Bioscience. "Express Genes." <https://www.twistbioscience.com/products/genes/express-genes> — Twist synthesis constraints.
11. IDT. "gBlocks and gBlocks HiFi Gene Fragments." <https://www.eu.idtdna.com/pages/products/genes-and-gene-fragments/double-stranded-dna-fragments/gblocks-gene-fragments> — IDT synthesis constraints.
12. GenScript. "FLASH Gene & Gene Synthesis." https://www.genscript.com/gene_synthesis.html — GenScript synthesis constraints.
13. SIB/ExPASy. "EPD." <https://epd.expasy.org/epd/> — Natural eukaryotic promoter reference.
14. Rauluseviciute et al. 2024. "JASPAR 2024." DOI: [10.1093/nar/gkad1059](https://doi.org/10.1093/nar/gkad1059) — TF binding profile database.
15. Santos-Zavaleta et al. 2024. "RegulonDB v12.0." DOI: [10.1093/nar/gkad1072](https://doi.org/10.1093/nar/gkad1072) — E. coli regulation database.
16. Addgene. "Plasmids 101: The Promoter Region." <https://blog.addgene.org/plasmids-101-the-promoter-region> — Practical promoter compatibility rules.
17. Addgene. "Plasmids 101: What is a plasmid?" <https://blog.addgene.org/plasmids-101-what-is-a-plasmid> — Component classes for validation.
18. Sequence Ontology Consortium. "Sequence Ontology." <https://www.sequenceontology.org/> — Controlled vocabulary for feature normalization.
19. SYSTEM_DESIGN.md. Local source-of-truth sections 8, 11, 12, and 14 — Validation report and test contract.

12.0.8 Track H — Visualization

1. Lattice Automation. 2025. "seqviz npm package." <https://www.npmjs.com/package/seqviz> — Default circular/linear plasmid map renderer.
2. Lattice Automation. 2025. "seqviz GitHub README." <https://github.com/Lattice-Automation/seqviz> — Feature, primer, enzyme, and interaction support.
3. Lattice Automation. 2024. "seqparse npm package." <https://www.npmjs.com/package/seqparse> — Optional frontend-side parser.
4. TeselaGen. 2024. "openVectorEditor." <https://github.com/TeselaGen/openVectorEditor> — Future editing surface candidate.

5. Grant and CGView.js team. 2025. "CGView.js documentation." <https://js.cgview.ca/> — Alternative for dense circular genome maps.
6. Grant and CGView.js team. 2025. "CGView.js API documentation." <https://js.cgview.ca/api/index.html> — API/license details.
7. Stothard, Grant, and Van Domselaar. 2019. "Visualizing and comparing circular genomes using the CGView family of tools." <https://pubmed.ncbi.nlm.nih.gov/29939276/> — CGView family background.
8. GMOD/JBrowse. 2026. "JBrowse 2 embedded components." https://jbrowse.org/jb2/docs/embedded_components/ — Future genome-browser component option.
9. GMOD/JBrowse. 2026. "JBrowse 2 FAQ." <https://jbrowse.org/jb2/docs/faq/> — Embedded vs full-app tradeoffs.
10. GMOD/JBrowse. 2025. "@jbrowse/react-circular-genome-view." <https://www.npmjs.com/package/%40jbrowse/react-circular-genome-view> — Circular view package details.
11. Diesh et al. 2023. "JBrowse 2..." <https://pmc.ncbi.nlm.nih.gov/articles/PMC10108523/> — Modular browser reference.
12. IGV Team. 2026. "igv.js documentation." <https://igv.org/doc/igvjs/> — Genome-track viewer alternative.
13. IGV Team. 2026. "igv.js quickstart." <https://igv.org/doc/igvjs/QuickStart/> — Shows track/reference configuration model.
14. Robinson et al. 2023. "igv.js..." <https://academic.oup.com/bioinformatics/article/39/1/btac830/6958554> — Browser-only genome viewer reference.

12.0.9 Track I — System Architecture & ML In Production

1. Lewis et al. 2020. "Retrieval-Augmented Generation..." <https://arxiv.org/abs/2005.11401> — RAG improves grounding/updateability but is not a correctness proof.
2. Cohan et al. 2020. "SPECTER..." <https://aclanthology.org/2020.acl-main.207/> — Domain scientific embeddings matter.
3. Wadden et al. 2020. "Fact or Fiction..." <https://aclanthology.org/2020.emnlp-main.609/> — Scientific claim verification is a retrieval/reasoning task.
4. Wu et al. 2024. "Medical Graph RAG..." <https://arxiv.org/abs/2408.04187> — Structured knowledge can strengthen biomedical RAG.
5. Zhao et al. 2025. "MedRAG..." <https://arxiv.org/abs/2502.04413> — Graph/context retrieval pattern for healthcare.
6. Es et al. 2023/2025. "Ragas..." <https://arxiv.org/abs/2309.15217> — RAG eval should split context relevance, faithfulness, and answer quality.
7. Saad-Falcon et al. 2023/2024. "ARES..." <https://arxiv.org/abs/2311.09476> — Automated RAG eval with small human sets.

8. Kwon et al. 2023. "PagedAttention." <https://arxiv.org/abs/2309.06180> — vLLM serving architecture for efficient long-context generation.
9. Hugging Face. "Text Generation Inference." <https://huggingface.co/docs/text-generation-inference/index> — Production serving reference, now maintenance-mode.
10. Hugging Face. "TGI Quantization." <https://huggingface.co/docs/text-generation-inference/conceptual/quantization> — Quantization tradeoffs and methods.
11. NVIDIA. "Triton Inference Server." <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html> — Heterogeneous model serving option.
12. NVIDIA. "Triton Model Repository." https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/model_repository.html — Versioned model repository pattern.
13. Dettmers et al. 2022. "LLM.int8()." <https://arxiv.org/abs/2208.07339> — Quantization can reduce memory but needs eval.
14. MLflow. "Model Registry Workflows." <https://mlflow.org/docs/latest/ml/model-registry/workflow/> — Registry/versioning pattern.
15. KServe. "Canary Rollout Strategy." <https://kserve.github.io/archive/0.13/model-serving/v1beta1/rollout/canary/> — Canary and rollback serving controls.
16. OWASP. "Top 10 for LLM Applications." <https://owasp.org/www-project-top-10-for-large-language-model-applications/> — LLM application threat model.
17. NIST. 2024. "Generative AI Profile." <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf> — AI risk management reference.

12.0.10 Track J — Biosecurity & Compliance

1. OSTP. 2024. "Framework for Nucleic Acid Synthesis Screening." https://www.whitehouse.gov/wp-content/uploads/2024/10/OSTP-Nucleic-Acid_Synthesis_Screening_Framework-Sep2024-Final.pdf — Current screening framework baseline cited by the research.
2. The White House. 2025. "Improving the Safety and Security of Biological Research." <https://www.whitehouse.gov/presidential-actions/2025/05/improving-the-safety-and-security-of-biological-research/> — Requires checking the post-EO framework status.
3. HHS ASPR. 2023. "Screening Framework Guidance for Providers and Users of Synthetic Nucleic Acids." <https://regulations.justia.com/regulations/fedreg/2023/10/13/2023-22540.html> — Broader DNA/RNA screening guidance and customer duties.
4. Federal Select Agent Program. 2025. "Select Agents and Toxins List." <https://www.selectagents.gov/sat/list.htm> — Select-agent/toxin relevance.

5. eCFR. Current. "15 CFR Part 774, Supplement No. 1." <https://www.ecfr.gov/current/title-15/subtitle-B/chapter-VII/subchapter-C/part-774/appendix-Supplement%20No.%201%20to%20Part%20774> — Export-control sequence relevance.
6. IGSC. 2024. "Harmonized Screening Protocol v3.0." <https://genesynthesisconsortium.org/wp-content/uploads/IGSC-Harmonized-Screening-Protocol-v3.0-1.pdf> — Industry screening protocol.
7. IBBIS. "Common Mechanism." <https://ibbis.bio/our-work/common-mechanism/> — Candidate screening implementation.
8. IBBIS. "Common Mechanism FAQ." <https://ibbis.bio/our-work/common-mechanism/faq/> — Screening limitations that must become product rules.
9. NIST. 2025 update. "Biosecurity for Synthetic Nucleic Acid Sequences." <https://www.nist.gov/programs-projects/biosecurity-synthetic-nucleic-acid-sequences> — AI-enabled design is an emerging screening problem.
10. Laird et al. 2025. "Inter-tool analysis of a NIST dataset..." <https://www.nist.gov/publications/inter-tool-analysis-nist-dataset-assessing-baseline-nucleic-acid-sequence-sc> — Screening tools can disagree despite high aggregate sensitivity.
11. Wang et al. 2025. "A Call for Built-In Biosecurity Safeguards for Generative AI Tools." DOI: [10.1038/s41587-025-02650-8](https://doi.org/10.1038/s41587-025-02650-8) — Generative bio-design tools need built-in safeguards.
12. Department for Science, Innovation and Technology. 2024. "UK screening guidance on synthetic nucleic acids." <https://www.gov.uk/government/publications/uk-screening-guidance-on-synthetic-nucleic-acids> — International screening regimes differ.

Chapter 13

Open Questions for Human Review

The following unresolved items are currently logged in `PROGRESS.md`.

- Which synthesis provider profile should be the default for "synthesis-ready" when no provider is selected: conservative cross-provider, Twist, IDT, GenScript, or user-selected only?
- Should the first product only score codon usage, or may it automatically rewrite coding sequences after explicit user consent?
- For commercial use, can Addgene-derived records be used for model training under the eventual approved data access license, or only for retrieval/display?
- Should GenBank-derived records be treated as commercially trainable by default, or should submitter-IP caveats require legal review before training?
- Should Evo 2 7B be the first generation candidate, with Carbon-3B as fallback, pending infrastructure and license review?
- Should the platform operate as a regulated-like biosecurity checkpoint before provider handoff, even where direct legal duties attach primarily to synthesis providers or procurement?
- What is the current controlling U.S. nucleic-acid screening framework after the May 5, 2025 Executive Order directive to revise or replace the 2024 framework?
- For circular plasmids, where should canonical base 1 be placed in generated designs: source record origin, ORI start, MCS/cloning site, or synthesis-provider convention?
- Should the MVP map be read-only, or should browser-side feature/sequence editing be in scope?
- Which vector types are in the first supported validity profile set: generic mammalian expression, lentiviral, AAV, bacterial expression, CRISPR/shRNA, or another subset?